

THINGS TO REMEMBER

IMP:- IF YOU WANT TO ARRANGE SET OF ELEMENTS THEN USE SORT WITH COMPARATOR TO MAKE IT EASY.

⊙ If (cond 1 & cond 2 & cond 3 ...) {
opⁿ; } [In this code, cond 1 will be checked
and then cond 2 & then cond 3...]

* If there's no permutations left for a sequence then it means no config. can be made now following the given condⁿ.

- vector < ll > v;
- ~~v~~ = vector < ll > (x, y)
 → no. of elements reserved
 → the value each of bucket will contain.
- If you are using unordered-map.
 Beware that for large no. of inputs it will lose its $O(1)$ time complexity and becomes inefficient.
 Thus, we ~~use~~ no. of items are contiguous
 ex:- ~~arr~~ [1, N] then rather use vector and use hashing to store the value at number's index. This will take space but will give $O(1)$ eff.

- STL ~~code~~ like priority queue, sets etc ~~can~~ usually consider first argument in case of pair < int, int > etc for comparison purpose.

- Spiral Array print
- Max length bitonic subarray
- Pigish & Magical Park
- 2D char Array
- Reading a ~~string~~ lists of strings
- Form biggest nos. (Using sort and cmp)

Q. I/P :- Hello I am your friend.
O/P :- Friend your am I Hello

Resizable dynamic array

```
vector < string > v;
string str;
while (cin >> str) {
    v.push_back(str);
}
reverse(v.begin(), v.end());
for (int i = 0; i < v.size(); i++) {
    cout << v[i] << " ";
}
```

when no. of input not given.

Difference b/w integer array & char array

int arr[] = {1, 2, 3};
cout << b << endl; } ⇒ address of first element (default)

char arr[] = {'a', 'b', 'c', 'd'};
cout << a << endl; } ⇒ abcd

Special behaviour (due to ~~character~~ operator "<<" overloading)
when it finds its char → directly print

Problems when we don't use '\0' in the end

```
char a[] = {'a', 'b', 'c', 'd', '\0'};
char s1[] = {'h', 'e', 'l', 'l', 'o'};
cout << s1;
```

o/p

it will print garbage values stored earlier and goes upto when '\0' is encountered.

Reading sentences using cin.get()

cin.get() reads one character at a time. In order to make it read sentences/paras.

Code :- void readline (int a[], int maxlen, char delim = '\n') {

```
int i = 0;
char ch = cin.get();
while (ch != delim) {
    a[i] = ch;
    i++;
    if (i == (maxlen - 1)) {
        break;
    }
    ch = cin.get();
}
a[i] = '\0';
return;
```

```
int main() {
    char a[1000];
    readline(a, 1000, '$');
    cout << a << endl;
    return 0;
}
```

delimiter (stops at '\$')

Input

1) Hello I am V. 2) Hello I am V.

o/p

1) Hello I am V. 2) am

C++ has a ~~built~~ inbuilt function for this

cin.getline();

* we can directly call `cin.getline(a, 1000, '$');` to get sentences / paras. stored in a char array.

maxlen delimiter array

PROBLEM WITH MULTIPLE STRINGS

3

=> \n → null string → we use here cin.get() to store null string

hello world (\n) coding (\n)

we cant cin 3rd string and then use getline

• strlen() → calculates the length of string not '\0'.

* cin.getline() is used to store sentences / paragraphs in character array with '\n' being the delimiter (default). We can also define our own delimiter in third argument.

For ex:-

cin.getline(arr, 1000)	cin.getline(arr, 1000, '\$')
------------------------	------------------------------

I/P → I am cool.
V & not.
O/P → I am cool.

I/P: - I am cool.
V & not: \$
O/P: - I am cool.
V & not.

Diff. in cin & cin.getline

cin >> str → I/O Hello bye

cin.getline(str, 1000) → Hello bye
space

O/P
Hello
Hello bye

Strings

string s1 = "Hello World"; / string s1("Hello World");

string s2 = s1; / string s2(s1);

- s1.empty() → if empty returns True else False
- s1.append("I love C++"); → s1 += ("I love C++");
- ~~cout <<~~ s1.length(); → length of string
- s1.clear(); → make string to empty string.
- Compare two strings

s0 = "Apple"; s1 = "Mango";
cout << s1.compare(s0);
Returns 0 when s1 == s0
> 0 when s1 > s0
< 0 when s0 < s1

Overloaded Operators

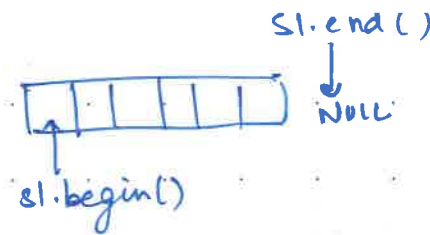
if (s1 > s0) {
cout << "s1 is bigger"; }
@himanshugupta

string s = "I want to have apple juice";
int idx = s.find("apple"); → O/P → 15

⇒ string word = "apple";
int len = word.length();
~~cout <<~~ s.erase(idx, len+1); → I want to have juice.

Iterators

for (auto it = s.begin(); it != s.end(); it++) {
cout << (*it) << " ";
↓
value stored in bucket.
points to beginning bucket
points to position after bucket



* auto → automatically defines the data type required for the variable.
For ex:- auto x = 5; → (int)
auto y = 5.5; → (float)
auto y = "Hello"; → (string)

* String class has function which works as auto for iterating along the string.

string::iterator

Ex:- for (string::iterator it = s.begin(); it != s.end(); it++) { };

Sorting

~~int n;~~ int n;
~~cin >> n;~~ cin >> n;
~~string s[100];~~ string s[100]; // vector
for (int i = 0; i < n; i++) {
get_line(cin, s[i]); }
Sort(s.begin(), s.end());
for (int i = 0; i < n; i++) {
cout << s[i] << endl; }

I/P :- 3
apple juice
mango juice
banana shake
O/P :-
a...
b...
m...

• We can use comparators in third argument of sort()

For ex:- bool compare (string a, string b) {
 if (a.length() == b.length())
 return a < b;
 return a.length() > b.length(); }

Sort(s, s+n, compare);

Tokenization using strtok()

```
int main() {
    char s[100] = "Today is a rainy day";
    char *ptr = strtok(s, " ");
    while (ptr != NULL) {
        cout << ptr << endl;
        ptr = strtok(NULL, " ");
    }
    return 0; }
```

ptr (points to an array which contains first token of string)

on next tokens of same string we take "NULL" at string place

O/P :- Today is a rainy day

Code to apply Tokenization

```
char *mystrtok(char *str, char delim) {
    static char *input = NULL;
    if (str != NULL) {
        input = str;
    }
    if (input == NULL) {
        return NULL;
    }
    // ... (rest of the function code) ...
}
```

// start extracting tokens & store them in a dynamic array (we want the memory to remain after function call terminates).

```
char *output = new char[strlen(input)+1];
int i=0;
```

```
for (; input[i] != '\0'; i++) {
```

```
    if (input[i] != delim) {
        output[i] = input[i];
    }
    else {
```

```
        output[i] = '\0';
        input = input + i + 1;
        return output;
    } }
```

pointing input to next string
 when we do this

// corner case
 output[i] = '\0';
 input = NULL;
 return output;

→ input at starting
 i (current i will be equal to length of first token)
 i (new space)

input + i + 1
 Today is a rainy day

now input points to this part of string and again loop will run until we get next delimiter in loop.

Ex of strtok() [since strtok() accept char-array]

```
string extractStrAtKey(string str, int key){
    char *s = strtok((char*)str.c_str(), " ");
```

↑
Type casting
in char*

↓
changing str
into char
array

```
while (key > 1){
    s = strtok(NULL, " ");
    key--;
}
return (string)s;
```

↑
Type casting
back

```
int main(){
    string s("10 20 30 40");
    int key;
    cin >> key;
    cout << extractStr(s, key);
```

I/P: -
2

O/P
20

Pair in C++ [STL]

Simple containers defined in <utility> header consisting of two data elements or objects.

```
pair<data-type1, data-type2> Pair_name;
```

Ex: - (1) pair<int, char> pair1;

pair1.first = 100;

pair1.second = 'G';

(2) pair<string, string> strPair[100];

(3) pair<int, char> pair3(pair1); we are assigning pair1's values

• Make_pair → pair<string, double> pair1;
pair3 = make_pair("Hello", 4.56);

Swket Strings

I/P: 'ababab'
k = 2

O/P
5

str

Two swaps of b results into max. length string of character 'a' or 'b' (if string consist of 'b' and 'a' find max. length string possible)

Code: -

```
int ans = 0;
int count[] = {0, 0};
int l = 0;
for (int r = 0; r < str.length(); r++) {
    char a = str[r];
    count[a - 'a']++;
    if (min(count[0], count[1]) > k) {
        count[str[r] - 'a']--;
        l++;
    }
    else ans++;
}
cout << ans;
return 0;
```

abababababababbb
ans = 10

ans maintains max. length of string with either 'b' or 'a' being r = k.
if count of 'a' or 'b' > k then we start deducing count from starting.

Array of pairs

```
pair<pair<int, int>, string> car;
```

car.second = "Audi";

car.first.first = 10;

car.first.second = 20;



Staircase search in 2D array

$10 \quad 20 \quad 30 \quad 40 \rightarrow$ start from $(0, n-1)$
 $15 \quad 25 \quad 35 \quad 45$
 $27 \quad 29 \quad 37 \quad 48$
 $32 \quad 33 \quad 39 \quad 50$

$x = 29$. No. to be found

steps
 if $x > arr[0][n-1]$
 $i++;$
 else if $x < ptr$
 $j--;$
 else
 return (i, j)

O/P
 $(2, 1)$
 T.C.
 $O(n+m)$
 Linear

Sum of all submatrix from A given matrix

EX:-

1	1
1	1

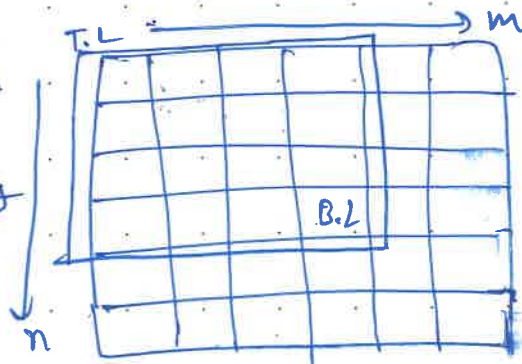
 (2x2)

4 submat $(1 \times 1) \rightarrow 4 \times 1 = 4$
 2 submat $(1 \times 2) \rightarrow 2 \times 2 = 4$
 2 submat $(2 \times 1) \rightarrow 2 \times 2 = 4$
 1 submat $(2 \times 2) \rightarrow 1 \times 4 = 4$

Sum $\leftarrow 16$

Brute force

if we know [Top left coord.] & [Bottom Right Corner]. Then we can specify a submatrix.



for $(li = 0 \rightarrow n-1)$
 for $(lj = 0 \rightarrow m-1)$ } Extract all possible left Top Left
 for $(bi = li+1 \rightarrow n-1)$ } Extract all possible bottom right
 for $(bj = lj+1 \rightarrow m-1)$ } possible submatrices
 for $(i = li \rightarrow bi)$
 for $(j = lj \rightarrow bj)$
 $Sum += arr[i][j];$

$O(n^4)$

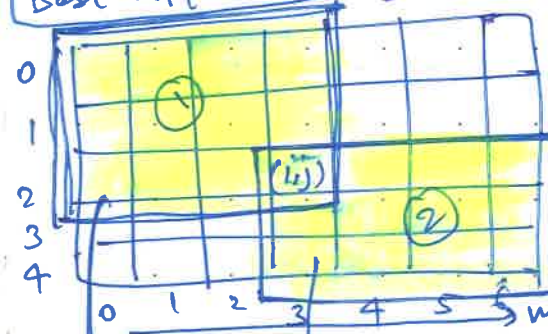
Moderate Approach

what we can do get sum is that we will store prefix sum in each bucket. $O(n^4)$

	0	1	2
0	a	b	c
1	d	e	f
2	g	h	i

	a	a+b	a+b+c
d+a	d+a	d+e+a+b	a+b+c+d+e+f
g+d+a	g+d+a	g+h+d+e+a+b	a+b+c+d+e+f+g+h+i

Best Approach



In this region ① TL will lie for (i, j)
 In this region ② B.R will lie for (i, j)

if we see each element in 2D array we can find no. of submatrices it is part of.

Because each element is part of max than 2 submatrices.

prefix sum matrix
 (can be used in 2D Array & S)

And also each element is contributing for the sum.

* Contribution of each cell/element in my final sum
 $= arr[i][j] \times \text{no. of submatrices this cell is part of}$

$\left\{ \begin{array}{l} \text{no. of cells in region ①} = x \\ \text{no. of cells in region ②} = y \end{array} \right.$

To define a submatrix we need pair **TL & BR**.

Total submatrices that can be made with $(x \times y) \rightarrow (i, j)$ is present

$\left\{ \begin{array}{l} x = (i+1) \times (j+1) \\ y = (n-i) \times (m-j) \end{array} \right.$

cont. of (i, j) in my sum

for $(i=0 \rightarrow n-1)$

for $(j=0 \rightarrow m-1)$

$sum += arr[i][j] \times \left[\begin{array}{l} (i+1) \times (j+1) \\ \times (n-i) \times (m-j) \end{array} \right]$

$O(n^2)$

value of cell

no. of submatrices (i, j) will be part of

Submatrix Sum Query on a matrix

I/P

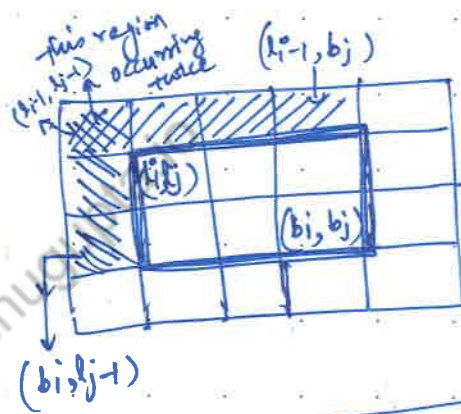
(l_r, l_c) and (b_r, b_c)

O/P

Sum of submatrix formed by TL coord. and B.R. coord.

	0	1	2	
0	a	b	c	
1	d	e	f	
2	g	h	i	

For ex. (1,1) (2,2)
 O/P (e+f+h+i)



* Best approach to solve this Q would be through prefix sum matrix.

In prefix sum, certain values will repeat so we must remove regions which are doing so.

$$\therefore \text{Sum} = pre[b][b] - pre[l-1][b] - pre[b][l-1] + pre[l-1][l-1]$$

Max sum submatrix in Row & Col wise Sorted matrix

If we start from Bottom right we may find max sum submatrix quickly.

-5	-4	-1
-3	2	4
2	5	8

"There's nothing more important than our good health - that's our principal capital asset."

In this case, SUFFIX SUM MATRIX is used.

Now, traverse through matrix and ~~when~~ max value in this matrix.

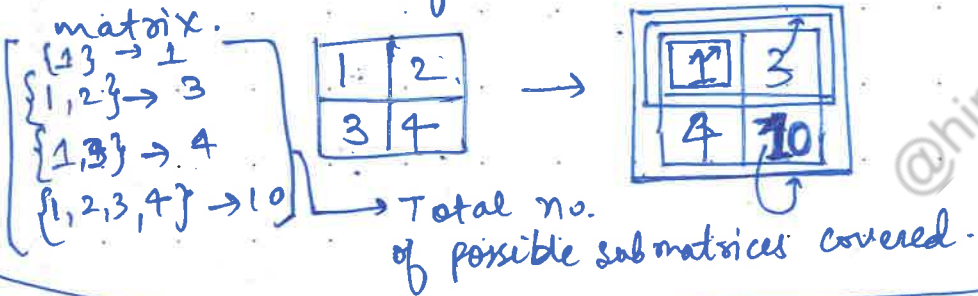
will be max sum of submatrix

$8+5+4+2-1-4-5$	$9+5+4+2-1-4$	$8+4-1$
$8+5+4+2-3$	$8+5+4+2$	$8+4$
$8+5+2$	$8+5$	8

Max sum = 19

Submatrix with max sum

PREFIX SUM MATRIX → Each element shows sum of each submatrix of matrix.



~~Code~~ // col wise addition fast

```

Code: - for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {
            arr[i][j] += arr[i][j-1];
        }
    }

```

Row wise

```

for(int i=n-1; i>=0; i--) {
    for(int j=n-2; j>=0; j--) {
        arr[j][i] += arr[j+1][i];
    }
}

```

```

int res = INT_MIN;
for (int i=0; i<n; i++) {
    for (int j=0; j<m; j++) {
        res = max(res, arr[i][j]);
    }
}

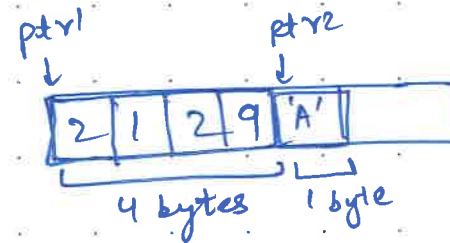
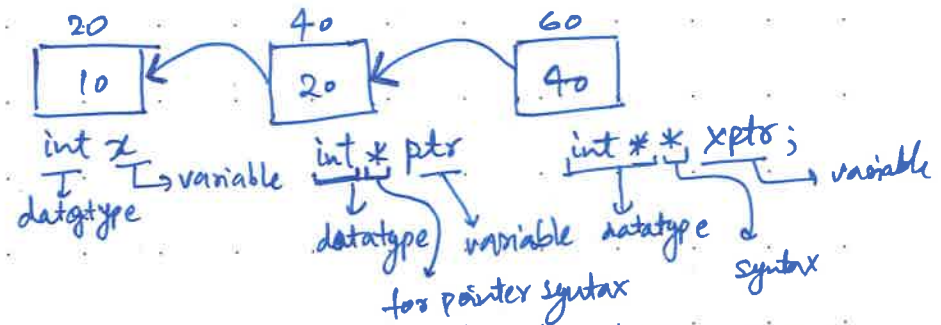
```

Pointers

int x = 10; → cout << &x; → Address in hexadecimal

char ch = 'A';
cout << &ch; → A // It doesn't work with char variables due to '<<' operator overloading.

cout << (void*)&ch; → Explicit Typcasting from char* to void* (delusion)



*ptr1 will point at start of address from where no starts in memory.

* Pass by Value

```

void increment(int a) {
    a = a+1;
    cout << a << endl;
}

```

```

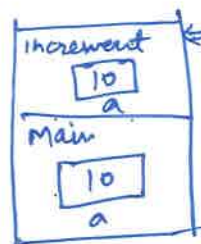
int main() {
    int a = 10;
    increment(a);
    cout << a << endl;
}

```

O/P
11
10

Explanation

When main calls increment(a) it makes another temp copy especially for this function.



Call stack

and uses $a=10$ and prints 11 after incrementing. but then this copy gets destroyed from the call stack. In $\text{main}()$, $a=10$ remains.

Call by reference

```
void increment(int *a_ptr) {
    *a_ptr = *a_ptr + 1;
    cout << *a_ptr << endl;
}
```

```
int main() {
    int a = 10;
    increment(&a);
    cout << a;
}
```

Pointer to

a [int *a_ptr = &a]

declaration + init.

address has been passed.

* The only valid arithmetic operations applicable on pointers are:

① Addition of integer to a pointer $\rightarrow$

Ex:- $\text{int } *ptr = \&x; \quad \&x = 10;$

$\text{cout} << (ptr + 3);$

address = 1024

$\hookrightarrow$ O/P :- 1036

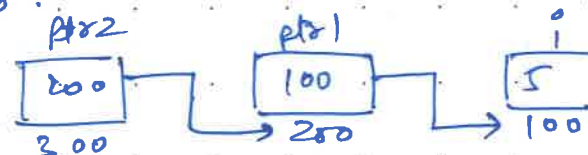
② Subtraction of integer to a pointer.

[Subtracting two pointers of same type] $\rightarrow$ gives no. of elements b/w them.

[Adding of two pointers is not allowed]

Dereference operator[*]

Used to find value pointed to by the pointer.



$100 \leftarrow ptr1 = \&i; \quad ptr = 200$
 $*ptr1 = 5; \quad *ptr2 = 100$

$**ptr2 = 5$

* The pointer left unused after declaration points to garbage value.

* An array is actually a pointer pointing to first element of array. (8 bytes for int)

$\therefore a[i] == *(a+i)$

* $\text{sizeof}(\text{pointer})$ - returns the amount of memory used by pointer variable

* $\&\text{array} == \&\text{array}[0]$
 $\therefore \&(\text{array} + i) == \&\text{array}[i]$

* $*\text{pointer} = \text{"abc"}$ $\rightarrow$ stores address of string "abc".

You can directly point to integer value with actually initializing variable for it.

```
void *memory = malloc(sizeof(int));
```

```
int *ptr = (int *) memory;
```

```
*ptr = 20;
```

```
cout << *ptr;
```

```
free(memory);
```

Memory Allocation

- 1. Static Memory Allocation/
Compile time Allocation
- 2. Dynamic Memory Allocation/
Run-time Allocation

1) Static Memory Allocation

Static memory is allocated for declared variables by the compiler. Firstly, when we declare variables and arrays virtually a memory is defined and a symbol table and on run-time it gets loaded onto RAM. The memory is allocated during compile time.

```
int main() {  
    int x;  
    char ch;  
    int a[100];  
}
```

Sizes & locations are fixed in this.

Advantages :- 1) Fast

Disadv:- less flexible

Symbol Table

a	400
ch	401
x	402

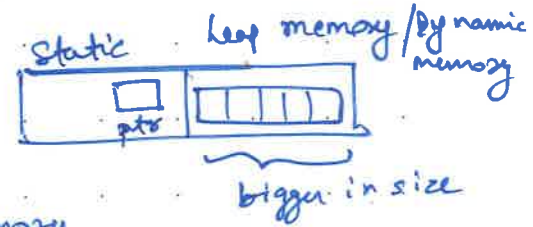
2) Dynamic Memory Allocation

It is done at the time of execution (runtime). Functions 1) new 2) delete are used.

In this, programmer decides when to allocate when to free. → Runtime

```
int *ptr = new int[100];
```

when new array is defined an array is made in heap memory where an address is passed which can be stored in a pointer which is stored in static memory.



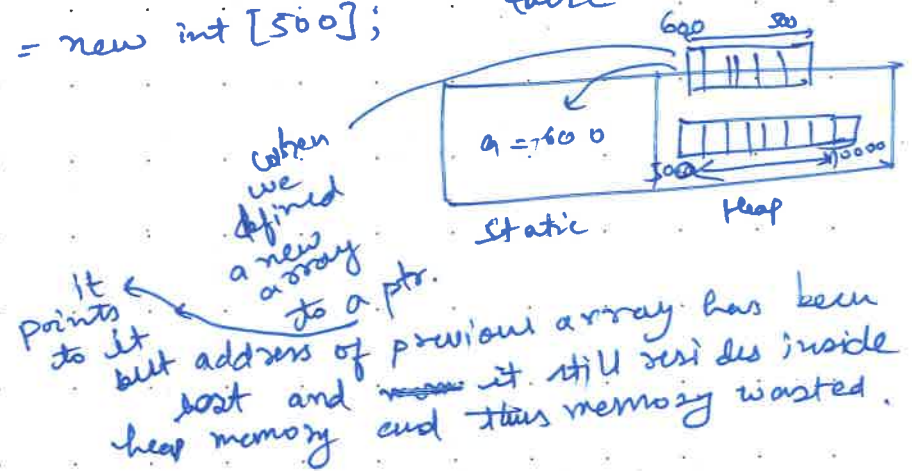
int x; → 4 bytes granted in static memory
→ Compile time

Memory leak [To be handled carefully]

1) int *a = new int[10000];

2) delete [] a; → if this step was not there

3) a = new int[500];



Adv:- Reusability, conservation of memory

Disadv:- Scope of errors, slower.

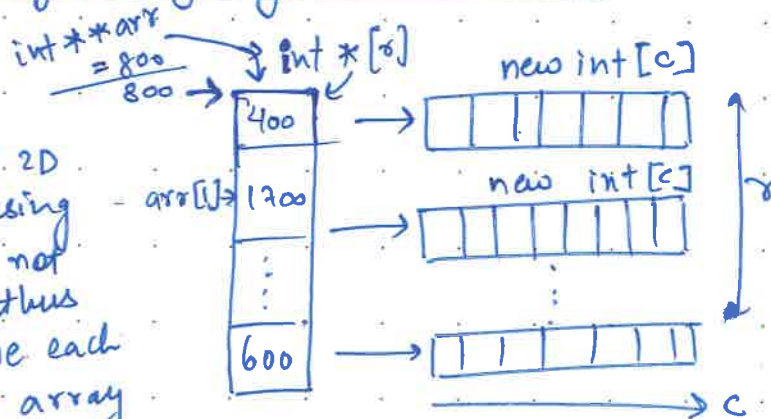
- * In static M.A, Allocation + Deallocation takes place by compiler. ^{once program goes out of scope}
- * User has to delete the memory allocated once program is at its end.

Ex:- `int b[100] = {0};`
`cout << sizeof(b) << endl;`
`cout << b << endl;` // symbol table
 // here b can't be overwritten, b is a part of Read Only Memory.

~~int n = 5;~~
~~int *a = new int[n]{0};~~
`cout << sizeof(a) << endl;`
`cout << a << endl;` // variable a that is created inside the static memory → overwritten i.e. `a = new char[30];` can be done.

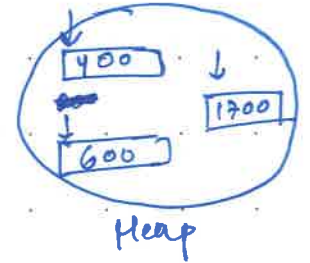
O/P:- ~~sizeof(b)~~ → 400
~~sizeof(a)~~ → 8 (Address in hexadecimal)

2D Array using dynamic allocation



Creating 2D Array using new is not possible thus we define each dynamic array pointed out to by a pointer element residing in another dynamic array pointed to by

a double pointer `**arr`. for ex:-
`**arr` stores address 800 and thus `arr` starts from 800 in which each element points each row array's first element.



- * In Heap, these row arrays are stored at non-contiguous locations.

Implementation

```
{
    int **arr;
    int r, c;
    cin >> r >> c;
    arr = new int * [r]; // array of row heads
    for (int i = 0; i < r; i++) {
        arr[i] = new int [c]; // defining row arrays pointed to by row heads
    }
    int val = 1;
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            arr[i][j] = val;
            val++;
        }
        cout << arr[i][c-1];
    }
}
```

* `arr[i][j] = *(arr[i] + j)`

1st pointer selects rows for that, it navigates through column.

Application of Static & Dynamic array in code

① `int * fun() {`

`int a[] = {1, 2, 3, 4, 5};` → static array

`cout << a << endl;`
`cout << a[0] << endl;`

`return a;` }

`int main() {`

`int *b = fun();`

`cout << b << endl;`

`cout << b[0] << endl;`

`} return 0;`

returns a local variable
pointer of address

O/P:- 0x7ffeec...

1

0x7ffec...

-28622952

1/garbage/seg fault

Explanation:-

mostly

Since `a[]` is static array, when `fun()` is called array `a[]` is created in a stack frame. but as soon as it returns `a`. All the local variables are deallocated (happens in static allocations). So just after returning value of `a` (address of array's first element) to `*b`, Array `a[]` no longer exists. And thus now if we print `b[0]` it will point to garbage value because of overwriting of programs at that memory location.

② `int * fun() {`

`int *a = new int[5];` → Dynamic array

`a[0] = 11;`

`a[1] = 2;`

`cout << a << endl;`

`cout << a[0] << endl;`

`return a;` }

`int main() {`

`int *b = fun();`

`cout << b << endl;`

`cout << b[0] << endl;`

`delete [] b;`

`}`

O/P:- 0x7fa7ac...
11
0x7fa7ac...
11

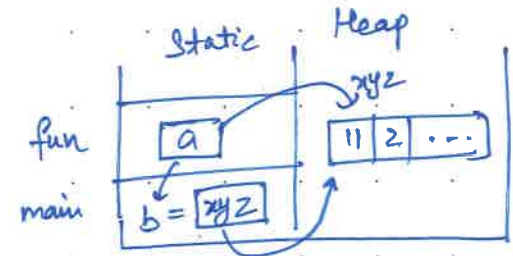
Deletes allocated memory for array from heap.

Explanation:-

Now `a` is a pointer pointing an array located in heap. So once it returns `a` at the end of `fun()`, (only static local variable is `*a`) so after returning the address "xyz" to `*b`, `*a` gets removed.

But dynamic array in heap still exists. pointed to by pointer `b` in `main()` function.

So we can again access that array through pointer `b` easily. In the end of `main()`, we will manually delete dynamic array and pointer `b` gets automatically deleted once "return 0" takes place.



Algo STL

1) Find Function

```
int arr[] = {1, 10, 11, 9, 100};  
int n = sizeof(arr) / sizeof(int);  
int key = 11;  
auto it = find(arr, arr+n, key);  
int index = it - arr;  
if (index == n) { cout << key << "not present"; }  
else { cout << index; }
```

returns address of key
address of first element
when iterator reaches to the end and points to nth index = null

2) Binary Search

```
bool present = binary_search(arr, arr+n, key);  
return True if exists  
else false
```

arr = {20, 30, 40, 40, 40, 50, 100}

```
auto lb = lower_bound(arr, arr+n, 40);  
cout << (lb - arr) << endl;
```

• lower_bound returns address of number $\geq$ key i.e. 40.

o/p: - index of first occurrence i.e. 2

In case key doesn't exist it returns index of first element $>$ key i.e. 50 at index 5

```
auto ub = upper_bound(arr, arr+n, key);  
cout << (ub - arr) << endl;
```

upper bound

→ returns address of element $>$ key.
if key = 40, then o/p → address of 50
∴ $ub - arr = 5$

→ We can use $(ub - lb)$ to find occurrence frequency of key in array and if the key doesn't exist it will return 0.

Ex: - when key = 40, $ub - lb = 3$

40, 40, 40, 50
↑ ↑
lb ub

Fastest sorting algorithm → Quicksort
Best and avg → $O(n \log n)$
worst → $O(n^2)$

* Sort() in C++ offers hybrid of Quicksort and Heapsort. For best & avg case it works as Quicksort() else works as Heapsort (which provides $O(n \log n)$).

```
for (auto it = lb; it != ub; it++) {  
    cout << *it << " ";  
}
```

(Cint*) can also be used

Comparator sort [If comparator returns false then swap takes place else condⁿ holds true]

* Passing a function in other function as an argument

```
bool compare(int a, int b) {  
    return a > b; }  
void bubble_sort(int a[], int n, bool (&cmp)(int a, int b)) {  
    if (cmp(a[i], a[i+1])) {  
        // swap  
    }  
}
```

usually a < b is returned for asc
we defined comparator here

bubble-sort (a, n, compare);

↳ sort series in asc.

When comparator func returns 0 then swap takes place else not.

Using comparator in a function to change functioning

```
bool compare (int a, int b) {
    return a < b;
}
```

```
int main() {
    int coins[] = {1, 2, 5, 10, 20, 50, 100, 200, 500, 1000};
    int n = coins.size();
    int money = 120;
    int lb = lower-bound(coins, coins[n], money, compare);
    // coins - 1;
    int m = coins[lb];
}
```

Compare is defined such that output satisfies it

* Rotate()

```
int arr[] = {10, 20, 30, 40, 50};
```

rotate(arr, arr+2, arr+n); → rotate the array from 2nd index element.

O/P: {30, 40, 50, 10, 20}

In vector -

```
rotate(v.begin(), v.begin()+3, v.end());
```

O/P: {40, 50, 10, 20, 30}

* Next-permutation() - gives next permutation

```
vector<int> v{1, 2, 3};
next_permutation(v.begin(), v.end());
next_permutation(v.begin(), v.end());
```

```
for (int x: v) { cout << x << " "; }
```

for all x in vector v.

O/P: - 2 1 3

↳ called next-permutation two times so on first call it changed to 1 3 2 then 2 1 3.

* swap(); * prev-permutation() - gives prev permutation

Binary Search [Divide & Conquer]

Binary Search - First Occ & Last Occ

arr
1 2 5 8 8 8 8 10 12 15 20
↑ first ↑ last occurrence

```
int first-occ(int a[], int n, int key) {
```

```
    int s = 0;
    int e = n-1;
    int ans = -1;
    while (s <= e) { int mid = (s+e)/2;
```

```
        if (a[mid] == key) {
```

```
            ans = mid;
```

```
            e = mid-1; }
```

```
        else if (a[mid] > key) {
```

```
            e = mid-1; }
```

```
        else s = mid+1; }
```

```
    return ans; }
```

When we find key at mid index we do the same for left side of array and so on. and along with that store index of element.

Same for end occurrence (s = mid+1);

Q Search an element in a sorted and rotated array.

I/P -

4	5	1	2	3
---	---	---	---	---

 ↑
 to be found
 key = 2

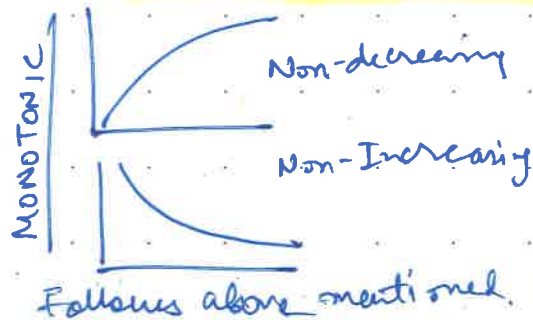
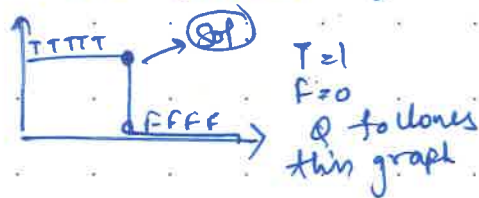
O/P :- Index 3

* Binary Search in Monotonic Search spaces

Ex. - Find square root of no. in int / floor value.
 $\sqrt{10} \Rightarrow 3$

(Used in most of B.S. Qs)

We are not given up with a sorted array with create our own set of array to find a value which closely fits up with the question requirement.



$\frac{0+5}{2} = 2.5 \Rightarrow 2.5^2 = 6.25 < 10 \therefore 0-2$
 $\frac{2+5}{2} = 3.5 \Rightarrow 3.5^2 = 12.25 > 10 \therefore 2-3$
 $\frac{2+3}{2} = 2.5 \Rightarrow 2.5^2 = 6.25 < 10 \therefore 2-3$
 $\frac{2+3}{2} = 2.5 \Rightarrow 2.5^2 = 6.25 < 10 \therefore 2-3$
 $\frac{2+3}{2} = 2.5 \Rightarrow 2.5^2 = 6.25 < 10 \therefore 2-3$
 $\therefore 3 = \text{closest no.} \leq 10$
 O/P - 3

Code

```

int square-root (int n) {
    int s = 0;
    int e = n;
    float ans = -1;
    while (s <= e) {
        int mid = (s + e) >> 1;
        if (mid * mid == n) {
            return mid;
        }
        if (mid * mid < n) {
            ans = mid;
            s = mid + 1;
        }
        else {
            e = mid - 1;
        }
    }
    // To find floating value of square no.
    float inc = 0.1;
    for (int times = 1; times <= 10; times++) {
        while (ans * ans <= n) {
            ans = ans + inc;
        }
        ans = ans - inc; // remove extra inc to make it <= n.
        inc = inc / 10;
    }
    return ans;
}
  
```

Midpoint of two nos.

$a = 9, b = 5$
 $1001, 101$
 $a + b = 14 = 1110$
 $a + b >> 1 \Rightarrow 0111$
 $\downarrow$
 7

Right shift operator

// overflow can happen result in TLE
 So (ll) $mid * mid$ or $mid = n / mid$;
 Till now we will get integer value of square root.

Factorial of a Large Number

Factorial of 100 has 158 digits. It is not possible to store these many digits even if we use long long int.

Array Method

- 1) Create an array 'res[]' of MAX size where MAX is maximum digits in output.
- 2) Initialize value stored in 'res[]' as 1 at 0th index and initialize 'res_size' (size of 'res[]') as 1.

Head and cheerfulness naturally beget each other.

3) Do following for all nos. from $x=2$ to n .

a) Multiply x with $res[]$ and update $res[]$ such that each digit resides in each bucket and update res_size to store multiplication result. Also use carry to do this multiplication correctly.

* We maintain $res[]$ in reverse way i.e. digits from right to left so that increasing the size of $res[]$ becomes easy.

$\boxed{9} \boxed{8} \boxed{1} \boxed{3} \rightarrow \boxed{3} \boxed{1} \boxed{8} \boxed{9}$
0 3 0 3

4) Run the loop in reverse from $res_size - 1$ and print all digit so that we will get required large factorial.

NOTE:- We can directly pass arr by $arr[]$ because arr is a pointer to the first element of array.

Log method

We can use log following property to find factorial.

$$N! = \prod_{i=1}^N i$$
$$\ln(N!) = \ln\left(\prod_{i=1}^N i\right) = \sum_{i=1}^N \ln(i)$$

$$\therefore e^{\ln(N!)} = N!$$

* clock() $\rightarrow$ This function returns approx. processor time that is consumed by the program.



A NEW-AGE COMPREHENSIVE HEALTH INSURANCE



Covers up to
6 family members



30-Day Pre & 60-Day
Post Hospitalization



No-Claim Bonus
up to 150%



540+ Day Care
Treatments

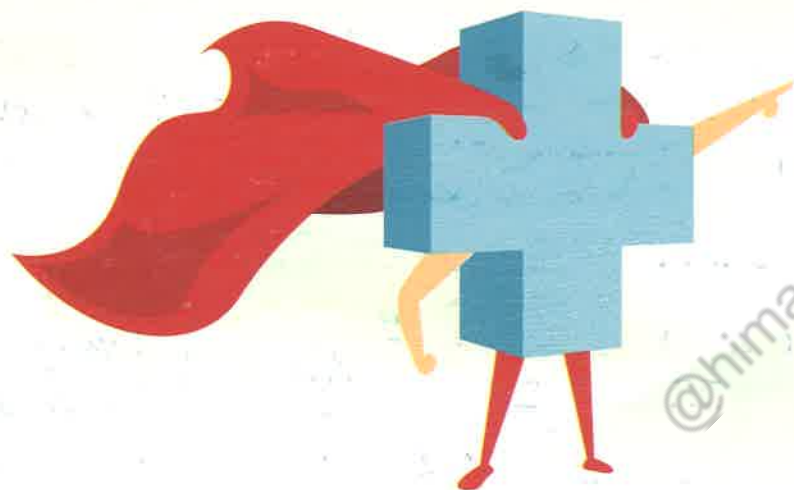


Annual Health
Check-up for all
Insured members



Unlimited Automatic
Recharge of Sum Insured

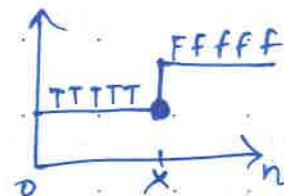
THE SUPERHERO OF HEALTH INSURANCE



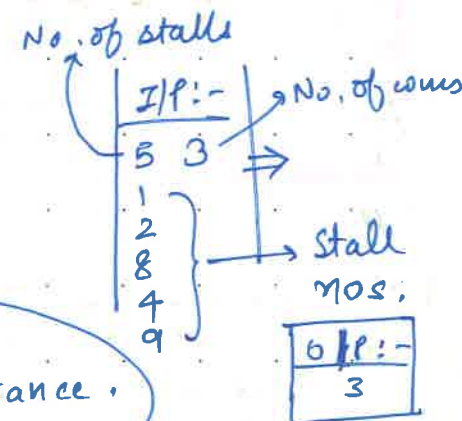
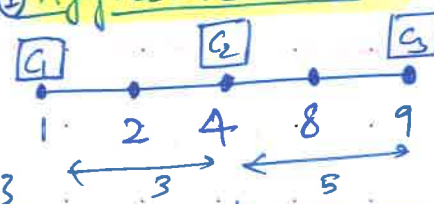
* How to ~~arrive~~ know when to use Binary Search or not.

Monotonic graph

For a given Q , the value till x will give us True but after x it starts giving me False. So the max possible solⁿ would be maximum value for which we get "True" i.e. x . Then we use binary search to eliminate either side of monotonic search space till we reach at the point where we get the answers, using mid of ~~max~~ the array. $[0-n]$.



Ex: 1. Aggressive Cows



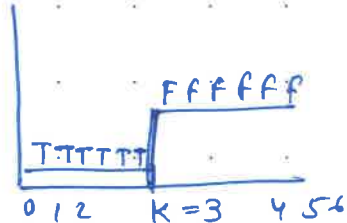
For each test cases. output the largest minimum distance.

Explanation:- If we position three cows at max possible distance from one another then minimum distance among a pair is 3.

Algorithm.

Only two cases are possible for which λ holds true. Either there will exist a $\min$ distance b/w each pair or not. So in that case we'll get either T or F.

If $(K=3)$ ~~was~~ ^{we} was to be the ~~max~~ largest min distance then it will hold true for $(0 \leq K \leq 3)$ also but not for $(>K)$.



(1) Select a monotonic search space of from 0 to $arr[n-1] - arr[0]$.

(stores pair of stalls)

max. distance possible b/w a pair.

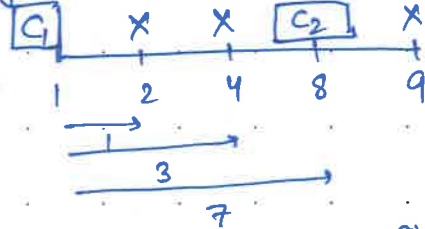
NOTE:- We are making monotonic search space of differences possible b/w each pair of cows.

$S = 0;$
 $e = arr[n-1] - arr[0];$

i.e. 0 to 8

(2) mid = 4

Since mid = 4, we will check whether largest min distance = 4 is possible.



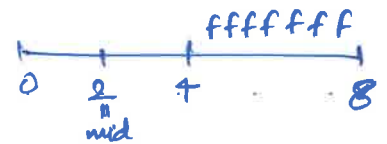
(No space left for C_3)

No line of stalls.

For 4 the given condition doesn't hold true.

(3) If for mid value the condition holds "FALSE" we will shift

$e = mid - 1;$



(4) we will make pairs such that the distance b/w each pair remains $\geq mid$. If the above condⁿ ~~can~~ ^{is} be outlined on member ~~the~~ holds true for all cow pairs then it means condⁿ holds "TRUE". So we will update a variable which stores largest minimum distance and shift $s = mid + 1;$

② Book Allocation ($O(N \log(\sum(arr[i]) - \max(arr[i])))$)

$N = 4$ books
 $M = 2$ students
Books = [10, 20, 30, 40]

O/P: - 60

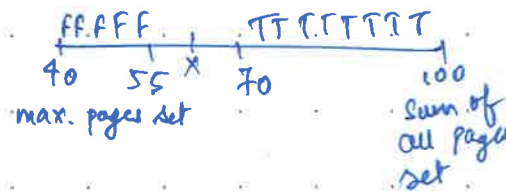
Minimize the max pages for each student.

Monotonic search space based on pages

EXP:-

10, 20, 30, 40
Max pages after minimising: 60, 51, 52

Exp



Like that we can find out 60

$S_1 = 10 + 20 + 30 \leq 70$
 $S_2 = 40 \leq 70$
That means all no. of pages > 70 will satisfy.

$S_1 = 10 + 20 \leq 55$
 $S_2 = 30 \leq 55$
 $S_3 = 40$ exceeded the no. of students.
That means less than 55 not possible

③ Problem PRATA (spoj.com)

④ Painters Problem (spoj) (Hard) → Book Allocation variant

⑤ EKO (spoj)

⑥ winning CB scholarship

⊕ Substrs (start index, ending index); → substring selector

VECTORS (Like list in python)

vector are same as dynamic arrays with the ability to resize itself automatically when an element is inserted or deleted. Iterators can be applied on vector since it stores element contiguously.

vector < data-type > variable-name;

→ vector < int > b(5, 10); // five int with value 10

→ vector < int > c(b.begin(), b.end());

→ vector < int > d { 1, 2, 3, 4, 5 };

// Iteration using iterators (normal for loop iteration is allowed)

→ for(auto it = b.begin();
it != b.end(); it++) {
cout << (*it) << " "; }

[Iterator has
datatype -
vector < int > :: iterator
(auto can be used)

or

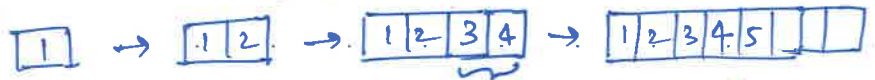
→ for(int x : d) {
cout << x << " "; }

(shortcut)

→ Push-back () - it pushes elements from back.

NOTE:- Size of allocated memory space gets doubled on each push-back().

⑤



(when you pushed '3' total size became 4. When you pushed '4' there was empty space.)
(On pushing 5 $2 \times 4 = 8$ size)

v.capacity() → 8

v.size() → 5

(size of underlying array)

d.capacity() → 5
because d is initialized already

→ v.max_size(); → max. no. of elements vector can hold.

→ insert(); → inserts new elements from a container element at specified posⁿ.

→ erase(); → remove elements from a container from specified range or range.

→ clear(); → removes all elements

→ emplace(); → extends element by inserting new element at posⁿ.

Ex:-

v.insert(v.begin+3, 4, 100);

quantity

no. to be added

From index 3 add 4 100s

d.erase(d.begin()+2, d.begin()+5);

↳ Removes elements at index 2 and 5.

- `front()` → first element of vector
- `back()` → last element of vector

To overcome memory consumption during push-back, you can use `reserve()`;

`v.reserve(100);` → 100 locations have allocated.

Now you can directly initialize each locations.

- `vector<pair<int, int>> v; / vector<struct pair>`
- `v.push_back(make_pair(x, y));`
- `sort(v.begin(), v.end(), Comparator);`

`v;`
`struct pair`
`{ int x;`
`int y; };`

Q. `vector<int> v = {1, 3, 2, 5}`
`cout << v.capacity();` → 4
`v.push_back(12);`
`v.capacity();` → 8

Imp:- Vector class can be implemented using dynamic array.

```
class myVector {
    int *data;
    int capacity;
    int elementsInVec;
public:
    myVector() {
        data = new int[1000];
        elementsInVec = 0; capacity = 1000;
    }
};

class Car {
public:
    string car_name;
    int x, y;
    Car(string n, int x, int y) {
        name = n;
        this->x = x;
        this->y = y;
    }
    void display() {
        return x*x + y*y;
    }
};

// Now we can use
vector<Car> v;
Car temp(name, x, y); // object of car class
```

Container Design [Implementation of vector using class]

In a "vector.h" file. and use `#include "vector.h"` to use

```
class Vector {
    int cs;
    int ms;
    int *arr;
public:
    // (private items of vector class) (can't be used outside)
    // (can be used by coder while using class)
```

```
    cs = 0;
    ms = 1;
    arr = new int[ms];
}
```

```
void push_back(const int d) {
    if (cs == ms) {
        // Array is full
        int *oldArr = arr;
        arr = new int[2 * ms];
        ms = 2 * ms;
        for (int i = 0; i < cs; i++) {
            arr[i] = oldArr[i];
        }
        // clear the old memory
        delete [] oldArr;
    }
}
```

```
arr[cs] = d;
cs++;
```

```
void pop_back() {
    cs--;
}

int front() const {
    return arr[0];
}
```

(we are not changing any value just returning it.)

"Health is the thing that makes you feel that now is the best time of the year."

```

int back() const {
    return arr[cs-1];
}

bool empty() const {
    return cs == 0;
}

int capacity() const { return ts; }

```

// operator overloading

```

void int operator[] (const int i) {
    return arr[i];
}

```

Syntax: `operator` → operator to be overloaded

NOTE: Above code only works for int values. In order to follow "vector<data-type> v" we will use Templates.

Templates

Just you have to do before class definition do this :-

```
template<typename T>
```

Now wherever `int` is used when dynamic array is initialized, use `T` and wherever in function we are returning element.

Bits info

000	+0
001	+1
010	+2
011	+3
100	-0
101	-1
110	-2
111	-3

Sign bit (1 bit)

when we consider signed int
Range of INT $\Rightarrow (-2^{31}-1)$ to $(2^{31}-1)$
for unsigned int
(0 to $2^{32}-1$)



Bit Manipulation

Operations - $\&$ (AND), $|$ (OR), $\sim$ (NOT), $\wedge$ (XOR)

Properties of XOR $\rightarrow a \wedge a = 0$
 $a \wedge a \wedge b = b$

Shift operators $\rightarrow$ (i) $\ll$ left shift
(ii) $\gg$ right shift

(i) $\ll$ left shift

$$5 \ll 1 \Rightarrow 101 \ll 1 \Rightarrow 1010 \Rightarrow 10$$

$$\therefore a \ll 1 \Rightarrow a = a * 2$$

$$\therefore a \ll b \Rightarrow a * 2^b$$

(ii) $\gg$ Right shift

$$1010 \gg 1 = 101 \Rightarrow 5$$

$$a \gg b = a / 2^b$$

int value

* Checking if no. is odd/even

$$n = a \& 1 \quad \begin{array}{l} \text{if } n == 1 \text{ (odd)} \\ \text{if } n == 0 \text{ (even)} \end{array}$$

Get & Set Bits

□ Get i th bit (from Right)

$$\begin{array}{r} 000101 \\ \& 000100 \\ \hline \text{Ans} = 000100 \end{array}$$

i-th bit

$$\left\{ \begin{array}{l} \text{If Ans} > 0 \Rightarrow \text{Set} = 1 \\ \text{If Ans} == 0 \Rightarrow \text{Set} = 0 \end{array} \right.$$

$$\text{Mask} = 1 \ll i$$
 , where i is the index from right side where we set on i.e (1).

Code:-

```

int getBit(int n, int i) {
    int mask = (1 << i);
    int bit = (n & mask) > 0 ? 1 : 0;
    return bit;
}
  
```

Set i-th bit [set bits $\Rightarrow$ 1s in a bin]

first we will perform OR op. with mask.

```

int mask = (1 << i);
int bit = (n | mask);
return bit;
  
```

Clear i-th bit

We can either perform AND op. with negation of mask value or directly perform XOR.

$$\begin{array}{r} 100101 \\ \& 111011 \Rightarrow \sim(1 \ll i) \\ \hline 100001 \end{array}$$

$i=2$

We will use Bit Masking to create mask/bit mask in which multiple bits in a byte, nibble, word etc can be set either on, off or inverted from on to off

Update Bit

$$\begin{array}{r} V=1 \quad n=10101 \\ i=2 \quad \text{(OR)} \quad 00010 \\ \hline 10111 \end{array}$$

$$\begin{array}{r} V=0 \quad 10101 \\ i=2 \quad \text{(AND)} \quad 00100 \\ \hline 10001 \end{array}$$

Clear range of Bits

*** Clear last i bits**

```

int mask = (~1 << i);
return n & mask;
  
```

$[-1 \text{ or } \sim 0 = 111111]$

$i=4$

$$\begin{array}{r} 1011011 \\ \& 1110000 \leftarrow \text{mask after left shift} \\ \hline 1010000 \end{array}$$

*** Clear range $[i, j]$**

```

int ones = (~0);
int a = ones << (j+1);
int b = (1 << i) - 1;
int mask = a | b;
int ans = n & mask;
return ans;
  
```

$$\begin{array}{r} 11100000 \\ \text{OR } 00000011 \\ \hline 11100011 \Rightarrow \text{mask} \end{array}$$

$111 \Rightarrow 7 = 2^3 - 1$

$$\begin{aligned} & \therefore 2^i - 1 \\ & = (1 \ll i) - 1 \\ & = 10000 - 1 \\ & = \text{i zeros} \\ & = 01111 \\ & \quad \text{i no. of 1s} \end{aligned}$$

*** Replace bits in N by M**

```

n = clearRange(i, j);
/* N = 100000000
   M = 10101
   ans = (M << i) | n;
  
```

*** Count Setbits in N**

Naive app.

```

ans = 0;
while (n > 0) {
    ans += (n & 1);
    n = n >> 1;
}
return ans;
  
```

T.C $\rightarrow O(N)$

Best App.

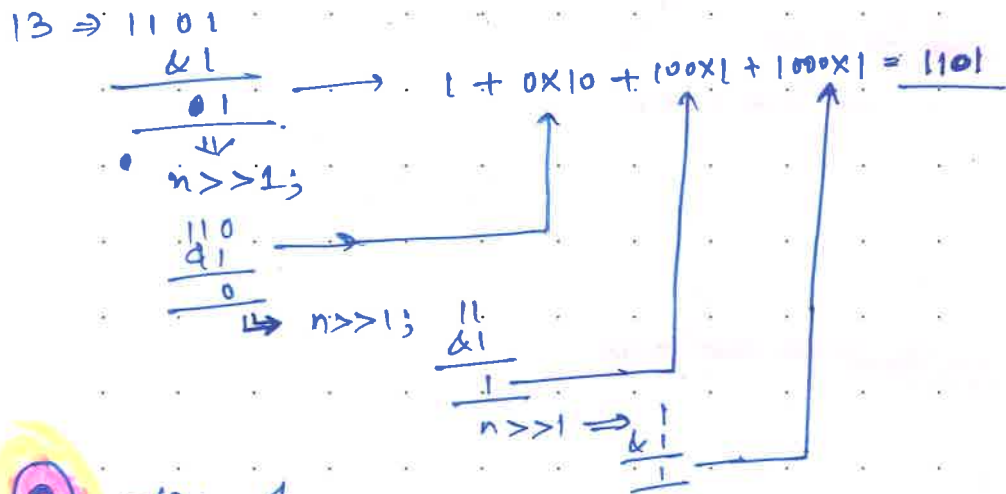
$$\begin{array}{r} \text{Gx} := 9 \rightarrow 1001 \\ \text{while (n > 0)} \{ \\ \quad n = n \& (n-1); \quad \text{removes (flips)} \\ \quad \text{ans}++; \quad \text{previous set bit at each iteration} \\ \} \\ \text{return ans;} \end{array}$$

T.C $\rightarrow O(\text{No. of setbits})$ In W.C $\rightarrow O(N)$

* builtin funcⁿ to count setbits

-- builtin - popcount(n);

■ Decimal to Binary using Bitwise



Q I/P :- 4
3 1 2 1
O/P :- 2 3

Exp :- All the elements appear twice except for two nos. which are present only once.

Algo

arr[] = 5, 1, 2, 1, 2, 3, 5, 7

$$res = 5 \wedge 1 \wedge 2 \wedge 1 \wedge 2 \wedge 3 \wedge 5 \wedge 7 = 3 \wedge 7$$

$$\begin{array}{r} 011 \\ (XOR) 111 \\ \hline 100 \end{array}$$

setbit at index 2nd (first setbit from right)

we will calculate XOR of set of nos. which has setbits at 2nd posⁿ.

$$x = 5 \wedge 1 \wedge 7 = 7$$

$$y = res \wedge x = (3 \wedge 7) \wedge 7 = 3$$

How we will check if a no. has set bit at ith index.

```
for(int i=0; i<n; i++){
    if((arr[i] & (1<<(index of setbit))) > 0){
        x = arr[i] ^ x;
    }
}
```

Q I/P :- 7
3, 3, 2, 1, 1, 1, 3

O/P :- 2

Exp :- All nos. present thrice except for one no. Find the no.

Algo

We can define the total no. of elements in array $= 3N+1$

That unique no.

3 $\rightarrow$ 011
3 $\rightarrow$ 011
2 $\rightarrow$ 010
1 $\rightarrow$ 001
1 $\rightarrow$ 001
1 $\rightarrow$ 001
3 $\rightarrow$ 011

So if unique no. has no set bit at particular posⁿ that means there may exist 3N setbits of each any or all triplets.

Total no. of setbits at each posⁿ.

0, 4, 6
We will take modulo of each total by 3. It will produce 0, 1, 0.

6 4 0 ... (6+bits long can be)
0 1 2
index

set bit at 2nd posⁿ

We are storing in an array this way.

NOTE: We can solve for $4N+1$, $5N+1$, and so on.

0 1 0 ...

Unique no. shall be $\Rightarrow 2^0 + 2^1 + 2^2 \dots \Rightarrow 2$

⊛ If $N=5$, Max no. of subsets N can have will be $\log_2 N$.

✓ Fast Exponentiation using Bitmarking (better than pow)

If you want to find a^5 .

Ex:- $a=3, n=5$.

Since we know $\rightarrow a^5 = a^{101} \Rightarrow a^4 \cdot a^1$

Algo

ans = 1

$n = 101$

$\therefore \text{ans} = 1 * 3^1$

$\downarrow$
 $\text{ans} = 1 * 3^2 * 3^1$
 $= 3^5 \Rightarrow 243$

if 1 bit $\rightarrow$ multiply by ans, the current value (a^p)
 0 bit $\rightarrow$ skip.
 And on every iteration $p = p * 2$

(in $O(\log n)$)

Since each binary no. has $\log N$ max bits if no. is N .

⊛ Generate Subsets using Bitmasking

Finding subseq. / subsets of given a string.
 I/O:- "abc"
 O/P:- "", a, ab, abc, ac, b, bc, c

Algo

	a	b	c
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

For ex:- if 011

so there we will traverse from right side and check set bits and print ab because at third pos 1 is there and a is not printed.

That means each 1 bits define a character of a string to be taken subsets from.

Number Theory

⊛ Prime Sieve | Eratosthenes Sieve

Algo

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
x			x		x		x	x	x		x		x	x	x		x	

Starting from 2

Mark multiples of each no. till N .

4, 6, 8, 10, 12, ... (non prime)

for 3,

6, 9, 12, 15, 18, 21, 24, ... (n.p.)

for 5,

10, 15, 20, 25

Skip if that no. is non-prime

$\therefore$ loop will be like

for ($i = 2 * \text{no}$; $i \leq \text{MaxRange}$; $i += \text{no}$) {

⊛

// optimisation can be done.

Primes[i] = false; }

On observation we can see

already on a divider 5, 10, 15, 20, 25, ... avoid

"The only way to keep your health is to eat what you don't want, drink what you don't like and do what you'd rather not."

∴ we can start printing from n^2 .

One more optimisation. → mark all even nos. except 2 as non-prime. We will have to traverse only from odd nos.

Complexity :- $\frac{N}{2} + \frac{N}{3} + \frac{N}{5} + \frac{N}{7} + \frac{N}{11} + \dots$

$= N(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots)$

$= O(N \cdot \log \log N)$

if $N=60$; $\log \log 60 = 7$
Approx. Linear

Code

```
void prime_sieve(int *p) {
    for (int i=3; i<=10^6; i+=2) {
        p[i]=1;
    }
    for (int i=3; i<=10^6; i+=2) {
        if (p[i]==1) {
            for (int j=i*i; j<=10^6; j+=i) {
                p[j]=0;
            }
        }
    }
    p[2]=1;
    p[1]=p[0]=0;
}
```

#define max 10⁶
(do this to avoid seg fault)

↓
Optimisation

Q Prime visits Problem

Q/P: 2 10 4
" 20 4
O/P :- No. of primes

In this we will maintain a suffix sum array where at further locations we will store count of all primes from 2 to that location.

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

0 1 2 2 3 3 4 4 4

So count[9] = 4

And for each queries we will output the no. of prime nos. in $O(1)$ time.

Time complexity = $O(N+Q)$ approx.

Naive approach

We would have calculated no. of queries for each queries T.C = $O(N \cdot Q)$

Prime Factorisation

(1) 2 $\sqrt{N}$ $\frac{N}{\sqrt{N}}$
For ex. This no. has divisors $\geq \sqrt{N}$ or $< \sqrt{N}$

2, 3, 4, 6, 9, 12, 18, 36

They form pairs so we can say smallest divisors can be found $\sqrt{N}$.

If $132 = N$
∴ Then $O/P := 2 \cdot 3 \cdot 11$

```
for (i=2; i<=sqrt(N); i++) {
    if (N%i==0) {
        cnt=0;
        while (N%i==0) {
            cnt++;
            N=N/i;
        }
        if (i!=30) {
            vector.push_back(make_pair(N, cnt));
        }
    }
}
```

Time complexity :- $O(\sqrt{N})$

(2) Using Sieve (Better approach)

Pre compute all the prime nos.

(Useful when many queries are given)

(((N log log N)))

```

// Sieve
vector<int> primeSieve(int *p, int n) {
    p[0] = p[1] = 0;
    p[2] = 1;
    for (int i = 3; i <= n; i += 2) {
        p[i] = 1;
    }
    for (int i = 3; i <= n; i += 2) {
        if (p[i]) {
            for (int j = i * i; j <= n; j += i) {
                p[j] = 0;
            }
        }
    }
}

```

Optimisation
 if you define p[max] globally #define max 106 to avoid seg fault
 // Iterate over odd nos.

j += 2 * i

Optimisation
 if i = 9 then j = i * i = 81
 then j += i → j = 90
 So we do 2 * i ← even

```

vector<int> factorize(int m, vector<int> &primes) {
    vector<int> factors;
    factors.clear();
    int i = 0;
    int p = primes[i];
    while (p * p <= m) {
        if (m % p == 0) {
            factors.push_back(p);
            while (m % p == 0) {
                m = m / p;
            }
        }
        i++;
        p = primes[i];
    }
    if (m != 1) {
        factors.push_back(m);
    }
    return factors;
}

```

NOTE:- While creating an array of x size be aware of the fact that insufficient memory size will give **SEG FAULT**

```

int N = 1000000;
int p[N] = {0};
vector<int> primes = primeSieve(p, 1000000);
int k; cin >> k;
while (k--) {
    // ...
}

```

* Counting Divisors (using Sieve)

For ex:- N = 60

No. of divisors ⇒

$$60 = 2^2 \times 3^1 \times 5^1$$

$(2^0, 2^1, 2^2)$ $(3^0, 3^1)$ $(5^0, 5^1)$
 p_1 p_2 p_3

Applying fnc

Total no. of divisors possible,

$$= (p_1 + 1)(p_2 + 1)(p_3 + 1)$$

case when we take power as 0, i.e. divisor = 1.

$$= (2 + 1)(1 + 1)(1 + 1) \Rightarrow 12$$

We can modify primesieve function into giving us the total count. first we count each primes and along with it maintain ans = 1; and do

$$ans = ans * (cnt + 1);$$

count of each primes in no.

In last

if (m != 1) {
 ans = ans * (1 + 1);
}

* GCD - Euclid's Algo

Ex:- a = 12, b = 20
 then $\frac{a}{b} = 4$

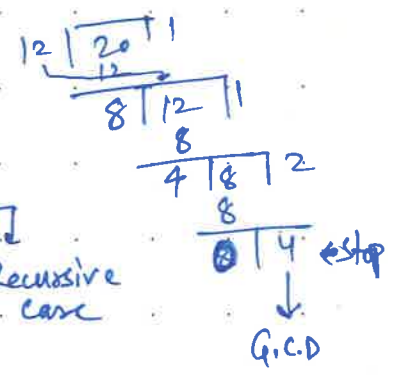
Algo

$$GCD(a, b) = GCD(b, a \% b);$$

Recursive case

$$GCD(a, 0) = a;$$

base case



Recursive approach

```
int gcd (int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

* $\text{gcd} \times \text{lcm} = a \times b$

$\text{lcm} = \frac{12 \times 20}{4}$

if $(a = b) \{$
 $\text{gcd}(a, b) \}$
 else $\{ \text{gcd}(b, a \% b) \}$

Properties of modulo (To make calculations easier)

- ① $(a+b) \% m = ((a \% m) + (b \% m)) \% m$
- ② $(a-b) \% m = ((a \% m) - b \% m + m) \% m$
- ③ $(a * b) \% m = ((a \% m) * (b \% m)) \% m$
- ④ $(\frac{a}{b}) \% m = ((a \% m) * (b^{-1} \% m)) \% m$
 (where $b^{-1} \Rightarrow$ multiplicative modulo inverse)
 $(6 * y) \% 7 = 1$
 $\hookrightarrow 6$

Divisible Subarrays (Using Pigeonhole principle + modulo properties)

You are given N elements $a_1, a_2, a_3, \dots, a_N$. Find the no. of good sub-arrays. Good sub-array $[a_i, a_{i+1}, a_{i+2}, \dots, a_j]$ such that $(a_i + a_{i+1} + a_{i+2} + \dots + a_j)$ is divisible by N .

I/O:-
 2
 5
 1 1 1 1
 5
 - e e r r

O/P:- 1
 15

Constraints:-
 $1 \leq N \leq 10^5$
 $|a_i| \leq 10^9$

Algo:-

Arr -

1	3	2	6	4
0	1	2	3	4

$N=5$

$O(N^2)$ is inefficient

Take prefix sum / cumulative array.

Pref:-

0	1	4	6	12	16
0	1	2	3	4	

We have to check whether

$(b-a) \% N = 0$

(where $b > a$) index value
 $(b-a)$ = sum of a subarray.

$b \% N = a \% N$ — ①

Now take modulo of each element in array by N .
 $(\% N)$

Arr =

0	1	4	1	2	1
---	---	---	---	---	---

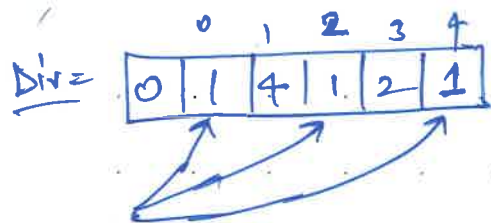
Now an observation can be made that $x \% 5$ can yield only 0, 1, 2, 3, 4. thus only five unique values.

$\therefore$ 6 boxes with 5 unique values and at least 2 boxes will have same value so that we can confirm eq. ①.

Using pigeon hole

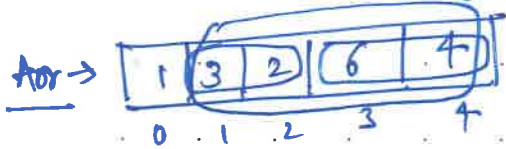
1	3	1	X	1
0	1	2	3	4

5 buckets



value at index 0th,
2nd and 4th
will satisfy

These ~~three~~ values can form
pairs to satisfy eq. (1).



For ex. Three
subarrays
marked
follows
above
observation.

We can use prefix sum
array to apply above
thing.

∴ No. of subarrays possible are
 $\Rightarrow 3C_2 = \frac{3 \times 2}{2} = 3$ subarrays

for ex:-

0	0	0	1	0	1	4	3
---	---	---	---	---	---	---	---

if modulo array was like the above.

There are 4 in bucket 0th
and 2 1s in bucket 1th.

∴ Answer = $4C_2 + 2C_2 = 7$ subarrays

NOTE - Take care of the case where no.
can be negative. So after taking
modulo $\Rightarrow x \% n = x \% n$
once again do this $\Rightarrow (x+n) \% n$
In case no. = -5. Then $-5 \% 4 = -1$
 $\therefore (-1+4) \% 4 = 3$ (ve)

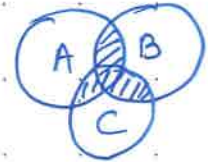
* There is no need to make prefix array
just store modulo directly and maintain a
sum which contain previous ~~prefix~~ (modulo
values to be taken mod of.

Check coding blocks for code

Counting Problems - Inclusion-Exclusion Principle

Q. How many nos. b/w 1 to 100 are multiples
of 2 or 3 or 5?

A → Multiples of only 2
B → Multiples of 3
C → Multiples of 5.



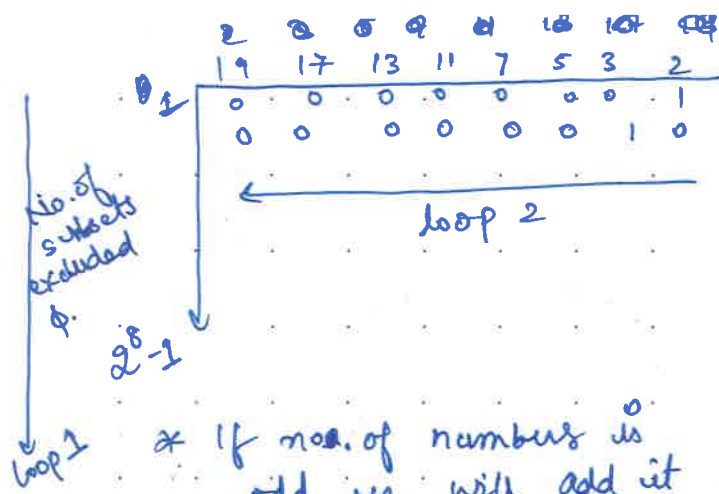
$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

$$\text{floor} \Rightarrow \left\lfloor \frac{100}{2} \right\rfloor + \left\lfloor \frac{100}{3} \right\rfloor + \left\lfloor \frac{100}{5} \right\rfloor - \left\lfloor \frac{100}{2 \cdot 3} \right\rfloor - \left\lfloor \frac{100}{2 \cdot 5} \right\rfloor - \left\lfloor \frac{100}{3 \cdot 5} \right\rfloor + \left\lfloor \frac{100}{2 \cdot 3 \cdot 5} \right\rfloor$$

Q. No. of numbers b/w 1 to n
which are divisible by any of the primes
numbers less than 20.

I/P:- 5 O/P:- 4
10 9
12 11
15 14
18 17

Algo:- $\{2, 3, 5, 7, 11, 13, 17, 19\} = 8$ elements
We will use bit masking to generate
subsets



For each subsets we can define which nos. can be taken in product as a denominator to apply in formula.

* If no. of numbers is odd we will add it to final^{ans} after dividing it by N. else we will subtract it from final answer.

Code

```

ll primes[] = {2, 3, 5, 7, 11, 13, 17, 19};
ll n;
cin >> n;
ll subsets = (1 << 8) - 1; ll ans = 0;
for (ll i = 1; i <= subsets; i++) {
    ll denom = 1;
    ll setBits = __builtin_popcount(i);
    for (ll j = 0; j <= 7; j++) {
        if (i & (1 << j)) {
            denom = denom * primes[j];
        }
    }
    if (setBits & 1) {
        ans += n / denom;
    } else {
        ans -= n / denom;
    }
}
cout << ans << endl;

```

* If we are given with max size of a variable to be 10^6 then declare global array of size 1000006. coz array size 10^7 to 10^8 declared globally (i.e. on heap) is possible. If you declare ~~arr~~ large arrays in funcⁿ or main it goes into stack (which is much smaller in size). Hence array of size 10^7 will not work. $\therefore$ larger arrays should be declared globally.

Recursion

* Its a method where solution to a problem depends on solutions to smaller instances of the same problems.

- You are calling the same function inside that function which perform function on smaller part of problem given and each of the ^{recursive} call is pushed into a recursion stack and when the function reaches constraint limit it starts returning value and pop() of each sub-functions take place.

Parts of Recursion Algo

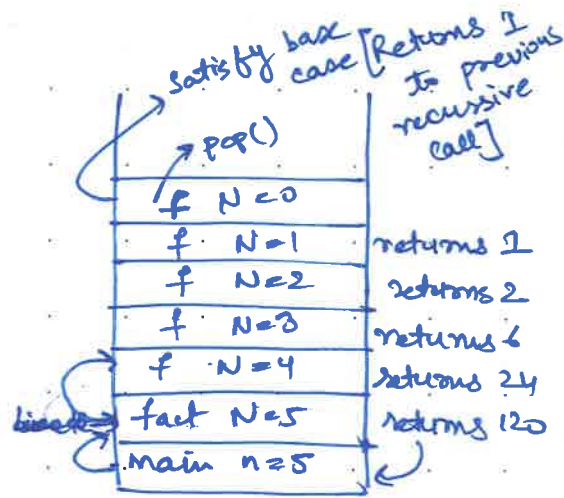
- 1) Base case
 - 2) Recursive call (to work towards Base case)
- For ex :- factorial
 $f(0) = 1$ (Base case)
 $f(n) = n \times f(n-1)$ (Recursive call)

Mechanism [Factorial]

Ex:-

```
int fact(int n){
    // base case
    if (n==0) {
        return 1;
    }
    // recursive call
    return (n * fact(n-1));
}

int main(){
    int n;
    cin >> n;
    cout << fact(n) << endl;
    return 0;
}
```

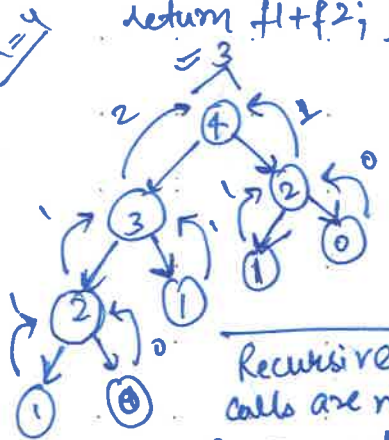


[once return is called the recursive call is popped out]

→ Memory (worst case) - $O(N)$

Mechanism [Fibonacci]

```
int f(int n){
    if (n==0 || n==1) {
        return n;
    }
    int f1 = f(n-1);
    int f2 = f(n-2);
    return f1 + f2;
}
```



Recursive calls are made in pre-order

```
int f(int n){
    if (n==0 || n==1) {
        return n;
    }
    int ans = f(n-1) + f(n-2);
    return ans;
}
```

T.C $\rightarrow O(2^n)$ Space Comp $\rightarrow O(N)$
 then recursive call stack calls are made

Recursive calls can be made in parallel manner for ease of visualisation

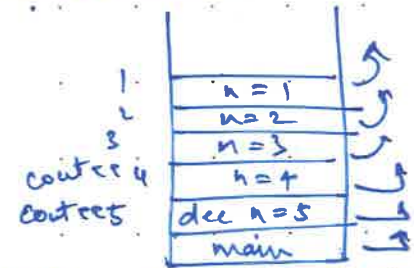
Increasing-Decreasing Recursion

```
void dec(int n){
    ① if (n==0) {
        return;
    }
    ② cout << n << " ";
    ③ dec(n-1);
}
```

O/P: - 5 4 3 2 1

```
void inc(int n){
    ① if (n==0) {
        return;
    }
    ② inc(n-1);
    ③ cout << n << " ";
}
```

O/P: - 1 2 3 4 5



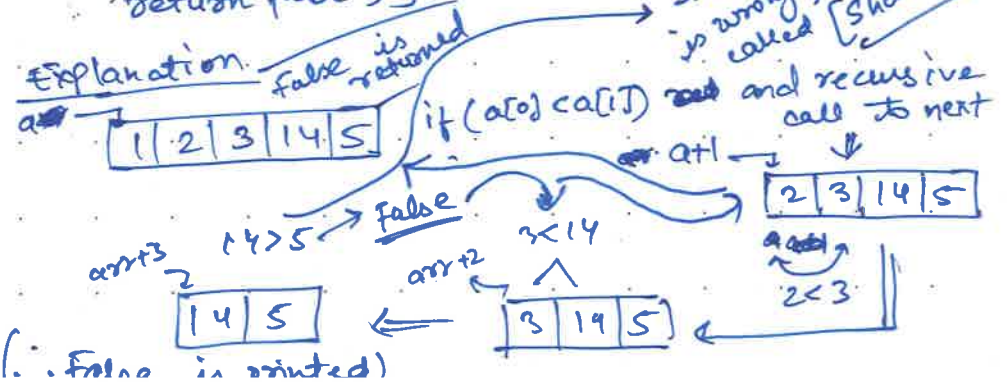
Prints and call other funcⁿ in decreasing manner and then since its void function at end of funcⁿ it starts popping out each calls.

→ Now recursive calls are made first and at end it starts printing and popping each recursive call in stack.

Recursion Array is Sorted or Not

```
bool isSorted(int *a, int n){
    if (n==0 || n==1) {
        return true;
    }
    if (a[0] < a[1] and isSorted(a+1, n-1)) {
        return true;
    }
    return false;
}
```

Explanation



(∴ False is printed)

Results in breakage of calls

Since first if condⁿ is wrong, isSorted is not called [Short circuit]

Recurrence First Occurrence

```
int firstOcc(int *a, int n, int key) {
    if (n == 0) { return -1; }
    if (a[0] == key) {
        return 0;
    }
    int i = firstOcc(a+1, n-1, key);
    if (i == -1) { return -1; }
    return i+1;
}
```

specify next call.
in the current func

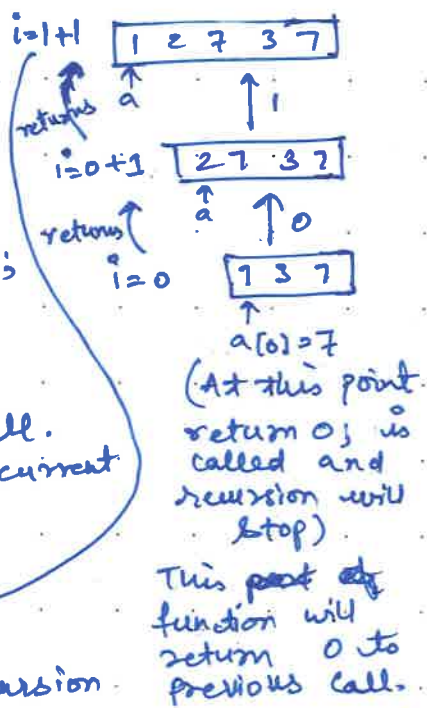
∴ 2 will be the answer

⇒ other method would be recursion in for loop fashion

```
int linearSearch(int *a, int i, int n, int key) {
    if (i == n) { return -1; }
    if (a[i] == key) { return i; }
    return linearSearch(a, i+1, n, key);
}
```

Recurrence Last Occurrence

```
int lastOcc(int *a, int n, int key) {
    if (n == 0) { return -1; }
    int i = lastOcc(a+1, n-1, key);
    if (i == -1) {
        if (a[0] == key) {
            return 0;
        } else {
            return -1;
        }
    }
    return i+1; // otherwise if i returned by subproblem is not
}
```

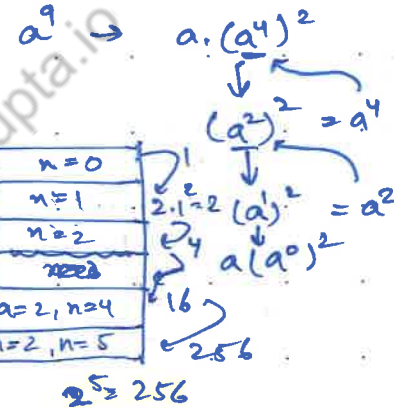


Recurrence Observation

- func () {
- (1) base cases → where recursive calls stop and returns something
- (2) recursive call → calls next subproblem
- (3) Condⁿ to be checked for each calls in stack and returned to previous calls in stack
- } (*) (3) point can be placed b/w (1) & (2) also if we want the case to be checked at every recursion alongside.

Fast Power Recursion

$a^n \rightarrow (a^{n/2})^2$ if n is even
 $\rightarrow a \cdot (a^{n/2})^2$ if n is odd.



takes $O(\log_2 n)$ time.

Code

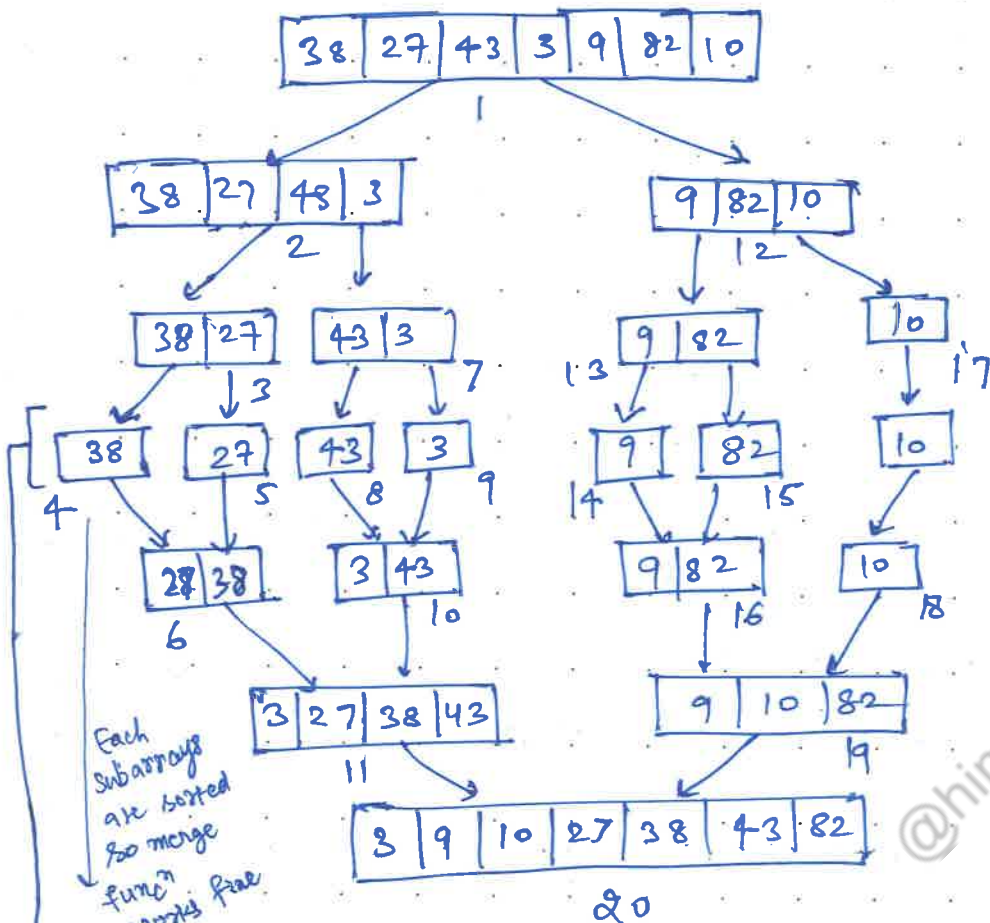
```
int fast-power(int a, int n) {
    if (n == 0) {
        return 1;
    }
    int smaller-ans = fast-power(a, n/2);
    smaller-ans *= smaller-ans;
    if (n & 1) { return smaller-ans * a; }
    return smaller-ans;
}
```

Bubble sort (Recursive)

```
void bub-sort(int *a, int j, int n) {
    if (n == 1) {
        return;
    }
    if (j == n-1) {
        return bub-sort(a, 0, n-1);
    }
    if (a[j] > a[j+1]) {
        swap(a[j], a[j+1]);
        bub-sort(a, j+1, n);
    }
}
```

base case for outer loop
 base case for inner loop.
 (when $j == n-1$ further recursion ($j+1$) stops and it returns another call with $j=0$ and $n=n-1$)

Merge Sort (Uses D&C)

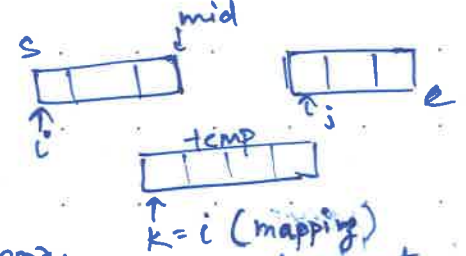


This level is the base case and thus the end point for recursive calls. When return to this previous call it call recursive funcⁿ on right sub array and ~~so~~ when it reaches end pt. we call merge func which merges the array after sorting. And each values is mapped according to the index values.

[THIS ALL HAPPENS IN POST-ORDER FASHION]
1. (Merge)

Code

```
void merge (int *arr, int s, int mid, int e) {
    int i = s;
    int j = mid + 1;
    int k = s;
```



```
    int temp[1000];
    while (i <= mid && j <= e) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }
```

```
    while (i <= mid) {
        temp[k++] = arr[i++];
    }
    while (j <= e) {
        temp[k++] = arr[j++];
    }
```

```
    for (int x = s; x <= e; x++) {
        arr[x] = temp[x];
    }
```

```
void mergeSort (int *arr, int s, int e) {
    if (s >= e) { // when single element in left
        return;
    }
    int mid = (s + e) / 2;
    mergeSort (arr, s, mid); // traverse left part (child)
    mergeSort (arr, mid + 1, e); // right part (child)
    merge (arr, s, mid, e); // function on root.
}
```

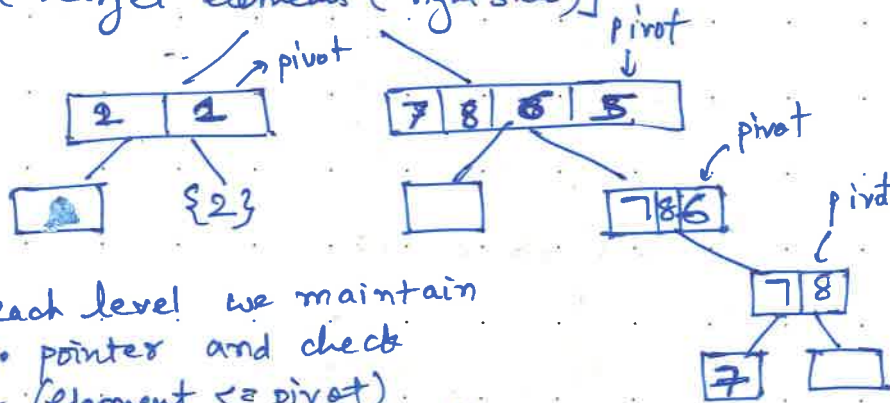
⇒ $O(n \log n)$ in all condⁿ

Let food be thy medicine and medicine be thy food.

QuickSort (uses D&C)

[2 | 7 | 8 | 6 | 1 | 5 | 4]

[We will make a partition using pivot where we divide nos. into smaller (left side) and larger elements (right side)]



At each level we maintain two pointers and check if (element $\leq$ pivot) we put it in smaller region else we do nothing.

At last we shift it to $i+1$ (where i is no. of smaller nos)

Code

```
int partition (int arr, int s, int e) {
```

```
    int i = s-1;
```

```
    int j = s;
```

```
    int pivot = arr[e];
```

```
    for (i = s; i < e; i++) {
```

```
        if (arr[i]  $\leq$  pivot) {
```

```
            i++;
```

```
            swap(arr[i], arr[j]);
```

```
        swap(arr[e], arr[i+1]);
```

```
    } return i+1; }
```

**GET HEALTH INSURANCE
WITHOUT MEDICAL
CHECK-UPS, AT ANY AGE!**



care freedom



Automatic Recharge
of Sum Insured



Short 2 year wait period
to cover Pre-existing Ailments



Covers insulin
dependent diabetes



Covers mental issues
like Depression, Autism &
Anxiety disorder



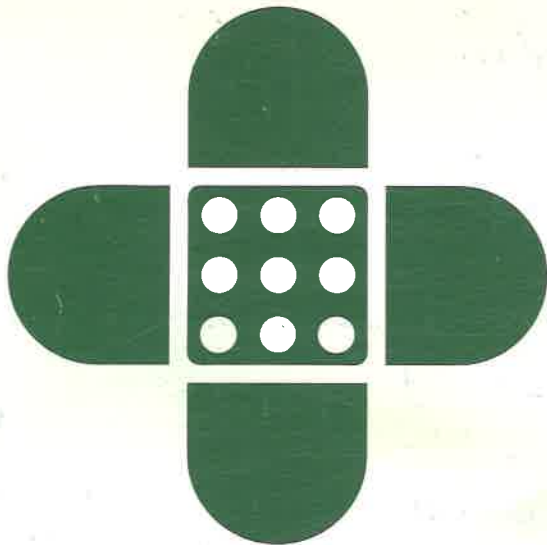
Annual Health Check-up
for all Insured members



Covers individuals with
multiple morbidities
(Hypertension, Lipid &
Obesity (BMI $\leq$ 37))

Conditions Apply

COMPREHENSIVE PERSONAL ACCIDENT COVER



Double SI payable due to
Accidental Death
in common carrier



3 Times benefit in case of
Permanent Total Disability
due to Accident



Double benefit in case of
Permanent Partial
Disability due to Accident



Fractures & Burns Cover



Child Education Cover



Reconstructive Surgery

```
void quicksort (int arr[], int s, int e) {
    if (s >= e) return;
    int p = partition(arr, s, e);
    quicksort(arr, s, p-1);
    quicksort(arr, p+1, e);
}
```

T.E.

Worst Case :- Array sorted (reverse/asc)
All elements are same.
 $O(n^2)$

This problem can also be taken care by shuffling elements in random order (random shuffle) if then

Sol:- Random index as pivot / choosing median as pivot / middle index as pivot.

Avg / Best Case :- $O(n \log n)$

Quicksort vs Mergesort

(1) Partition / Splitting can be done in any ratio (2-way, 3-way etc), not necessarily into half.

(2) Worst case :- $O(n^2)$

(3) Works well on smaller array. (Faster than others for smaller set)

(4) In-place
(requires no extra space other than stack memory)

(5) preferred for arrays.
(usage of random index access i.e. arr[5] etc. are used) (not possible in linked list)

(2) worst-case :- $O(n \log n)$

(3) works fine on any size. (maintains consistent speed)

(4) not in-place
(requires extra space)

(5) preferred for linked lists also because linked list's each node is stored at non-contiguous locations and thus

6. Quicksort is an unstable sort.

since merge-sort works sequentially it works best. 6. It is a stable sort.

Inversion Count Problem

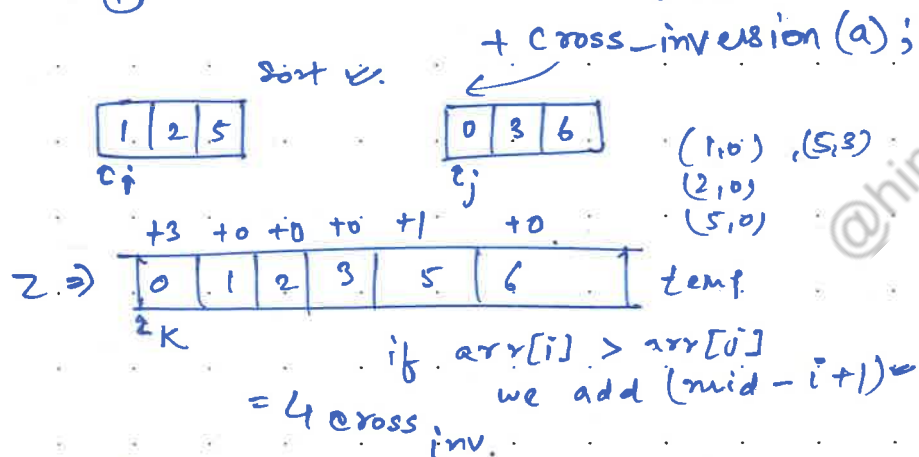
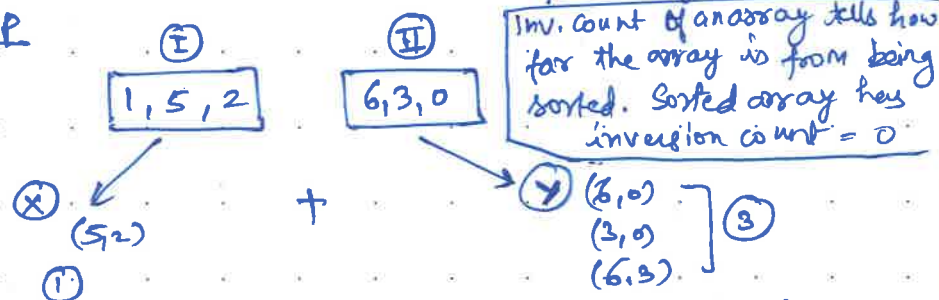
[Divide & Conquer]

arr[] = {1, 5, 2, 6, 3, 0}

(Explained better in Coding practice N.B)

Find pairs such that $a[i] > a[j]$

EXP



Code :-

```
int merge(int *a, int s, int e) {
    int mid = (s+e)/2;
    int i = s;
    int j = (mid+1);
    int k = s;
    int temp[100000];
```

int cnt = 0;

while (i <= mid && j <= e) {

if (a[i] <= a[j]) {

temp[k++] = a[i++];

else {

temp[k++] = a[j++];

cnt += mid - i + 1; }

while (i <= mid) { temp[k++] = a[i++]; }

while (j <= e) { temp[k++] = a[j++]; }

Copy it to original array;

return cnt;

int inversionCount(int *a, int s, int e) {

if (s >= e) { return 0; }

int mid = (s+e)/2;

int x = inversionCount(a, s, mid);

int y = inversionCount(a, mid+1, e);

int z = merge(a, s, e);

return x+y+z; }

String to integer

int stringToInt(char *a, int n) {

if (n == 0) { return 0; }

(Extract) int digit = a[n-1] - '0';

(call) int smallAns = stringToInt(a, n-1);

(Return) return smallAns * 10 + digit;

}

on assumption that it will have 123 as no.

Just make a image of things

"1234" → 1234
"123" → 123
123 * 10 + 4 = 1234

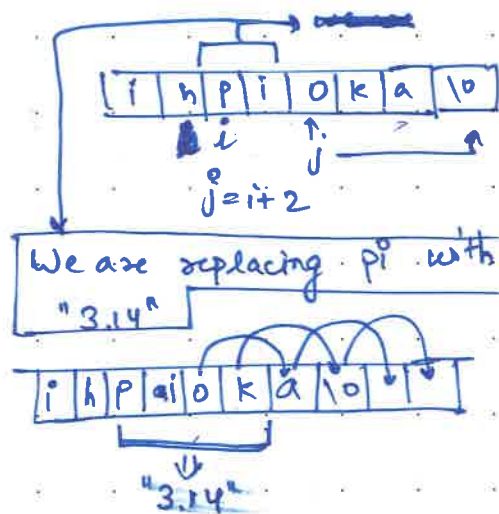
And call recursion without thinking what's gonna happen.

Replace Pi

- If we want to ~~convert~~ ~~take~~ shift the elements to a p by a constant no. to make up space for something to be added in b/w we can start shifting each element from rightmost part to left.

because when doing shifting left to right overwriting takes place which leads to loss of data.

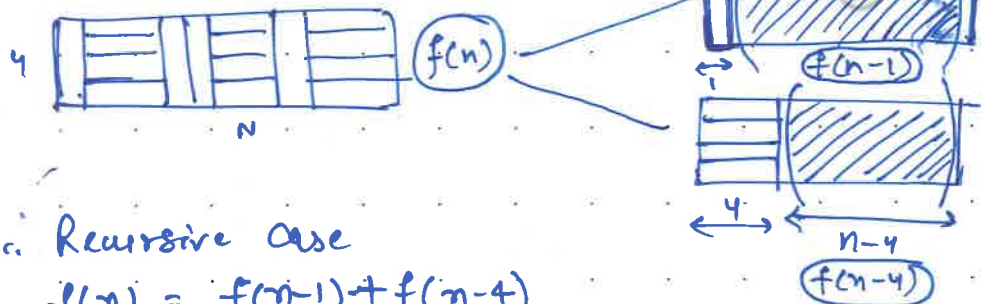
```
while(a[j] != 'p') {
    j++;
}
while(j > i+2) {
    a[j+2] = a[j];
    j--;
}
a[i] = '3';
a[i+1] = '4';
a[i+2] = '1';
a[i+3] = '4';
```



Tiling Problem

Given a wall of size of $4 \times N$, and tiles of size $(1,4)$ and $(4,1)$. In how many ways you can build the wall?

Explanation



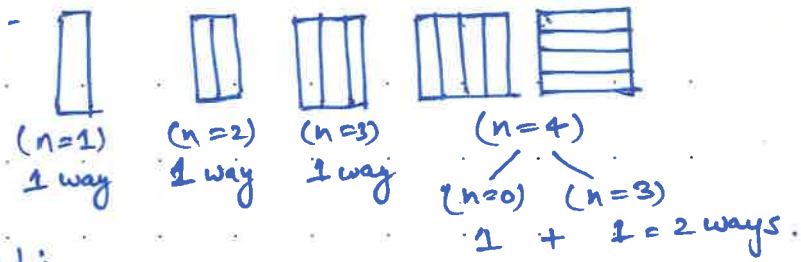
Recursive case

$$f(n) = f(n-1) + f(n-4)$$

$$f(n=4)$$

$$f(n=0) \quad f(n=3)$$

base case :-



if $(n \leq 3)$
return 1;

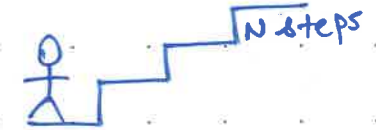
*Q can be given with wall $(n \times m)$ and you have to find no. of ways to place $(1 \times m)$ size tile

Ladder Problem

$N=4$

Jump he can take $1/2/3$ steps at a single time.

Find the ways to reach $N=4$ steps high.
o/p = 7.



Base case :-
 $N=0$:
return 1.

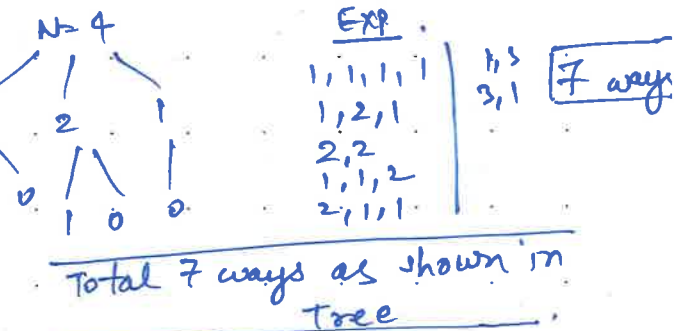
When we reach ending 0 it returns 1 which adds up to our no. of ways at each end.

Recursive case : $f(N) = f(N-1) + f(N-2) + f(N-3)$

② If $(N < 0)$: return 0 $\rightarrow$ In case we get $N < 0$ on subtracting then we return 0

$\therefore$ For k different steps

$$f(N) = \sum_{i=1}^K f(N-i)$$



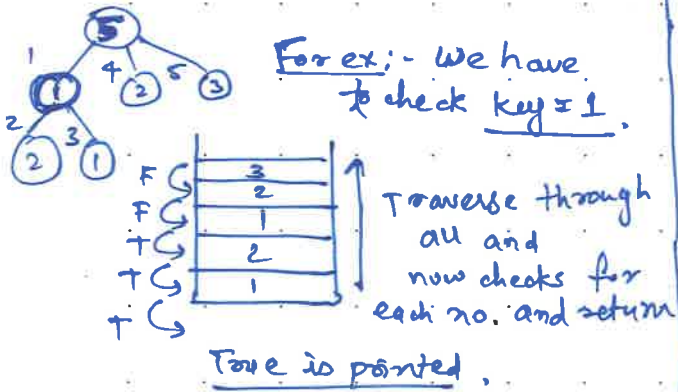
* Recursion through generic Tree [Significance of Return]

For ex. :- We are finding whether a key exist or not
if exist return true else false.

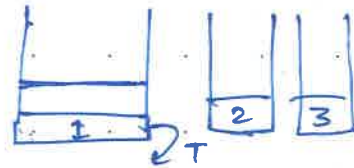
```
bool isPresent (Tree<int>* root, int x) {
    bool ans = false;
    for (int i = 0; i < root->children.size(); i++) {
        ans = ans | isPre (root->children[i], x);
    }
    if (root->data == x) { return true; }
    return ans;
}
```

```
bool :: (" ") {
    bool ans = false;
    if (root->data == x) {
        return true;
    }
    for ( ... ) {
        ans = ans | isPre ();
    }
    return ans;
}
```

For ex. :- We have
to check key = 1.



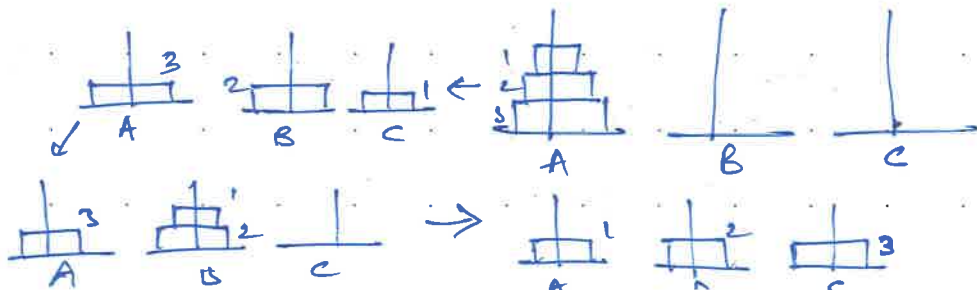
Traverse through
all and
now checks for
each no. and return



In this it will
return true as soon
as it get (key = 1).
Now it will check
other children on 1st
level.

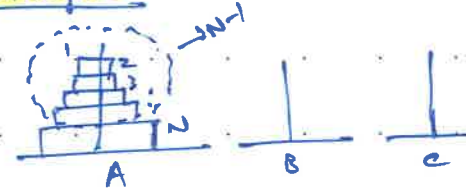
Tower Of Hanoi

Task → Move all disks to A to C
→ Pick 1 disk at one time
→ Can't place a big on small disk.

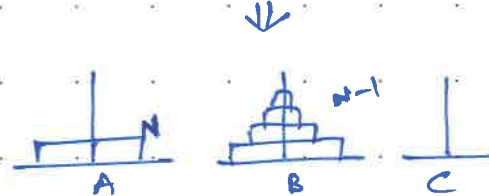


→ Total 7 steps

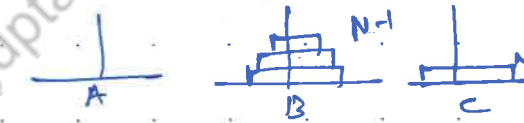
Analysis



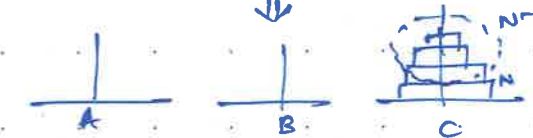
We will ~~move~~ move
(N-1) disks from
A to B.



Now after moving/
calling a f(N-1)
we will leave it
to recursion.



Now move/print
Nth block moves
from A to C.



Now move ~~(move)~~ /
Call f(N-1) from
B to C.

→ move (N-1, A, B)
→ Shift nth disk from A to C
→ move (N-1, B, C)

Code

```
void move (int n, char src, char helper, char dest) {
    if (n == 0) { return; }
    move (n-1, src, dest, helper);
    cout << "Shift disk " << n << " from " << src << " to " <<
        dest << endl;
    move (n-1, helper, src, dest);
}
```

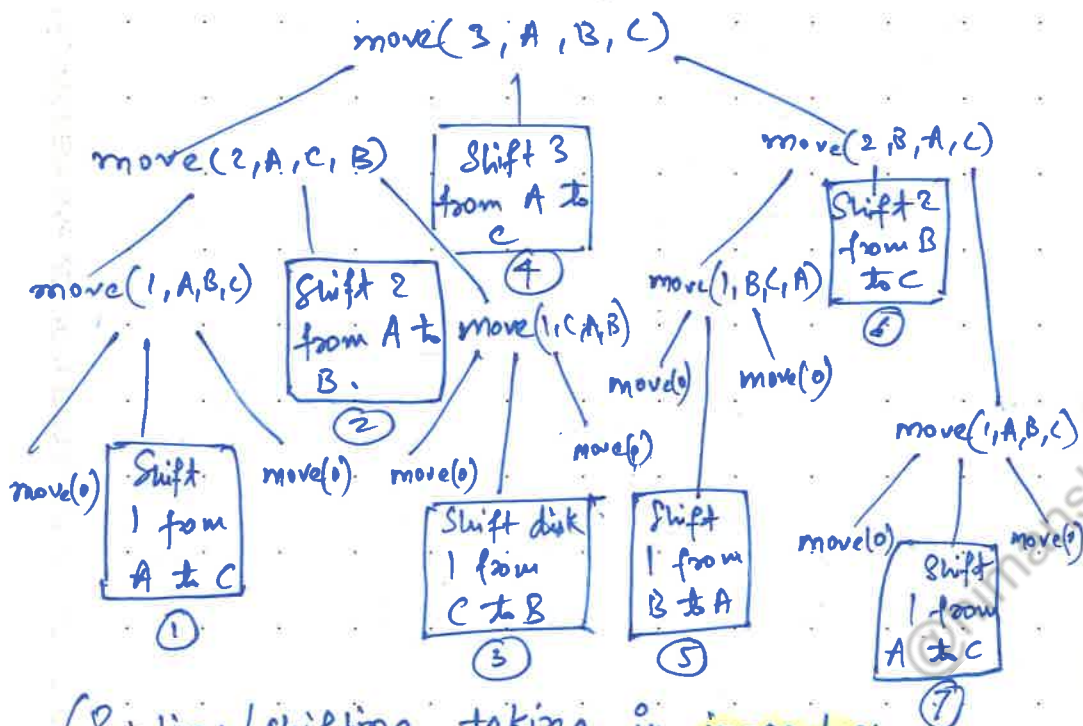
"Just because you're not sick doesn't mean you're healthy."

```

int main() {
    int n;
    cin >> n;
    move(1, 'A', 'B', 'C');
    return 0;
}

```

Recursion Representation



(Printing/shifting taking in in-order fashion)

array is sorted or not (bool & usage)

```

bool check(int *arr, int n) {
    static bool judge = true;
    if (n == 0) { return judge; }
    judge = judge & check(arr, n-1);
    if (arr[n] < arr[n-1]) { return false; }
    return judge;
}

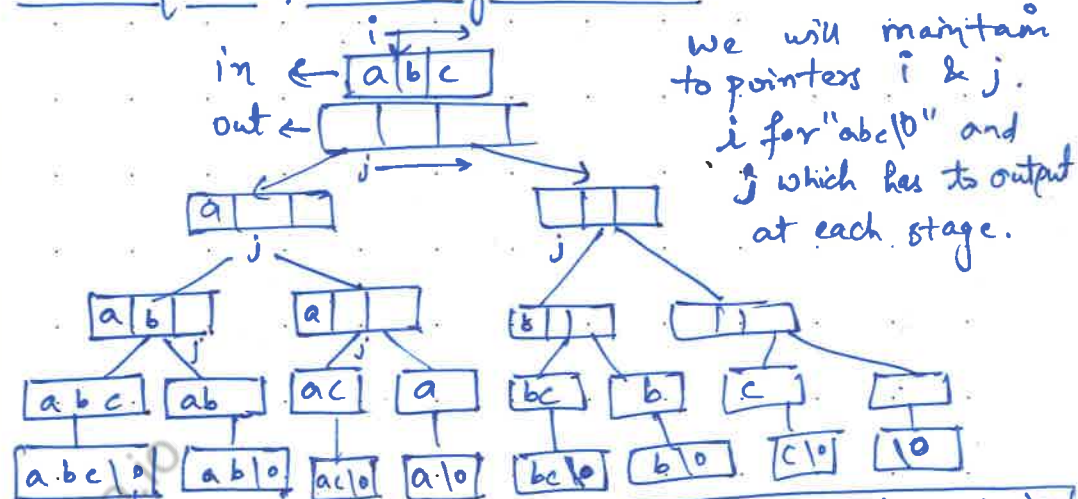
```

Static value change (it doesn't change on overwrite)

or else incoming value satisfies this return false to previous calls to get with.

Recursion (Subset Based)

Subsequences / Subsets generation



We will maintain pointers `i` & `j`. `i` for "abc|0" and `j` which has to output at each stage.

when `i` reaches last index (or `input[i] == '\0'`) we return.

Code → [Order is random using this method]

```

void subseq(char *in, char *out, int i, int j) {
    if (in[i] == '\0') {
        out[j] = '\0';
        cout << out;
        *count = *count + 1;
        return;
    }
}

```

// include the current char

`out[j] = in[i];`
 → `subseq(in, out, i+1, j+1);`
 // exclude the current char

→ `subseq(in, out, i+1, j);`

we increase `j` when we include

we don't increase `j` when we exclude.

```
int main() {
```

```
    char in[100];
```

```
    char out[100];
```

```
    int i = 0;
```

```
    int *count = &i;
```

```
    cin >> in;
```

```
    subseq(in, out, 0, 0, count);
```

```
    cout << endl << count; }
```

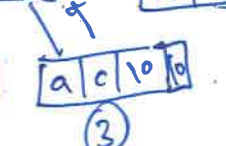
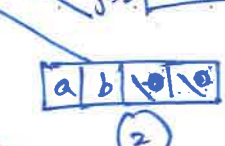
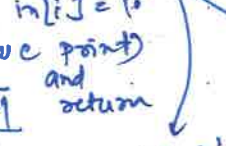
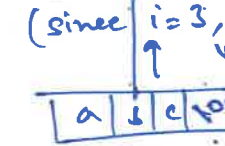
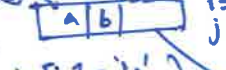
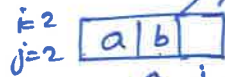
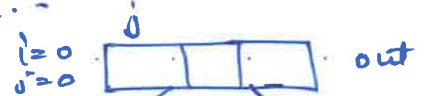
I/P:- abc

O/P:- abc . ab . ac

a bc b c ..

8

* Thing to consider that we are using out array (called by reference) So when we add an char to certain index it changes for all.



At this point it has 'a b c' stored but j is still at j=2. So we overwrite the value we are looking for. In this case we reached i=3 so j=2 we overwrite with '0'.

NOTE:- Call no. of recursions = no. of paths to be taken to solve a recursive problem

Ex:- In subsequence code we called two different recursion whether to include or exclude that no.

Generate Brackets Recursion

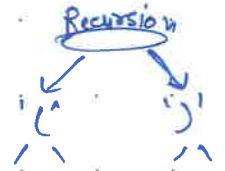
If N=2, O/P:- () () , (())

N=3, O/P:- () () () , (()) () , () (()) , (() ()) , ((()))

Code

```
void brack(char *out, int n, int idx, int open, int close) {
```

```
    if (idx == 2 * n) {
        out[idx] = '\0';
        cout << out << endl;
        return;
    }
```



```
    if (open < n) {
        out[idx] = '(';
        brack(out, n, idx+1, open+1, close);
    }
```

```
    if (close > open) {
        out[idx] = ')';
        brack(out, n, idx+1, open, close+1);
    }
```

* if first condition holds true and second also holds true that means close > open < n and thus both recursions can be called in a function leading to two ways with including '(' or including ')'

0-1 Knapsack Problem

Given weight and values of n items, put these items in a knapsack of capacity w to get the maximum total value in the knapsack. Such that sum of weights $\leq$ Total Knapsack wt.

- 0-1 tells us that either pick the complete item or don't pick it (0-1 property)

Ex:- value $V = \{60, 100, 120\}$ $w = 50$
weight $W = \{10, 20, 30\}$

O/P :- 220

Algo $\rightarrow$ array of weight size $\rightarrow$ Capacity

Profit (N, c)

max $\left\{ \begin{array}{l} \text{include} = \text{value}_{n-1} + \text{profit}(N-1, c - \text{wt}_{n-1}) \\ \text{exclude} = 0 + \text{profit}(N-1, c) \end{array} \right.$ where $c \geq \text{wt}_{n-1}$

$\rightarrow$ Base case if $(N=0 \parallel c=0) \{ \text{return } 0; \}$

Code

```
int profit(int N, int C, int *wt, int *prices) {
    if (N==0 || C==0) { return 0; }
    int ans = 0;
    int inc, exc;
    inc = exc = 0;
```

if $(\text{wt}[N-1] \leq C)$ $\{$
inc = prices[N-1] + profit(N-1, C - wt[N-1], wt, prices); $\}$

exc = profit(N-1, C, wt, prices);

ans = max(inc, exc);

return ans; } $\rightarrow$ returns max of exc & inc at each level.

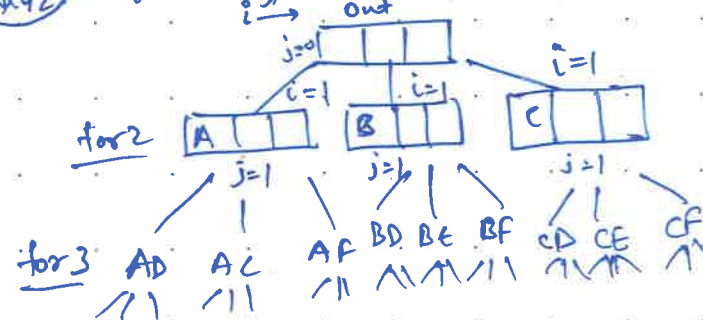
Phone-Keypad Problem



EX :- If no. = 23

$\begin{bmatrix} AD & BD & CD \\ AE & BE & CE \\ AF & BF & CF \end{bmatrix}$ will be generated (9 subsets)

If no. = 23346



Code

```
char keypad[10] = {"", "1", "2ABC", "3DEF", "4GHI", "5JKL", "6MNO", "7PQRS", "8TUV", "9WXYZ"};

void generate_names(char *in, char *out, int i, int j) {
    if (in[i] == '\0') {
        out[j] = '\0';
        cout << out << endl;
        return;
    }
```

Those who think they have no time for exercise will sooner or later have to find time for illness.

```

int digit = in[i] - '0';
if (digit == 1 || digit == 0) {
    generate_names(in, out, i+1, j);
    for (int k=0; keypad[digit][k] != '\0'; k++) {
        out[i] = keypad[digit][k];
        generate_names(in, out, i+1, j+1);
    }
    return;
}

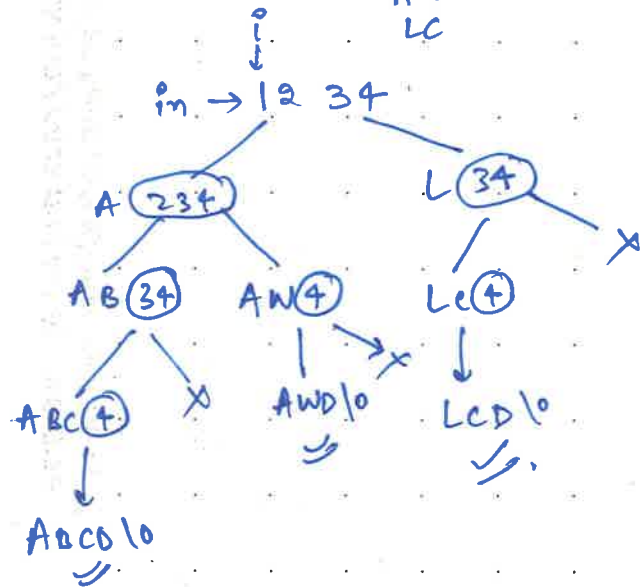
```

→ it will take care of filling the rest.

String Generation Acode

I/P: 123 d/p: - ABC
 AW
 LC

exp:- 1' 2' 3' = ABC
 1' 23' = AW
 12' 3' = LC



Code

```

void gen_strings(char *in, char *out, int i, int j) {
    if (in[i] == '\0') {
        out[j] = '\0';
        cout << out << endl;
    }
}

```

```

// one digit at a time
int digit = in[i] - '0';
char ch = digit + 'A' - 1;
out[i] = ch;
gen_strings(in, out, i+1, j+1);
}

```

// two digit at a time

```

if (in[i+1] != '\0') {
    int sec_digit = in[i+1] - '0';
    int no = digit * 10 + sec_digit;
    if (no <= 26) {
        ch = no + 'A' - 1;
        gen_strings(in, out, i+2, j+1);
    }
}
return;
}

```

• Move all X to End

I/P: - a x b x c O/P: - abcxx [Do inplace-shifting]
 (Code on coding blocks)

• Subset Sum

I/P: - 4
 1 3 2 -3

O/P: - Yes

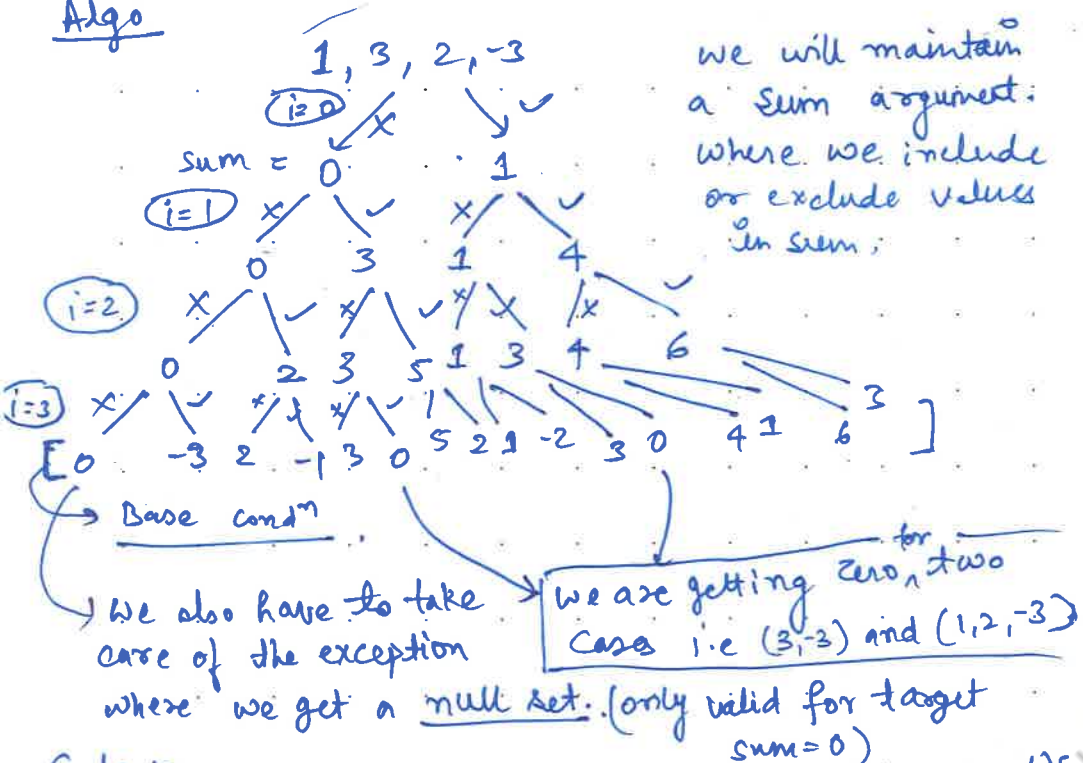
Explanation:-

There exists a subset for which
 $3 + (-3) = 0$ i.e. {3, -3} and {1, 2, -3}

Logic

We have to find subsets with a target sum. In this, target sum = 0

Algo



Code :-

```
void subset(int *arr, int n, int i, int sum, int *count){
    if (i==n){
        if (sum==0){
            *count = *count + 1;
            return;
        }
        return;
    }
    subset(arr, n, i+1, sum+arr[i], count);
    subset(arr, n, i+1, sum, count);
    return;
}
```

* We will get count=3 for 2+1, exception case

Overlapping Subproblem

property - When many subproblems are

called more than once.

same

then T.C of program varies

exponentially. In that case we

not take tabulation / Memoization DP.

Recursion - BackTracking

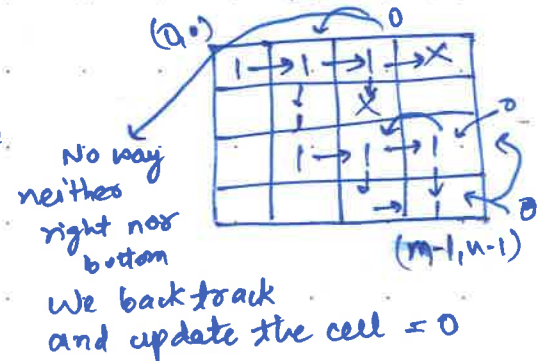
Rat In a Maze

→ Find path from src to dest

→ Count no. of paths

→ Print all possible paths from src to dest:

- As many times we are reaching base case i.e. (m-1, n-1) we backtrack and overwrite the cell = 0 so that our path comes to be correct (No redundancy)



Code

```
bool RatInMaze(char maze[10][10], int soln[10][10],
    int i, int j, int m, int n){
    if (i==m && j==n){
        soln[m][n] = 1;
        // Print the path
        for (int i=0; i<=m; i++){
            for (int j=0; j<=n; j++){
                cout << soln[i][j] << " ";
            }
            cout << endl;
        }
        return true;
    }
}
```

// rat should be inside grid

if (i>m || j>n) return false;

if (maze[i][j] == 'X') return false;

// Assume sol exists through current cell.

`soln[i][j] = 1;`

`bool rightSuccess = ratInMaze(maze, soln, i, j+1, m, n);`

`bool downSuccess = ratInMaze(maze, soln, i+1, j, m, n);`

// backtracking - When we start returning
→ `soln[i][j] = 0;` after hitting base case

`if (rightSuccess || downSuccess) {
 return true; }`

`return false;`

`int main() { char maze[10][10] = { "0000",
 "00x0",
 "000x",
 "0x00" };

int soln[10][10] = {0};
int m = n = 4;
bool ans = ratInMaze(maze, soln, 0, 0, m-1, n-1);
if (ans == false) { Path doesn't exist; }`

Recursion - Observation (Pt-2)

`func () {`

- ① → Any return condition
→ changes to be made in array etc.

[Recursive calls

- ② → Any return condition
→ changes to be made in array etc.

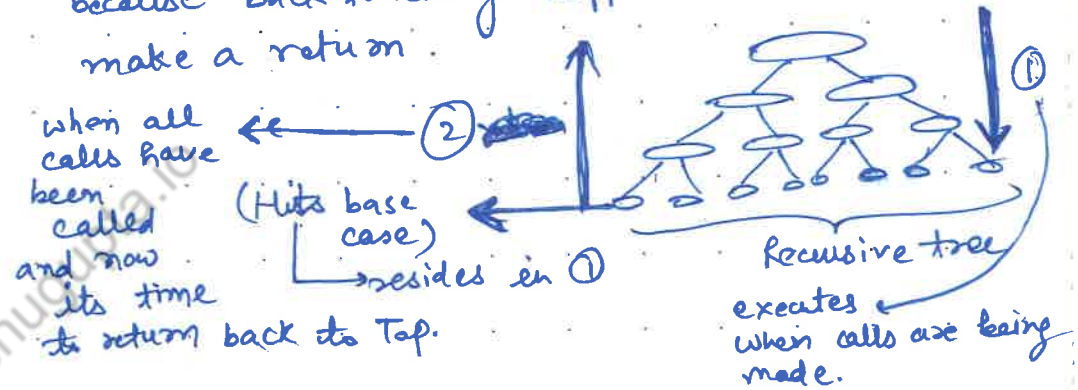
bottom
(where calls
are being
made)

Top
(when all
calls have
been made and
returning)

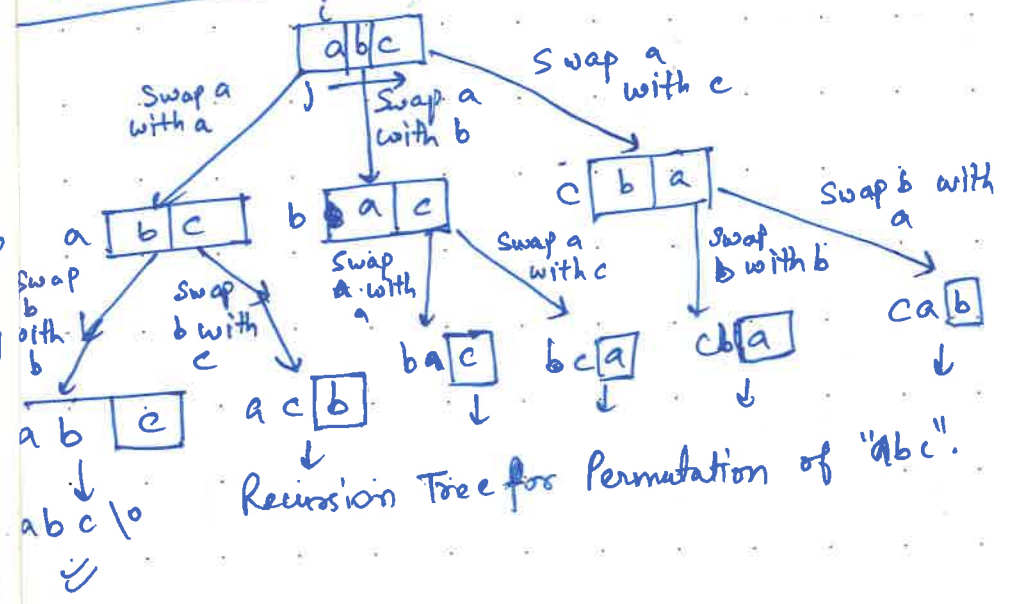
⊗ When the calls are going to bottom, code

① will be executed and ~~when~~
after hitting base case which lies
in code ① ~~the~~ the calls return & pops
and then executes code ② while going
to Top. [Top & bottom are dir of recursive tree
movement]

Above statement is important for backtracking
because backtracking happens while calls
make a return.



Permutations



Recursion Tree for Permutation of "abc".

NOTE: - Since we do swapping at each call we are changing the ~~arr~~ original array and thus on further swaps we will get redundant (missing/wrong) permutations. So on going to top, at each level, we will perform reverse swap to restore original array at each level.

Code

```
void permute (char *in, int i) {
    if (in[i] == '\0') {
        cout << in << " ";
        return;
    }
    for (int j = i; in[j] != '\0'; j++) {
        swap(in[i], in[j]); // swapping
        permute(in, i+1);
        swap(in[i], in[j]); // BackTracking (reverse swap on returning back to top) - To restore original array
    }
}
```

* Set can be used to contain strings in
→ lexicographic order
→ Unique manner.

Unique permutations

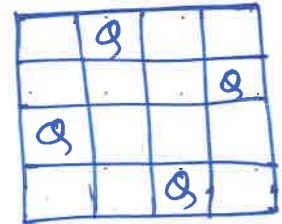
```
void permute (char *a, int i, set <string> &s) {
    if (a[i] == '\0') {
        string t(a); // casting
    }
```

```
S.insert(t);
return;
for (int j = i; a[j] != '\0'; j++) {
    * Same *;
}
int main() {
    set <string> s;
    pass it }
```

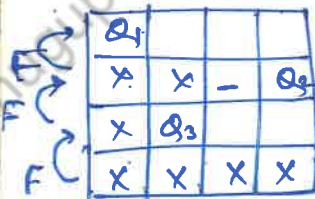
Works for
I/P :- aba
O/P :- aab, aba, baa

N-Queen Problem

Place N Queens such that all queens are safe.



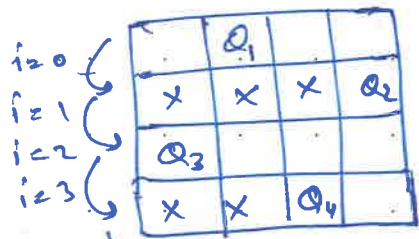
Algo



when there is no Queen position in a row we return false.

- Now third row will check if there is any other possible position for Q3. but there isn't so it will also return false.
 - Q2 is already at last posⁿ in its row. so it will also return false.
- Eventually Q1 moves a posⁿ in its row to check if now there is any configuration which holds true for Q1's new posⁿ.

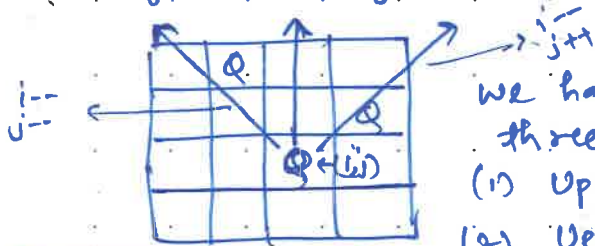
"If you don't take care of your body, where are you going to live?"



Now all rows satisfy the condition very well and no row is compromised.

Code
i=4=N (base case)

Q How we will check if each posⁿ (i,j) is safe or not?



We have to check for three directions -
(1) Upper left diagonal
(2) Upper column
(3) Upper right diagonal

* There is no need to check in lower dirⁿ

Code

```
bool isSafe (int board[][10], int i, int j, int n) {
    // check upper col
    for (int row = 0; row < i; row++) {
        if (board[row][j] == 1) {
            return false;
        }
    }
    // Check upper left diagonal
    int x = i;
    int y = j;
    while (x >= 0 && y >= 0) {
        if (board[x][y] == 1) {
            return false;
        }
        x--;
        y--;
    }
}
```

// Check for upper right diagonal

```
x = i; y = j;
while (x >= 0 && y < n) {
    if (board[x][y] == 1) {
        return false;
    }
    x--;
    y++;
}
```

return true; // posⁿ is safe.

bool solve NQueen (int board[][10], int i, int n) {

if (i == n) {

// You have successfully placed queens in n rows (0, ..., n-1).

for (int i = 0; i < n; i++) {

for (int j = 0; j < n; j++) {

if (board[i][j] == 1) {

cout << "Q ";

else { cout << "- "; }

cout << endl;

} return true;

// Try to place the queen in the current row and call on the remaining part which will tell us

for (int j = 0; j < n; j++) {

if (isSafe(board, i, j, n)) {

// place the Queen, assuming i, j is right posⁿ.

board[i][j] = 1;

bool nextQueenRakhPaye = solve NQueen (board, i+1, n);

if (nextQueenRakhPaye) { return true; }

// Means i, j is not the right posⁿ - Assumption is wrong

board[i][j] = 0; // BackTrack

return false; // No ans found

return false if we want to print all answers

```

int main() { int n;
             cin >> n;
             int board[10][10] = {0};
             solveNQueen(board, 0, n); }

```

In above code, if we return true at base case it will print only 1st config. because we are checking for each position in 1st row.

The moment we find the config to be true, we return true and come out of funcⁿ.

~~either false, or false~~ This is happening because for each iteration of calls we are checking & returning true so in case of true it will not iterate further. for other posⁿ in 1st row.

Sudoku Solver

Given a $N \times N$ Sudoku, N is a perfect square, write an algo to find possible soln.

If $N \times N$ is grid-size
the $\sqrt{N} \times \sqrt{N}$ is subgrid size

Algo (logic)

- (i) We will apply brute force.
- (ii) First, we will solve first row (row-wise method).
- (iii) At first blank space, we insert any value which

doesn't violate Sudoku rule.

(iv) We are assuming that inserted value can give us ~~one~~ the soln.

After filling this blank ~~space~~ cell we move onto next where we insert one more value (assumed).

And then call recursion on rest of the grid.

(v) In case, if we didn't get the solution this recursive call will return me False.

(vi) Now we change the assumed inserted value (i.e 7) to a different value that satisfy the rule.

(vii) In case we have no number to change now. we return False to previous inserted value (i.e 1) and then we change it and again proceed to further cells and so on.

(viii) If we get our soln, ~~we return~~ recursive calls on rest of grid returns true to 7 and then 7 returns true to 1. and we end the recursion.

Rules

- (i) No same no. in that row or column
- (ii) No same no. in that subgrid.

code

```
bool canPlace (int mat[3][9], int i, int j, int n, int number) {
    // Check rows & column
    for (int x=0; x<n; x++) {
        // Row & Col. check
        if (mat[x][j] == number || mat[i][x] == number) {
            return false;
        }
    }
}
```

```
int rn = sqrt(n);
int sx = (i/rn)*rn;
int sy = (j/rn)*rn;
// Check subgrid
for (int x=sx; x<sx+rn; x++) {
    for (int y=sy; y<sy+rn; y++) {
        if (mat[x][y] == number) {
            return false;
        }
    }
}
return true;
}
```

Diagram illustrating subgrid calculation:

For $i=4, j=5$, $rn=3$, $sx=3, sy=3$. The subgrid is the 3x3 area starting from (sx, sy) .

```
bool Sudoku (int mat[3][9], int i, int j, int n) {
    // Base case (when we traverse all rows)
    if (i==n) {
        // Print matrix
        return true;
    }
    // Case Row ended
    if (j==n) {
        return sudoku(mat, i+1, 0, n);
    }
    // Skip the pre-filled cells
    if (mat[i][j] != 0) {
        return sudoku(mat, i, j+1, n);
    }
}
```

0	0	1	2
1	0	1	2
2	0	1	2

Subgrid (1,1)

For ex: If $(i,j) = (4,5)$ then

$$\left(\frac{4}{3}\right) = 1 \Rightarrow 1 \times 3 = 3$$

$$\left(\frac{5}{3}\right) = 1 \Rightarrow 1 \times 3 = 3$$

$\therefore (sx, sy) = (3, 3)$

// Rec Case

// Fill the current cell with possible solutions

```
for (int number=1; number<=n; number++) {
    if (canPlace(mat, i, j, n, number)) {
        // Assume
        mat[i][j] = number;
        bool couldWeSolve = sudoku(mat, i, j+1, n);
        if (couldWeSolve == true) {
            return true;
        }
        // Backtrack here
        mat[i][j] = 0;
        return false;
    }
    // Backtrack while going previous cell.
}
```

Diagram illustrating backtracking:

For the current cell, we try numbers 1 to n. If a number works, we move to the next cell. If not, we backtrack and try the next number.

Sum it up - (Assignment) / challenges

You are given an array and you have to find unique combinations in array whose sum equals to Target sum.

Code

```
bool subseq (int arr, int out, int i, int j, int target, int n, int sum) {
    if (i==n) {
        if (sum==target) {
            return true;
        }
        out[i] = arr[i];
        bool inc = subseq(arr, out, i+1, j+1, target, n, sum);
        out[i] = 0;
        // BackTrack 1 -> we assign 0 on returning so that it doesn't reflect on exclude recursion part.
        bool exc = subseq(arr, out, i+1, j, target, n, sum);
        out[i] = 0;
        // BackTrack 2 -> we assign 0 so that it doesn't reflect on other parts.
        return inc || exc;
    }
}
```

Diagram illustrating subsequence generation:

For $i=0, j=0$, we include $arr[0]$ and move to $i=1, j=1$. If we exclude $arr[0]$, we move to $i=1, j=0$.

Object Oriented Programming

OOP aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access the data except that function.

Characteristics - Class, Abstraction, Polymorphism, Inheritance, Encapsulation, Object.

Ex:- We have a blueprint of car → Class
Actual car in real-world → object
It has features like - model no, name, price etc

```
class Car {
    int price;
    int model-no;
    char name[20];
}
```

```
int main() {
    car Car c; // creating obj
    Car arr[20]; // array of class car
    28 ← sizeof(c);
    28 ← sizeof(Car);
}
```

It tells us the size object will contain but actually not consumes space itself (Cuz its a class)

Data Members and Functions

Class

→ Data Members

→ functions related to that object

ex:- [update-price], [apply discount] etc.

⊛ Now when we define a class, by default the data members can't be updated outside the class because the data members in class is set to private by default.

↳ [Access Modifier] ←

So we need to set the class to public.

class Car {

⊛ Public!

float ~~car~~ price;
int model-no;
char name[20];

o/p:-

BMW Model no 112 Price 100

FO → Discounted price

int main() {

Car c;

C.price = 100;

C.model-no = 112;

C.name[0] = "BMW";

cout << c.name << "Model no"

<< c.model << "Price" <<

C.price << endl;

cout << c.get-discounted-price

(0.30);

float get-discounted-price(float x) {
return price * (1.0 - x); }

float apply-discount(float x) {
price = price * (1.0 - x);
return price; }

→ This will update the price value itself.

30% of discount

"From the bitterness of disease man learns the sweetness of health."

```
void print() {
    cout << name << endl;
    cout << model-no << endl;
    cout << price << endl; }

```

```
int main() {
    car c.print(); }

```

o/p:- BMW
112

70 → After updating the price using apply-discount(0.50)

Getters & Setters

In case you don't want to allow someone to change data members/variables outside the class.

We use private on that specific variable.

class Car {

private:

~~float~~ float price;

public:

int model-no;

char name[20];

void set-price(^{float} p) {

if (p > 0) {

price = p; }

float get-price() {

return price; }

int main() {

c.set-price(79);

Hitans = c.get-price; }

Inside class we can define a function which will help us change or print the value of price.

setters

getters

* In this manner we can safeguard the variables from an unauthorised change to a negative value outside the class.

• Function in class which is used to set values of class variables is called Setters and

Class function to get the class variables value is getters.

* Some special ~~features~~ functions that C++ classes give you when you create a class/object.

- 4 different special fns that are always present inside a class :-

1. Constructor - responsible for creating an object

2. Copy constructor -

3. Copy assignment operator -

4. Destructor -

Constructor & Parameterised constructor

- gets called automatically and only once.

- Same name as class name

So whenever you ~~initialize~~ create a class object, Constructor is being called.

for ex:-

class Car {

{

Car() {

cout << "Inside Car constructor" << endl; }

→ // parameterised constructor

~~car~~ Car(float p, int m, char *n) {

price = p;

model-no = m;

strcpy(name, n); }

→ Assignment

??

```
int main(){
```

```
    Car c, e;  
    Car d(222, 333, "Audi");  
    d.print();  
}
```

Qp:- Inside car constructor → c was created
Inside car constructor → e was created
Name Audi
Model No 333
Price 222] → Parameterised Constructor being called. Constructor was called both times

* Parameterised Constructor is used to set ^{assign} values for each class object's variables directly. unlike defining it individually in main function. and it is also safe because we are calling a class function to ~~set~~ assign values.

• So in above car class we used two ^{at same time} same class constructor for different purpose, and is called Constructor Overloading.

Copy constructor

- make a copy of the given object
- exists by default

Car e(d); → ~~copy~~ assign the same values as object d.

e.set-price(456); → we can change the value after copying also
or

Car e = d;

Code (for copy constructor) (we write in class)

Car(Car &x) { we pass the object by reference
 price = x.price;
 model-no = x.model-no;
 strcpy(name, x.name); }

Q. Why pass by ref. only?

A. If the argument is passed by value, its copy constructor would call itself to copy the a actual parameter to formal parameter. This process goes on till the system runs out of memory. That's why we use call by reference so that copy constructor doesn't get invoked.

Shallow & Deep Copy.

When shallow copy has been made we use deep copy in place of it. in order to eliminate data inconsistency.

This case occurs only when we store data such as string in heap memory and rather passes a pointer to assign the object's variable.

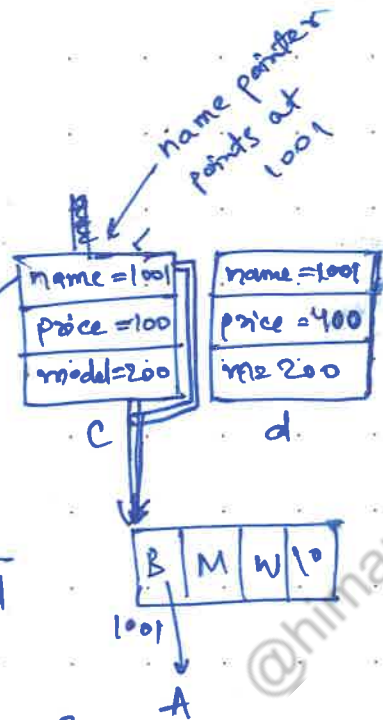
Explanation

```
class Car {  
    private: int float price;  
    public: int model-no;  
    char *name; → pointer
```

```
Car() { name = NULL; }
```

```
Car(float p, int m, char *n) {
    price = p;
    model = m;
    name = new char[strlen(n)+1];
    strcpy(name, n);
};
```

```
int main() {
    Car c(100, 200, "BMW");
    Car d(c);
    d.name[0] = 'A';
    d.set price(400);
}
```



* Now when this is executed it changes 'B' to 'A' at 1001th location.

and if we do

```
{ c.print();
d.print(); }
```

 we will get name as "AMW" in both cases.

* This above ~~copy~~ copy is called Shadow Copy.

∴ Thus we do Deep copy.
// Updated copy constructor.
Car(Car &x) {
 price = x.price;

```
model-no = x.model-no;
```

```
// deep copy
name = new char[strlen(x.name)+1];
strcpy(name, x.name); }
```

we defined another dynamic array so that previous one remain unaffected.

Copy Assignment Operator

- Copies when you do assignment

→ Car d = c; // Copy constructor

→ Car d;

d = c; // Copy assignment operator (by default)

```
void operator=(Car &x) {
```

```
    price = x.price;
```

```
    model-no = x.model-no;
```

```
    // deep copy
```

```
    name = new char[strlen(x.name)+1];
```

```
    strcpy(name, x.name);
```

```
}
```

Destructor

- clear out the memory occupied by the object
- automatically when the object goes out of scope.
- except for the cases when you dynamically allocated memory.

Destructor func can be defined like

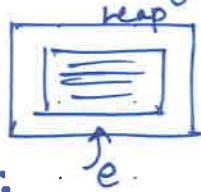
```
~Car() {
    if(name != NULL) {
        delete [] name;
    }
}
```

for static array you don't do anything. for dynamic array

* Car c(100, 200, "BMW"); // static car object
 * car *e = new Car(10, 20, "Ferrari"); ^{destructor deletes it auto.}

// Dynamically ~~created~~ created car

* You have to delete dynamically created object because



• object is stored in heap memory.

• e → print();

delete e; → // to delete the e

* We must have to define our own destructor in case data member resides in heap memory (dynamically allocated).

①
②
③ ↑ delete order

Initialization list, Consts

Const data members = Once Init, they can't be changed. It is read only variable. (x++, x=10; not allowed)

① Const int x = 5; → Initialization ✓

② const int x;
x = 5; → Assignment. X

* While using const we can only initialize the const variable. assignment is not allowed.

```
# Car(): msp(99) {
```

Initialization list

- It lets you initialize data members with function definition only.

```
# Car(float p, int m, char *n, int minprice)
```

```
{ : price(p), msp(minprice) }
```

Const int msp;

```
# Car(Car &x) : msp(x.msp) {
```

// we want to (minimum selling price) fix

• Constant parameters - parameters that have been passed but actually is not being changed in any manner in the function.

```
# Car(const Car &x) {
```

we didn't change it so we declare it const

```
# float get-discount(const float x) {
```

```
return price*(1.0-x);}
```

Benefit → It makes your code sophisticated at industry level and debugging becomes easy.

Const functions - which don't update data members of the class.

```
# void print() const { }
```

```
# float get-price() const {
```

```
return price; }
```

"True healthcare reform starts in your kitchen."

Q How debugging becomes easy using const.?

For ex:- There are 8 const function and 2 non-const func. We got a price change from 100 to 90. We need to check the error. In this case, we need to check only 2 functions which potentially have made the changes.

Generic Programming

Can be done on two levels

- Algorithm (Any func)
- containers (ex. vector)

Generic Function - works for all types of data types. We can create by ourselves using template.

Linear Search (Generic function)

template < typename T >

int Search (T arr[], int n, T key) {

- Normal linear function which returns index - }

Now we can use this function with any data type for arr, key.

STL Containers

① Sequence containers - implement d.s. which can be accessed in a sequential manner.

- vector • list • deque • arrays • forward list
- ↓ (DLL) in STL ↓ (SLL) in STL

② Containers adapters - provide a different interface for sequential containers.

- queue • priority-queue • stack

↓
(~~array~~ • follows)
heap D.S

③ Associative Containers: implement sorted D.S. that can be quickly searched ($O(\log n)$ complexity).

- set • multiset • map • multimap

④ Unordered Associative Containers - implement unordered D.S. that can be quickly searched

- unordered-set • unordered-multiset
- unordered-map • unordered-multimap

Iterators - Iterators as an entity that helps us to access the data within container with certain restrictions.

- Input Iterators - An entity through which you can read data from container and move ahead. Ex. Keyboard (Read only)
- Output Iterators - An entity through which you can write into the container and move ahead. (write) eg. printer
- Forward Iterators - input & output iterators in single direction.

Eg. SLL.

- Bidirectional - Forward Iterator that can move in both directions.
eg. DLL.

- Random access Iterators - read/write in both dirⁿ & also can take jumps.
eg. Array.

Q. Why to use Iterators?

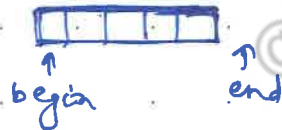
→ We made search() func to work on all data-types using Templates.

→ Now we can make search() function to work on all types of containers using Iterators.

Code (Can work with any data type/container)

```
template < class ForwardIterator, class T >
ForwardIterator search(ForwardIterator start, ForwardIterator end, T key) {
```

```
    while (start != end) {
        if (*start == key) {
            return start;
        }
        start++;
    }
    return end;
}
```



int main() {

list<int> l;

l.push_back(1); l.push_back(2); ... 3 & 5

auto it = search(l.begin(), l.end(), 5);

cout << it << endl;

O/P:- 5

* STLs are written in this manner.

Comparator class

If you want to compare two class objects you have to define comparator function, class.

#Code

```
class BookCompare {
public:
    bool operator()(i) {
        cout << "Inside compare fn";
    }
    bool operator()(Book A, Book B) {
        if (A.name == B.name) {
            return true;
        }
        return false;
    }
};
```

int main() {

Book b1 ("C++", 100); // old ed

Book b2 ("Python", 120);

Book booktofind ("C++", 110); // new edition

BookCompare cmp;

if (cmp(b1, booktofind)) {
 cout << "same books";
 }

#Code

```
template < class FI, class T, class Compare >
```

```
FI search(Forward FI start, FI end, T key, Compare cmp) {
```

```
    while (start != end) {
```

```
        if (cmp(*start, end)) {
            return start;
        }
```

```
        start++;
```

```
    }
    return end;
}
```

Complete search
Generic
function

• auto it = search(l.begin(), l.end(), booktofind, cmp);

operator overloading

for ex.

BookCompare cmp;

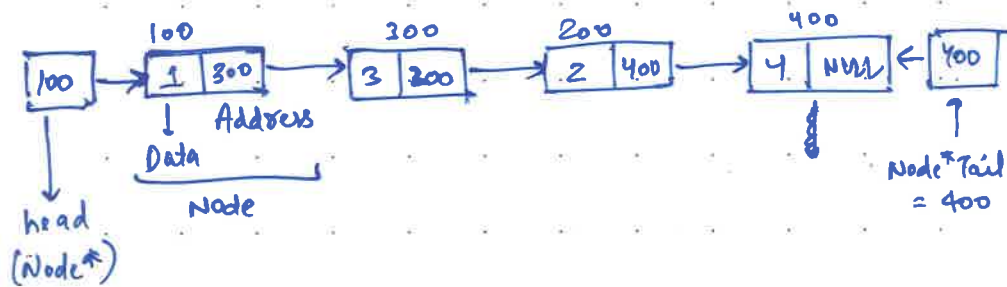
↓
It's an object but it will call

It looks like a function call

Function or functional object

Linked List

It's a linear data structure, in which the elements are not stored at contiguous memory locations. Therefore, it is beneficial to use LL when memory locations of each node are scattered at random locations.



Insertion - 1

```

class Node {
public:
    int data;
    Node* next;
    // constructor
    Node(int d) {
        data = d;
        next = NULL;
    }
}
  
```

// using functions (procedural programming)

```

① void insertAtHead(Node*& head, int d) {
    if (head == NULL) {
        head = new Node(d);
        return;
    }
  }
  
```

ways of passing by reference

- ① f(Node*& head)
- ② f(Node** head)

↪ f(&head)

```
Node* n = new Node(d);
```

```

n -> next = head;
head = n;
}
  
```

OR

```

② void insertAtHead(Node** head-ref, int d) {
  
```

```

    Node* newNode = new Node(d);
  
```

```

    newNode -> next = (*head-ref);
  
```

```

    (*head-ref) = newNode;
  }
  
```

```
void print(Node* head) {
```

```

    while (head != NULL) {
        cout << head->data << " -> ";
        head = head -> next;
    }
    cout << "NULL";
}
  
```

```
int main() {
```

```

    Node* head = NULL;
    insertAtHead = (head, 3);
    insertAtHead = (head, 2);
    insertAtHead = (head, 1);
    print(head);
}
  
```

O/P

1 -> 2 -> 3 -> NULL

* If we pass head ptr by value then it will make another copy of head ptr for that function itself.

Why we use dynamic allocation?

A) Once the function reaches out of scope the static allocated memory has been deleted. But since we want to access it outside we use dynamic allocation.

Two ways of accessing data members from pointer of class.

- (*) n -> next = head;
- n -> next = head

For ② type of passing, insertAtHead(&head, 3) will be done.

~~void insertInMiddle(node~~

and once the function ends the copy of head also gets deleted.

int length of LL (node *head) {

int cnt = 0;

while (head != NULL) {

cnt++;

head = head->next;

return cnt;

void insertAtTail (node **head, int data) {

node * new_node = new node(data);

node * tail = *head;

if (*head == NULL) {

*head = new_node;

return;

while (tail->next != NULL) {

tail = tail->next;

tail->next = new_node;

tail = new_node;

return;

void insertInMiddle (node **head, int data, int p) {

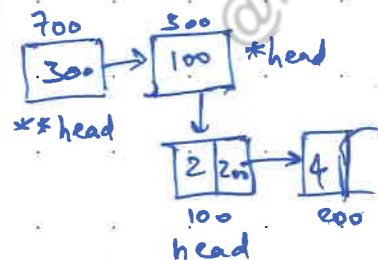
if (*head == NULL || p == 0) {

insertAtHead (head, data);

else if (p > length of LL (*head)) {

Double
pointer
to head node.

points to
point or
pointing to
head



insertAtTail (head, data);

} else {

int jump = 1;

node * temp = *head;

while (jump <= p-1) {

temp = temp->next;

jump++;

node * n = new node(data);

n->next = temp->next;

temp->next = n;

} }

Deletion

void deleteHead (node **head) {

if (*head == NULL) { return; }

node * temp = *head->next;

delete *head;

*head = temp; }

= Delete In Middle

= Delete At Tail

Searching

bool search (node *head, int key) {

node * temp = head;

while (temp != NULL) {

if (temp->data == key) {

return true; }

head = head->next; }

return false; }

// recursively

```
bool searchRecursive (node *head, int key) {  
    if (head == NULL) {  
        return false; }  
    if (head->data == key) {  
        return true; }  
    else { return searchRecu (head->next, key); }  
}
```

- Taking Input (LL)

① void take-input (node **head) { }

② node * take-input () {
 int d;
 cin >> d;
 node * head = NULL;
 while (d != -1) {
 insertAtHead (&head, d);
 cin >> d;
 }
 return head;
}

```
int main () {  
    node * head = take-input ();  
    print (head); }
```

```
{ int d;  
  node * head = NULL;  
  while (cin >> d) {  
      insertAtHead (head, d);  
  }  
  return head; }
```

* For undefined number of nodes we have to take input of.

Operator overloading for LL

For ex. If you want to print LL with only
cout << head; or take input by using
cin >> head;

⇒ void operator << (ostream &os, node *head) {

↓
class name
of "cout"

print (head); → The func we defined
return; } earlier

```
int main () {  
    node * head = take-input ();  
    cout << head; → Now it will print LL  
}
```

• cout << head1 << head2; → If you want to
print two LL in this.
It will give error

↙
The operator overloaded
function for this ~~return~~ is
of void type

∴ cout << head1 << head2 } ⇒ void << head2
↓
void doesn't make
sense.

(*) ∴ If we can be able to return cout object
instead of void then the above mentioned
problem can be resolved.

This effect is known as Cascading of operators.

For ex:- You can concatenate several LL together. by overloading '+' in this manner

⇒ LL₁ + LL₂ + LL₃ + LL₄ + ...;

• Solution through cascading

```
ostream& operator<<(ostream& os, node* head){
    print(head);
    return os; }
```

we changed the return type to cout object's class

cout<<head1<<head2; → will work now ✓

O/P:- 1 → 2 → 3 → NULL
4 → 5 → 6 → NULL

• Taking Input LL through cin >>

```
istream& operator>>(istream& is, node*& head){
    *head = take_input(is);
    return is; }
```

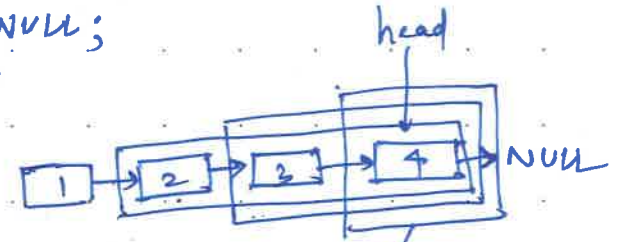
```
int main(){
    node* head1; *head2;
    cin>>head1>>head2; }
```

∴ cin>>head1>>head2;
 returns cin
 returns cin;
 valid statement

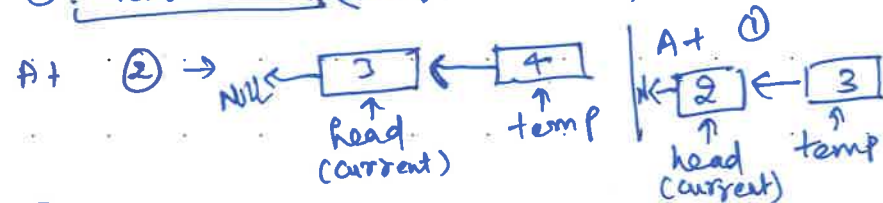
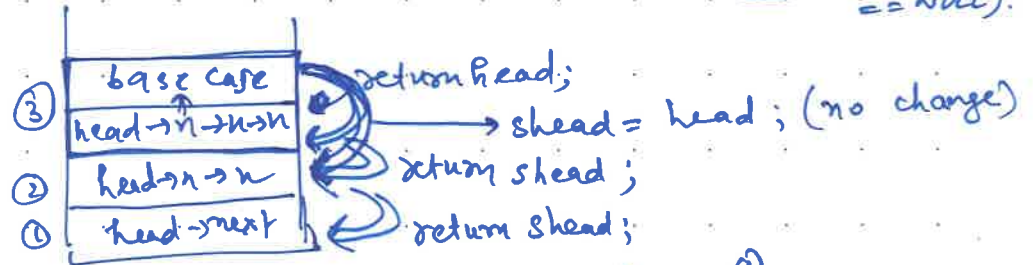
Reverse a LL using Recursion

```
node* recReverse(node* head){
    if (head->next == NULL or head == NULL) {
        return head; }

    node* shead = recReverse(head->next);
    node* temp = head->next;
    temp->next = head;
    head->next = NULL;
    return shead;
}
```



(current head of that subpoint)
shead = head;
 return head
 It reaches one of the base case (head->next == NULL).



Then, N ← 1 ← 2

Midpoint Runner Technique

```
node * midpoint (node * head) {
    if (head == NULL || head->next == NULL) {
        return head;
    }
    node * slow = head;
    node * fast = head->next;
    while (fast != NULL && fast->next != NULL) {
        fast = fast->next->next;
        slow = slow->next;
    }
    return slow;
}
```

Merge Sort of LL (using recursion)

```
node * merge (node * a, node * b) {
    if (a == NULL) { return b; }
    if (b == NULL) { return a; }
    node * c;
    if (a->data < b->data) {
        c = a;
        c->next = merge(a->next, b);
    } else {
        c = b;
        c->next = merge(a, b->next);
    }
    return c;
}
```

Insertion Sort in LL

- Intersection of two LLs

Sorting using merge Sort technique

```
node * mergeSort (node * head) {
    if (head == NULL || head->next == NULL) {
        return head;
    }
```

$O(N)$ {

```
node * mid = midpoint (head);
node * a = head;
node * b = mid->next;
mid->next = NULL;

a = mergeSort (a);
b = mergeSort (b);
node * c = merge (a, b);
return c;
}
```

$$T(n) = \underset{\substack{\downarrow \\ \text{find mid}}}{kn} + 2 \underset{\substack{\downarrow \\ \text{2 calls to half LL}}}{T(n/2)} + \underset{\substack{\downarrow \\ \text{merging}}}{kn}$$

$$= kn + 2T(n/2) \Rightarrow O(n \log n)$$

$\Downarrow$
T.C.

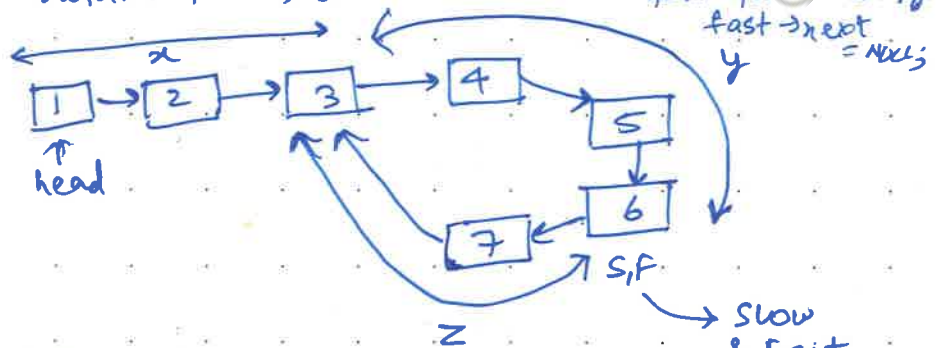
Circular LL

```
void insert (node * head, int d) {
    node * n = new node(d);
    node * temp = head;
    n->next = head;
    if (temp != NULL) {
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = n;
    } else {
        head = n;
    }
}
```

- LL Floyd's Cycle

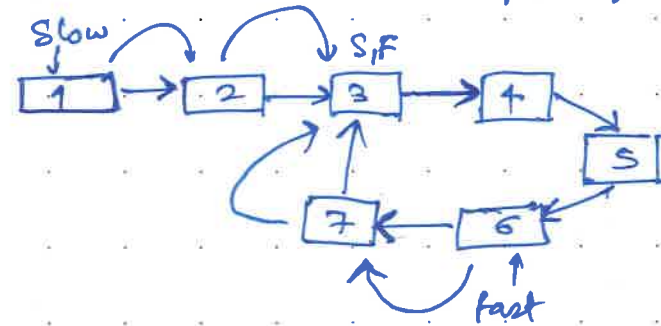
If we maintain slow & fast pointers then if there exists loop then slow & fast will meet at certain node else fast reaches NULL.

```
bool detectCycle(node* head) {
    node* slow = head;
    node* fast = head;
    while (fast != NULL && fast->next != NULL) {
        fast = fast->next->next;
        slow = slow->next;
        if (fast == slow) {
            return true;
        }
    }
    return false;
}
```



Case 2: [3] is the node where loop starts. We assume x, y and z .
 $D_{slow} = x + y$; $D_{fast} = x + 2y + z$ - (*)

And we bring slow ptr to the head and let the fast ptr stay at [6]. And we now move fast and slow both together but ~~only~~ both will move at 1 node / per jump speed.



When S and F meet, they ~~are~~ would be at that node from where loop starts.

$$\therefore x = x$$

Proof:-

$$\text{From (*) } D_f = 2D_s$$

$$x + 2y + z = 2x + y$$

$$x + z = 2x \Rightarrow x = z$$

distance travelled by fast ptr is always double of slow ptr.

• Deletion in Circular LL

```
node* del = getNode(head, data);
```

```
if (del == NULL) return;
```

```
if (head == del) { head = head->next; } // case when del is head
```

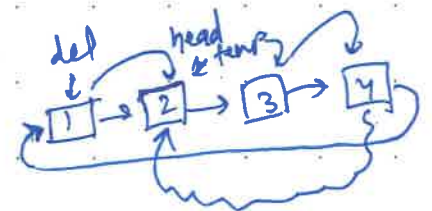
```
node* temp = head;
```

```
while (temp->next != del) {
```

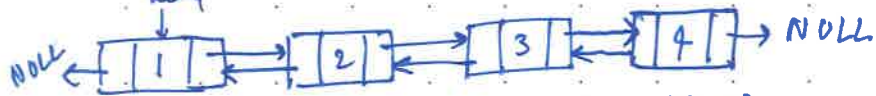
```
    temp = temp->next;
}
```

```
temp->next = del->next;
```

```
delete del;
```



Doubly LL
head



(Notes in another n.b) → (Detailed)

4 bytes + 4 bytes storage for next and prev.

STL (List)

- Forward-list → SLL
- list → DLL

Code

```
→ list<int> l;
```

```
→ list<int> l1 {1, 2, 3, 10, 8, 5};
```

```
→ list<string> l2 {"apple", "guava", "mango"};
```

```
→ l2.push_back("banana");
```

```
→ You can also iterate over list
```

```
    for (auto & s : l2) & cout << s << "-->";
```

```
→ l2.sort();
```

```
→ l2.reverse();
```

```
→ l2.pop_front(); // removes first element from list
```

(l2.front()) → to get front of list

```
→ l2.pop_back() → // removes last element
```

(l2.back()) → to get back of list

```
→ You can use iterators begin() & end()
```

also.

```
→ l2.remove(f) → removes all occurrences of f
```

a string

→ auto it = l2.begin();

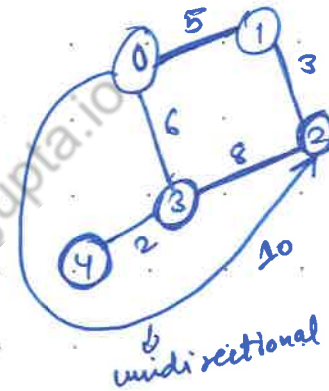
it++;
it++; } → we can't directly do l2.begin() + 2;

l2.erase(it); (Direct access is not granted in list)

or
l2.insert(~~l2.begin()~~ it, "Fruit juice");

Adjacency list for weighted graph.

list < pair<int, int>>



undirectional

0 → (3, 6) (1, 5) (2, 10)
vector 0 is connected with

1 → (2, 3) (0, 5)

2 → (3, 8) (1, 3)

3 → ...

4 → ...

We will use array of lists to implement this

list<> l[100]; ← list Array.

→ 100 lists

Implementation (A graph)

```
int main() {
```

```
    list<pair<int, int>> *l; → dynamic list
```

```
    int n; → no. of vertices
```

```
    cin >> n;
```

```
    l = new list<pair<int, int>>[n];
```

int e; → No. of edges
cin >> e;

for(int i=0; i<e; i++) {

int x, y, wt;

cin >> x >> y >> wt;

l[x].push_back

(make_pair(y, wt));

l[y].push_back

(make_pair(x, wt));

}

// Print the linked lists

for(int i=0; i<n; i++) {

// Print each linked list

for(auto xp: l[i]) {

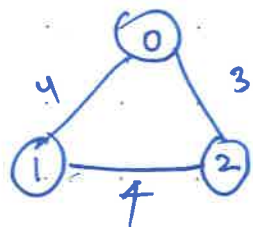
cout << "(" << xp.first << ", " << xp.second << ") ";

}

cout << endl; }

}

I/P:-
3 3
0 1 4
0 2 3
1 2 4

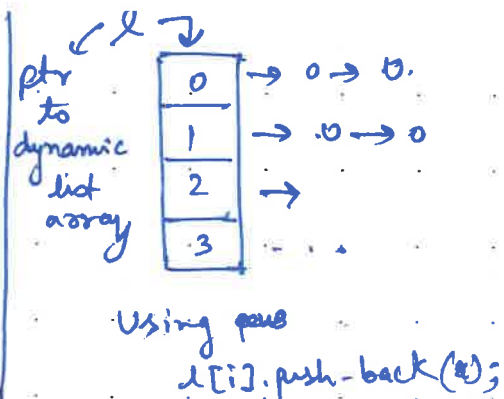


O/P:- ~~undirected~~

0 → (1,4), (2,3)

1 → (0,4), (2,4)

2 → (0,3), (1,4)



Using pair

l[i].push_back(x);

1 → (2,5)

[1 is connected with 2 with wt. = 5]

for undirected we ~~add~~

for both vertices

Stack

→ Intro ✓

→ Implementation using LL / array. ✓

→ Implementation using Vector

class Stack {

private:

vector<int> v;

public:

void push(int data) {

v.push_back(data);

}

bool empty() {

return v.size() == 0; }

void pop() {

if(!empty()) { v.pop_back(); }

int top() {

return v[v.size()-1]; }

int main() {

Stack<int> s;

Stack<char> st;

for(int i=65; i<70; i++) {

st.push(i); }

while(!s.empty()) {

cout << s.top() << endl;

s.pop(); }

→ to make it generalise for all data types now replace all int with T

Now we can use it.

Linked List - Palindrome

```
bool isPalindromeUtil(node*& left, node*& right) {  
    if (right == NULL) {  
        return true; }  
    if (right == NULL) {  
        return true; }  
}
```

```
bool isp = isPalindromeUtil(left, right->next);
```

```
if (isp == false) { return false; } → if get  
false from  
also further  
calls we  
return false
```

```
bool ispl = (right->data ==  
             (*left)->data);
```

```
*left = (*left)->next;
```

```
return ispl; }
```

```
bool isPal (node* head) {  
    isPalindromeUtil(&head, head); }
```

Stack approach

```
bool isPal (Node* head) {
```

```
    Node* slow = head;
```

```
    stack<int> s;
```

```
    while (slow != NULL) {
```

```
        s.push(slow->data);
```

```
        slow = slow->ptr;
```

```
    while (head != NULL) {
```

```
        int i = s.top();
```

```
        s.pop();
```

```
        if (head->data != i) { return false; }
```

```
        head = head->ptr; }
```

```
    return true; }
```

Reversing a Template

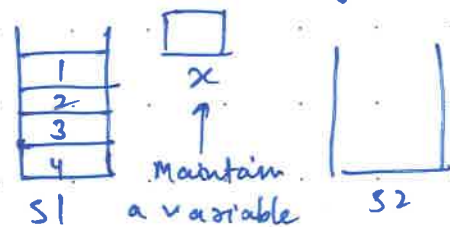
Stack STL

```
stack<data type> name;
```

- empty() - Returns whether stack is empty - $O(1)$
- size() - size of stack - $O(1)$
- top() - returns top most element - $O(1)$
- push(x) - push element x - $O(1)$
- pop()

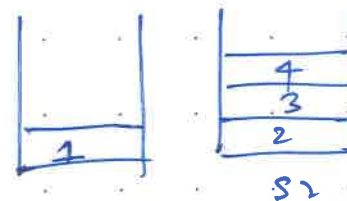
Reverse a stack using another stack

Algo -



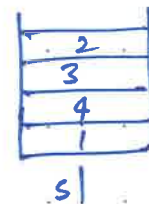
i) $x = s1.top()$;
now we pop()
it.

Then we $s1.pop()$
• 2, 3, 4
and push it in
S2.



s1.push(x);

Then push all elements
back from S2 to S1



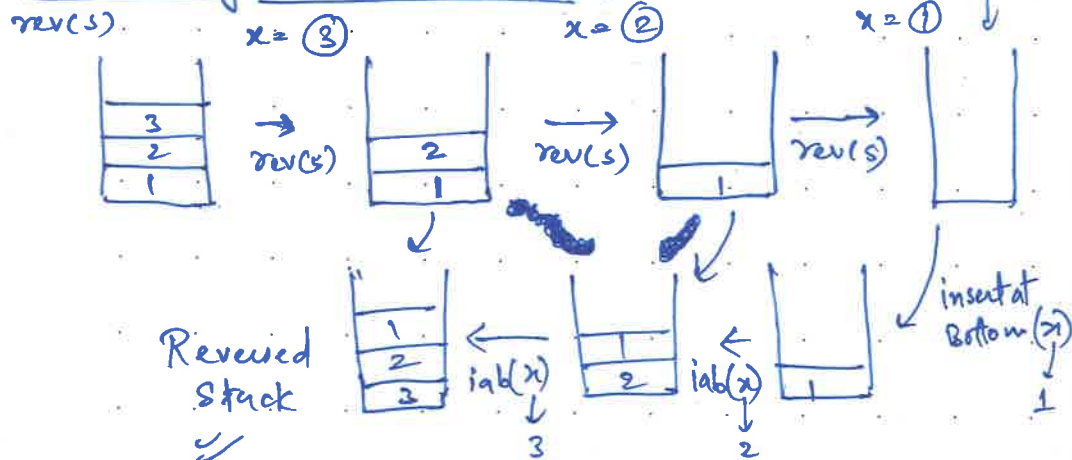
Time - $O(n^2)$
space - $O(n)$

Code

```
void reverseStack(stack<int> &s1) {
    stack<int> s2;
    int n = s1.size();
    for (int i = 0; i < n; i++) {
        int x = s1.top(); // store x
        s1.pop(); // pop it out
        transfer(s1, s2, n-i-1); // Transfer n-i-1 elements from s1 to s2
        s1.push(x); // first push the x.
        transfer(s2, s1, n-i-1); // Transfer n-i-1 elements from s2 to s1.
    }
}

void transfer(stack<int> &s1, stack<int> &s2, int n) {
    for (int i = 0; i < n; i++) {
        s2.push(s1.top());
        s1.pop();
    }
}
```

Recursively reverse a stack



Code

```
void insertAtBottom(stack<int> &s, int x) {
    if (s.empty()) {
        s.push(x);
        return;
    }
    int data = s.top();
    s.pop();
    insertAtBottom(s, x);
    s.push(data);
}

void reverseStack(stack<int> &s) {
    if (s.empty()) {
        return;
    }
    int x = s.top();
    s.pop();
    reverseStack(s);
    insertAtBottom(s, x);
}
```

Callstack
↓
implicit
(inbuilt internal)

Stack
↓
explicit

Recursively
put
things
at
last

"You can't enjoy wealth if you're not in good health."

Balanced Paranthesis

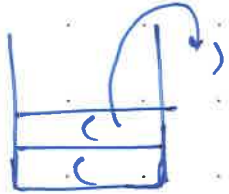
```
bool isValidExp(char *s) {
    stack<char> st;
    for (int i = 0; s[i] != '\0'; i++) {
        char ch = s[i];
        if (ch == '(') {
            st.push(ch);
        }
        else if (ch == ')') {
            if (st.empty() || st.top() != '(') {
                return false;
            }
            st.pop();
        }
    }
    return st.empty();
}
```

```

    }
    return st.empty();
}

int main() {
    char s[100] = "((a+b)+(c-d+f))";
    if (isValidExp(s)) { cout << "Yes"; }
    else { cout << "No"; }
}

```

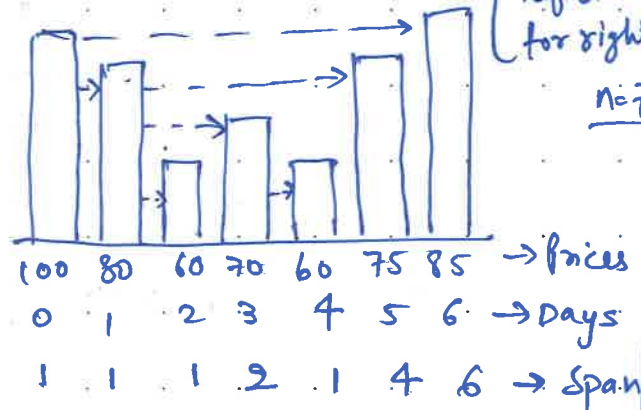


When we see $(())$ it follows LIFO order

This open bracket is last and first one to get its closing bracket.

if $)$ comes we check if $($ is there and pop $($ and so on.

Stock Span Problem

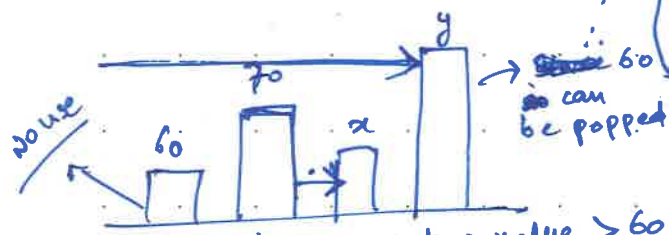


LUNCHTIME PROBLEM
In this we are checking only towards left side, we can also check for right side and add the values to get who can see each other.

$n=7$

6, 85	6-0=6
5, 75	5-1=4
4, 60	4-3=1
3, 70	3-1=2
2, 60	2-1=1
1, 80	1-0=1
0, 100	1

Stack



Since we got a value > 60 ... value more than 60

⇒ At index 5, 75 comes up which is greater than 60, 70, [60 (already popped in the case 70)]. So we will pop 60 & 70. And the next previous greater element in the stack is 80. So we will calculate span = 5 - index of 80 = 5 - 1 = 4

⇒ At index 6, 85 comes up and is greater than 80 so we'll pop 80 also, and left out with 100 which is greater than 85 and will work as stopper for it.

[If at index 6, 105 would have come then span = 6 + 1 = 7.]

Code

```

void StockSpan (int prices[], int n, int span[]) {

```

```

    stack<int> s;

```

```

    push(0);

```

```

    span[0] = 1;

```

```

    for (int i = 1; i < n; i++) {

```

```

        int currentPrice = price[i];

```

```

        while (!s.empty() and price[s.top()] <= currentPrice) {

```

```

            if (!s.empty()) s.pop();

```

```

            &span[i] = i - prev-high = s.top();

```

```

            span[i] = i - prev-high; } else { span[i] = i+1; }

```

```

        s.push(i);

```

```

    }

```

This will be checked first and then the condition if the condition are reversed then it will give seg fault

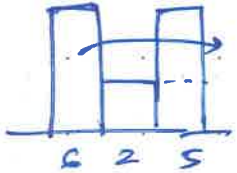
Marks	Frequency
0	6
1	2
2	5
3	4
4	5
5	1
6	6

begin

$n = 7$

$$\text{max area} = 4 \times 3 = 12 \text{ units.}$$

Ex.



2.

Algo

```
s.push(index)
```

```

if ( a[index] >= s.top() ) {
    s.push(index); }

```

↓
No obstruction at
this index we normally
start

else
area = a[s.top] * (index - s.top() - 1)

obstruction
index
(right closest
smaller
value)

Obstruction
On left
closest
side
(present
in stack)

- Exact Q - Largest Area under Histogram using stack (on utube)

- Design a stack class, that supports following opⁿ.

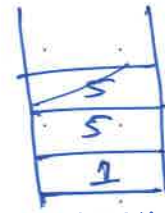
- push(), pop().

- Top()

- Push(), pop().
- Top()
- Get min() - [After each push/pop it must be able to give us min/max of the stack in min time]
- Get max()



Stack

 $\mu^2 \gamma$ 

max

Stack min
Now if I pop 3 out of the stack. then
min.top and max.top will still give us
correct answers. $\checkmark$

class MinMaxStack {

```
vector<int> stack;
vector<int> min;
vector<int> max;
```

public:

```
void push (int data) {
    int cur_min = min = data;
    int cur_max = max = data;
    if (min.size() > 0) {
        cur_min = min (min[min.size()-1],
                        data);
        cur_max = max (max[max.size()-1],
                        data);
    }
    min.push_back (cur_min);
    max.push_back (cur_max);
    stack.push_back (data);
}
```

→ [DIY other func]

Queue

- Linear ← ATM, Bank, Tickets...
- FIFO

Applications

① WhatsApp Sender

① Hi
② How are you?

① ② ...

Receiver

① Hi
② How are you?

② operating system, → Scheduling

③ Tree/Graph / FB / Quora / ...

↳ Level order Traversal

↳ BFS

Fun Fact: We can create whole binary no. and use it directly using `bitset<>`

```
bitset<5> b;    O/P:- 10010
b[4] = b[1] = 1;
cout << b;
```

Implementation → [array, vector, LL, stack]

→ Array Implementation ✓ (In other N.B.)

class Queue {

```
int *arr;
int f, r, cs, ms;
```

public:

```
Queue (int ds = 5) {
```

```
    arr = new int ds[5];
```

```
    cs = 0;
```

```
    ms = ds;
```

```
    f = 0;
```

```
    r = ms - 1;
```

```
bool full () { return cs == ms; }
```

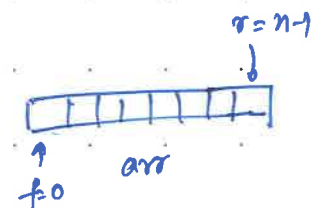
```
void push (int data) {
    if (!full()) {
        r = (r+1) % ms;
        arr[r] = data;
        cs++;
    }
}
```

```
void pop () {
```

```
    if (!empty()) {
        f = (f+1) % ms;
        cs--;
    }
}
```

Destructor

```
~Queue () {
    if (arr != NULL) {
        delete [] arr;
        arr = NULL;
    }
}
```



```
bool empty () {
    return cs == 0;
}
```

```
int front () {
    if (!empty()) {
        return arr[f];
    }
}
```

Queue using LL (It doesn't have size limit) $\Rightarrow$

```

class Queue() {
    int cs;
    list<int> l;
public:
    Queue() { cs = 0; }
    bool isEmpty() { return cs == 0; }
    void push(int data) { l.push_back(data);
        cs = cs + 1; }
    void pop() { if (!isEmpty()) { cs--;
        l.pop_front(); } }
    int front() {
        return l.front(); } }
    
```

Queue STL

```

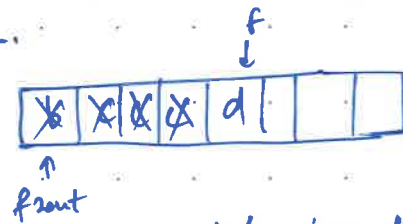
queue<data-type> q;
• empty()
• size()
• push(q) and pop()
• front()
• back()
    
```

First Non Repeating Character in Running Stream of Characters

a a b c c b c d
 I/p
 O/p a -1 b b b -1 -1 d
 (queue)

Queue can be used wherein FIFO is being followed and only first element to satisfy the condⁿ will be able to execute first.

Algo



freq. map
 a $\rightarrow$ 2
 b $\rightarrow$ 2
 c $\rightarrow$ 2 $\rightarrow$ 3
 d $\rightarrow$ 1

(i) when we push 'a' since freq. of a $\rightarrow$ 1 in queue we print the queue.front() = 'a'.

(ii) when we add another 'a', now freq. of 'a' $\rightarrow$ 2. so we pop front() and then second 'a' becomes front() and then we check its freq. and pop it also. and print (-1). when queue becomes empty.

(iii) Then we add 'b' and update its freq. and check front()'s freq. and print it.

(iv) Then we add 'c' and check front()'s freq. 'b' $\rightarrow$ 1. so we print front().

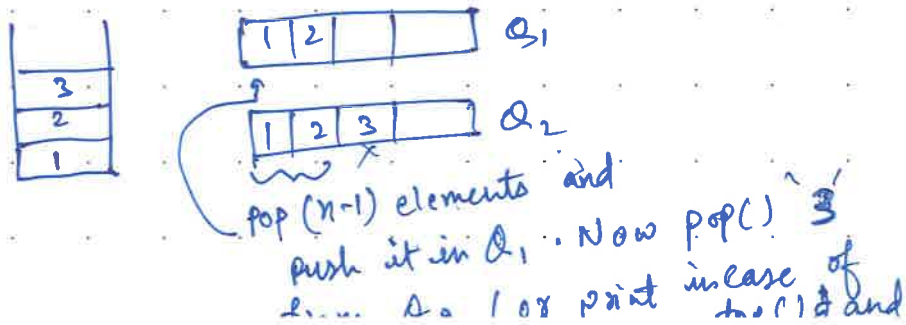
(v) we add another 'c' and check front()'s freq. and print front(). and so on...

"Your body hears everything your mind says."

Implementation

```
int main() {
    queue <char> q;
    char ch;
    cin >> ch;
    while (ch != '\0') {
        q.push(ch);
        freq[ch - 'a']++;
        while (!q.empty()) {
            int idx = q.front() - 'a';
            if (freq[idx] > 1) {
                q.pop();
            }
            else {
                cout << q.front() << endl;
                break;
            }
        }
        if (q.empty()) {
            cout << "-1" << endl;
        }
        cin >> ch;
    }
}
```

Stack using two queues

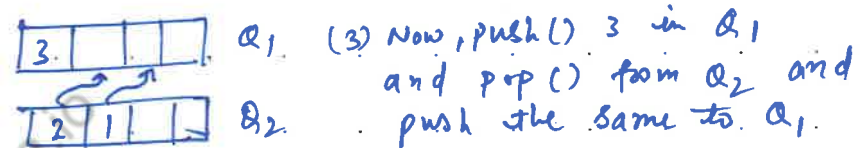
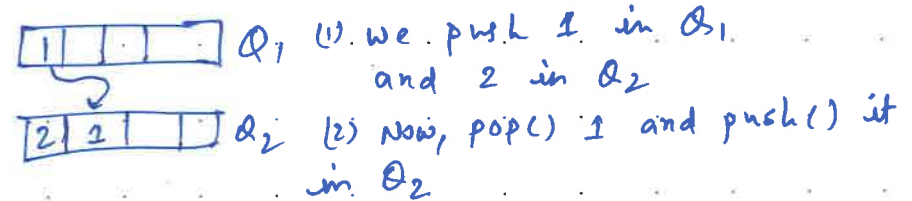


push it to Q₁) and so on.

or

we can do
⇒ Alt way (push() is expensive)

push() → O(1)
pop() → O(N)
push → O(N)
empty → O(1)



and so on...

Here { push → O(N) with max opⁿ
pop → O(1)

Implementation

```
template < typename T >
class Stack {
    queue < T > q1, q2;
public:
    void push (T x) {
        q1.push(x);
    }
    void pop() {
        if (q1.empty()) {
            return;
        }
        while (q1.size() > 1) {
            T element = q1.front();
            q2.push(element);
            q1.pop();
        }
        q1.pop(); → last element
    }
}
```

Swap the ~~names~~ of $q1$ & $q2$ so that only $q1$ contains the updated elements after $pop()$.

~~Queue & its implementation~~

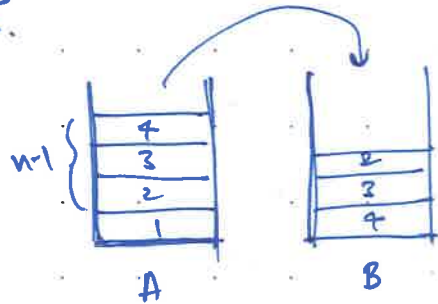
~~Queue~~ swap($q1, q2$);

~~Queue~~ int $top() \{$ while($q1.size() > 1$) {
 $T_element = q1.front();$
 $q2.push(T_element);$
 $q1.pop();$ }

$T_element = q1.front();$
 $q1.pop();$
 $q2.push(T_element);$
 $swap(q1, q2);$
 $return T_element; \}$

Queue using 2 stacks

$pop() \rightarrow O(n)$
 Don't push 1 in stack B. Push $(n-1)$ elements.

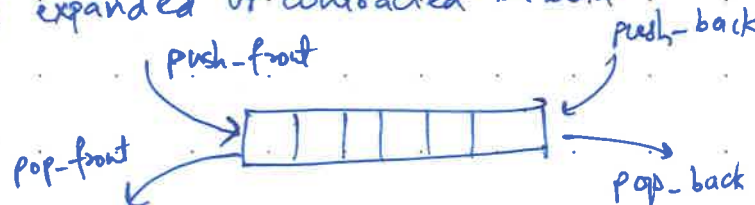


$push() \rightarrow O(1)$

$front() \rightarrow O(n)$

Deque - Double-ended queue

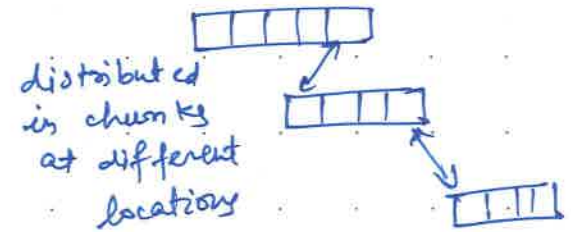
$\rightarrow$ ~~array~~ sequence containers with dynamic sizes that can be expanded or contracted on both ends.



$\rightarrow$ It allows random access through index

$\rightarrow$ It comprises the ability of vector, LL.

$\rightarrow$ Unlike vectors, deque are not guaranteed to store all its elements in contiguous storage locations



$\rightarrow$ Works fine with long sequences - (no reallocation required)

Disadv

Frequent insertion or removals of elements from posⁿ other than beg/end performs worse.

Arraysub #sliding-window #interview #spo
 Given an array and an integer K , find the max for each and every contiguous subarray of size K .

IP:- 9

1 2 3 1 4 5 2 3 6

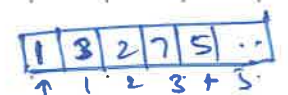
3

OP:-

3 3 4 5 5 5 6

Code

```
int a[];
deque<int> Q(K);
int i;
for(i=0; i<K; i++){
    while(!Q.empty() && a[i] > a[Q.back()]){
        Q.pop_back();
    }
    Q.push_back(i);
}
```



when ... come on 0 we push-back 0th index

// We go to next index but $3 > 0$ so
we pop 0th index and push back 1th index
back

When we go to next index $2 < 3$ so
we just push back 2nd index.

In sliding window if 1, 2, 3 were to be there
then we only keep 2nd index in Q, because
for further sliding window it will be max.
but in case of 1, 3, 2, ..., 3 can be out of
the sliding window for ~~case~~ above case.

// Process the remaining elements

for (; i < n; i++) {

cout << a[Q.front()] << " ";

// 1. Remove the elements which are not the part
of the window (Contraction)

while (!Q.empty() & & (Q.front() <= i - k)) {
Q.pop-front();}

// 2. Remove the elements which are not useful
and are in the window

while (!Q.empty() & & a[i] >= a[Q.back()]) {
Q.pop-back();}

// 3. Add the new elements (Expansion)

Q.push-back(i); }

cout << a[Q.front()].recrd; → last element

}

Max. length Unique Character Substring | Sliding Window

Algo

I/P: B C D E C G H length = 5 → O/P

(1) We maintain current_len and max_len.

(2) We will use hash-map/
array to store index
of previously occurred
element.

(2) We will set each value
to -1 by default
and will update index
of first ~~occurrence~~ occurrence
element for each index number in array

length of
longest substring
non-repeating
characters

Expansion + Contraction { if visited[a[i]] == -1 then we increase
current_len and update max_len
else
i.e. current_len = i - last-occ
↓
visited[a[i]]

• contract the size, from back,
of sliding window

and update max_len (if curr_len > max_len)

update last-occ =
(visited[

Sliding window Qs are best solved using
deque.

Binary Tree.

Preorder build (Recursive)

class node {

public:

int data;

node* left;

node* right;

node(int d) {

data = d;

left = right = NULL; }

node* buildtree() {

int d;

cin >> d;

if (d == -1) return NULL;

node* root = new node(d);

root->left = buildtree();

root->right = buildtree();

return root;

8, 10, 1, -1, -1, 6, 9, -1, -1, 7, -1, -1, 3, -1, 14, 13, -1, -1, -1

void print(node* root) {

if (root == NULL) return;

cout << root->data << " ";

print(root->left);

print(root->right);

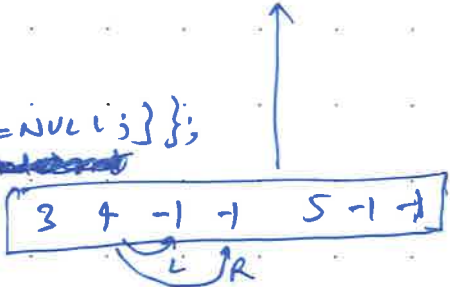
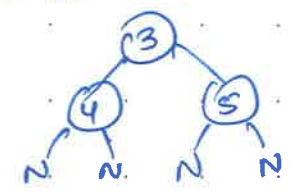
}

int main() {

node* root = buildtree();

return 0;

Use template <typename T> to make it generic



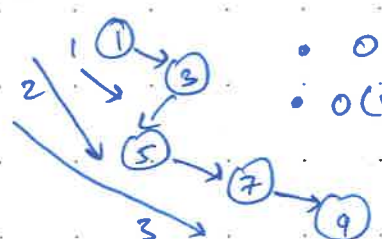
```
int height(node* root) {
    if (root == NULL) return 0;
    int ls = height(root->left);
    int rs = height(root->right);
    return 1 + max(ls, rs);
}
```

- Post order =

- Pre order =

* - Level order (in Q practice N.B.) =

↓
worst case → Skew Tree - $O(n^2)$



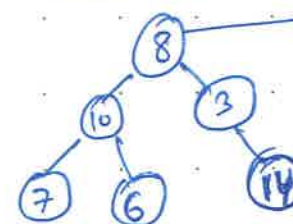
- $O(n)$ → height
- $O(1) + O(2) + \dots$

↓
recursion goes from root to each level

- Can be done in $O(n)$ using BFS traversal

= $O(n^2)$

BFS Traversal (Queue method)



pop() and push() → 10 and 3 & print()

pop() and print() and push() → 7 & 6

pop() and print() and push() → 14

(Code in Coding problems N.B.)

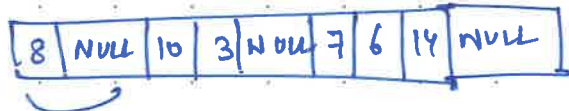
"Eat to live, not live to eat."

~~void bfs (node * root)~~

if we want to print level-wise in this way

8 $\hookrightarrow$
10 3 $\hookrightarrow$
7 6 14 $\hookrightarrow$

We maintain
NULL for each
level end.



We push it in queue beforehand

```
if (q.front() == NULL) {
    cout << endl;
    q.pop();
    if (!q.empty()) {
        q.push(NULL);
    }
}
```

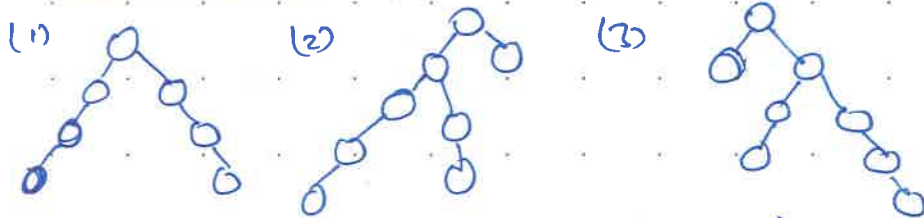
Count no. of nodes & Sum of all nodes

```
→ int count (node * root) {
    if (root == NULL) {
        return 0;
    }
    return 1 + count (root->left) + count (root->right);
}
// (Pre order recursion)
// O(N)
```

→ for sum

```
return root->data + sum (root->left) + sum (root->right);
```

Diameter of BT



```
int diameter (node * root) {
    if (root == NULL) {
        return 0;
    }
    int h1 = height (root->left);
    int h2 = height (root->right);
    // return h1 + h2 + 1;
}
```

```
int op1 = h1 + h2;
int op2 = diameter (root->left);
int op3 = diameter (root->right);
return max (op1, max (op2, op3));
```

Worst case - $O(N^2)$

Diameter of B.T (Optimised)

At each level for each node we find the height of its left child and right child which takes $O(N)$ and this calculation of ht. function takes place for N nodes.

$\therefore O(N^2)$

But if we use bottom-up (storing/returning the height to parent node) so that it takes only $O(1)$. And thus for N nodes it takes

$\therefore O(N)$.

class Pair {

```
public:
    int height;
    int diameter;
};
```

```
Pair fastDiameter (node * root) {
```

```
    Pair p;
    if (root == NULL) {
        p.diameter = p.height = 0;
    }
    return p;
}
```

```
    Pair left = fastDiameter (root->left);
```

```
    Pair right = fastDiameter (root->right);
```

```

p.height = max(left.height, right.height) + 1;
p.diameter = max(left.height + right.height,
                  max(left.diameter, right.diameter));
return p;

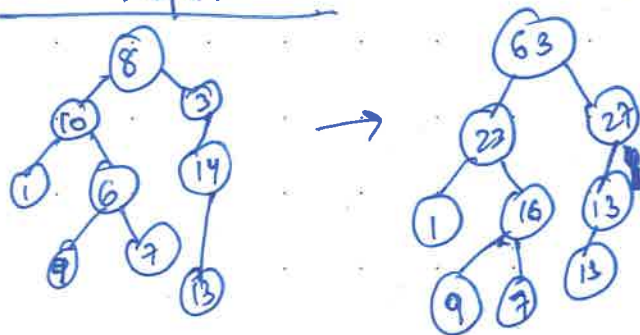
```

things to do are after recursive calls
 * (we assign p object and then return it)

As you can observe that the style of recursive ~~code~~ func are post order.

* We are solving things with pair class so that's why here left is a class object and left.height and left.diameter are its data members

- Sum Replacement



Code

```

int replaceSum(node * root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) {
        return root->data;
    }
    int leftsum = replaceSum(root->left);
    int rightsum = replaceSum(root->right);

```

```

int temp = root->data;
root->data = leftsum + rightsum;
return temp + root->data;
}

```

check Balanced Tree

~~HBPair~~ class

```

class HBPair {

```

```

public:
    int height;
    bool balance;
};

```

```

HBPair isHeightBalance(node * root) {

```

```

    HBPair p;

```

```

    if (root == NULL) {

```

```

        p.height = 0;

```

```

        p.balance = true;

```

```

        return p;
    }

```

```

    HBPair left = isHeightBalance(root->left);

```

```

    HBPair right = isHeightBalance(root->right);

```

```

    if (abs(left.height - right.height) <= 1 &&
        left.balance && right.balance) {
        p.height = max(left.height, right.height) + 1;

```

```

        p.balance = true;
    }

```

```

    else {

```

```

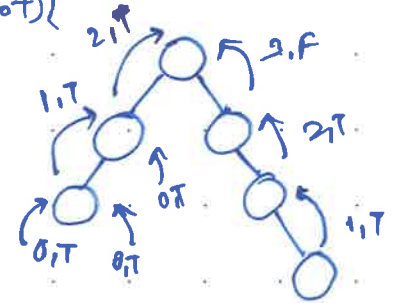
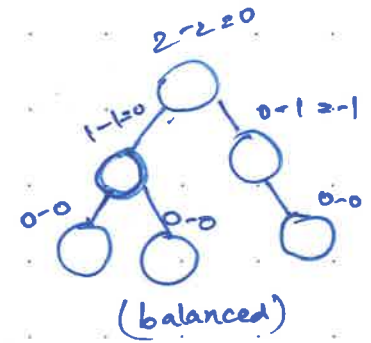
        p.balance = false;

```

```

        return p;
    }

```



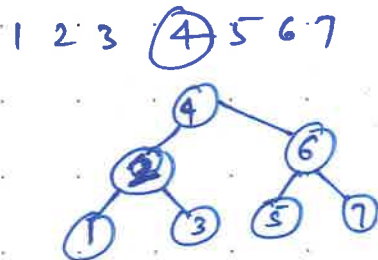
$$|h_1 - h_2| \leq 1$$

Apply bottom-up

$$O(N)$$

Build Balanced Tree from Array

```
node * buildTreeFromArray (int *a, int s, int e) {
    if (s > e) {
        return NULL;
    }
    int mid = (s + e) / 2;
    node * root = new node(a[mid]);
    root->left = buildTreeFromArray(a, s, mid - 1);
    root->right = buildTreeFromArray(a, mid + 1, e);
    return root;
}
```



Tree from preorder and inorder

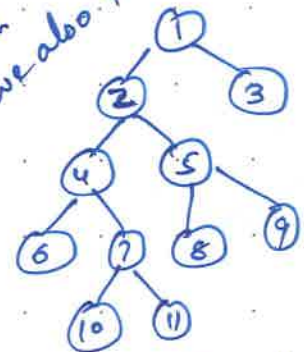
```
node * create (int *in, int *pre, int s, int e) {
    static int i = 0;
    if (s > e) { return NULL; }
    node * root = new node(pre[i]);
    int index = -1;
    for (int j = s; j <= e; j++) {
        if (in[j] == pre[i]) {
            index = j;
            break;
        }
    }
    i++;
    root->left = create(in, pre, s, index - 1);
    root->right = create(in, pre, index + 1, e);
}
```

return root; }

Right View

We can proceed in root first - post order manner (but we also take root first) so that we jump onto right elements at each element first.

We will also maintain max-level pointer which points to max level we got to go so far.



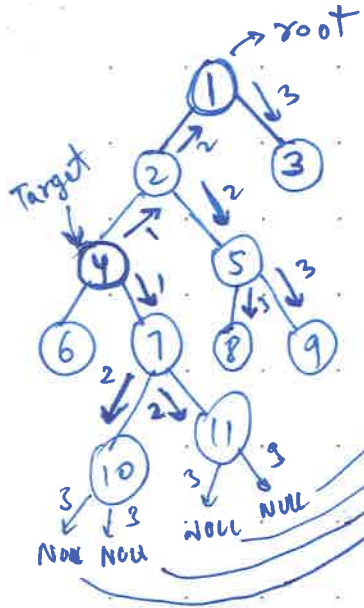
At each level we need to print last element so as to get right view.

Code

```
void right (node * root, int level, int * maxlevel) {
    if (root == NULL) { return; }
    if (*maxlevel < level) {
        cout << root->data << " ";
        *maxlevel = level;
    }
    right (root->right, level + 1, maxlevel);
    right (root->left, level + 1, maxlevel);
}
```

root
↓
right
↓
left

Nodes at K distance from given node



For ex:-

$K = 3$ and Target = 4

(1) From target node, we use printAtLevelK() function.

(2) 8, 9 and 3 also present at distance 3 from target node.

(4) Then we check $(K - 2 - DL)$.

Case:- If $(K - 2 - DL) > 0$

then we print at $K - 2 - DL$ in the right subtree.

Case:- If $(K - 2 - DL) < 0$ then we print the root subtree.

(5) In case of ② → $K - 2 - DL = K - 2 - 0 = 1$

So ② will call right subtree and ask it to print nodes at 1 distance. (8, 9)

(6) Then this will return $\frac{D+1}{2}$ to root which is ①

So $K - 2 - DL = K - 2 - 1 = 0$.

So distance of target node from left subtree

So now ① will call its right subtree to print node at 0 distance i.e. ③

Case 2:-
(7) If $D + 1 = K$ then we directly print ancestor node.
(returned value to previous ancestor)

* If target node is in right subtree then do opposite opⁿ.

Implementation

void printAtDistanceK(node* root, node* target, int K) {

if (root == NULL) {
return -1; }

if (root == target) {
printAtLevelK(target, K);
return 0; }

function we use in level order

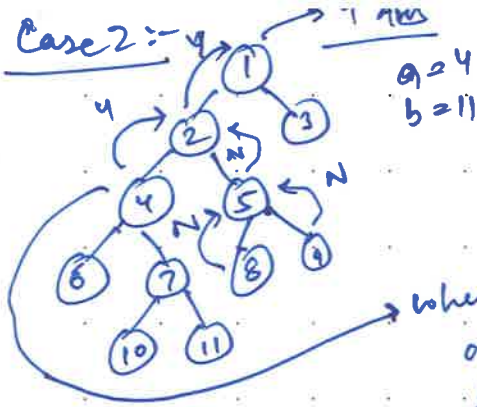
for left part

// next step - Ancestors

int DL = printAtDistanceK(root->left, target, K);

if (DL != -1) {

// Ancestor itself / You need to go to right subtree



when we reach here we stop and return 4 and since from other subtrees we received NULL we keep on returning 4 and at last print it.

Max path sum from any node to any node

~~op1~~ Cases where we can get max path sum from

op1 = root $\rightarrow$ data

op2 = root $\rightarrow$ data + max-from-left

op3 = root $\rightarrow$ data + max-from-right

op4 = MFL + MFR + root $\rightarrow$ data.

Code

```
int maxPathSum(node* root, int* max-sum) {
```

```
    if (root == NULL) { return 0; }
```

```
    int max-from-left = maxPathSum(root->left, max-sum);
```

```
    int max-from-right = maxPathSum(root->right, max-sum);
```

```
    int op1 = root->data;
```

```
    int op2 = root->data + max-from-left;
```

```
int op3 = max-from-right + root->data;
```

```
int op4 = max-from-left + max-from-right + root->data;
```

```
int current_max = max(max(max-from-left, max-from-right), root->data);
```

* max-sum

= max(*max-sum,

contains max path sum till now

max(current, max-from-left

+ root $\rightarrow$ data

+ max-from-right));

return current;

}

int main() {

node* root = buildtree();

int max-sum = INT_MIN;

int help = maxPathSum(root, &max-sum);

cout << max-sum;

```
node* buildGeneric() {
```

```
    int d, child;
```

```
    cin >>
```

```
    node* root = new node(d);
```

```
    if (child == 0) { return root; }
```

```
    if (child == 1) { root->left = buildGen(); }
```

```
    else if (child == 2) {
```

```
        root->left = build();
```

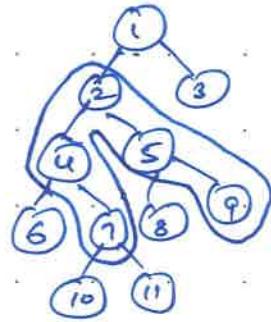
```
        root->right = build();
```

```
        return root; }
```

Shortest Distance b/w nodes of a B.T.

Algo.

- 1) Find LCA of a & b.
- 2) find distance of a from LCA and b from LCA.
- 3) Take sum of it.



distance b/w
7 and 9

// find the level of a give node
from given root.

```
int search (Node* root, int key, int level) {  
    if (root == NULL) return -1;  
    if (root->data == key) {  
        return level;  
    }
```

```
    int left = search (root->left, key, level+1);  
    if (left != -1) {  
        return left;  
    }
```

```
    return search (root->right, key, level+1);  
}
```

// find dis b/w two nodes

```
int findDis (Node* root, int a, int b) {  
    Node* lca-node = lca (root, a, b);
```

```
    int l1 = search (lca-node, a);
```

```
    int l2 = search (lca-node, b);
```

```
    return l1 + l2; }
```

BST

Insertion & Build

node* build () {

int d;

cin >> d;

Node* root = NULL;

while (d != -1) {

root = insertInBST (root, d);

cin >> d;

}
 return root;

}

node*
insert (Node* root, int
data) {

if (root == NULL) {
 return new
 Node (data);
 }

if (data <= root->data) {

root->left =
 insert (root->left,
 data);

}

else {
 root->right =
 insert (root->right,
 data);

}
 return root;

② Inorder of BST

— sorted

Searching →

best case → root $O(1)$

Avg case → $O(\log n)$

worst → $O(n)$ → skewed tree.

```
bool search (Node* root, int data) {
```

```
    if (root == NULL) return false;
```

```
    if (root->data == data) return true;
```

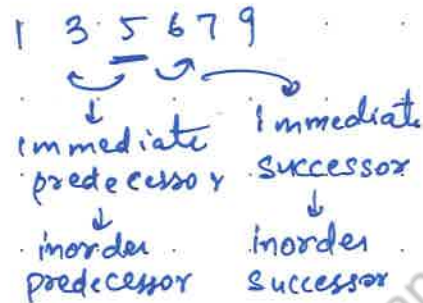
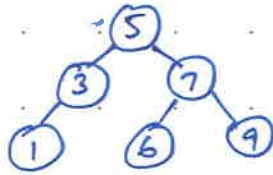
```
    if (data < root->data) {  
        return search (root->left, data);  
    }
```

else return search(root->right, data); }

Deletion

Cases:- A node with (i) 0 child (ii) 1 child (iii) 2 children.

- In case of 2 children, if we want to remove 5, then either we can replace 3 or 6 in place of it.



Code

```
node* deleteInBST(node* root, int data) {
```

```
    if (root == NULL) {
        return NULL;
    }
```

```
    else if (data < root->data) {
        root->left = deleteInBST(root->left, data);
        return root;
    }
```

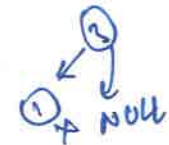
```
    else if (data == root->data) {
```

// 1. Node with 0 child - leaf node

```
    if (root->left == NULL && root->right == NULL) {
        delete root;
        return NULL;
    }
```

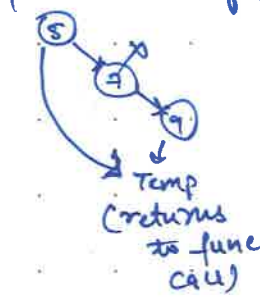
// 2. case - only 1 child

```
    if (root->left != NULL && root->right == NULL) {
        node* temp = root->left;
        delete root;
        return temp;
    }
```



if (root->right != NULL && root->left == NULL) {

```
    node* temp = root->right;
    delete temp;
    return temp;
}
```



// 3. Case - 2 children

```
node* replace = root->right;
```

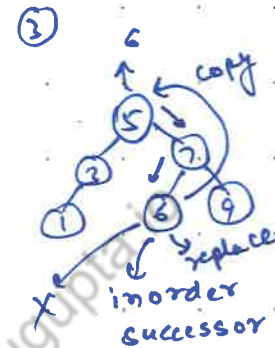
// find the inorder successor from right subtree

```
while (replace->left != NULL) {
    replace = replace->left;
}
```

```
root->data = replace->data;
```

```
root->right = deleteInBST(root->right, replace->data);
```

```
return root; }
```



(When we find

inorder successor it is definite that it no has left child node because its already the lowest in right subtree. So just we have to do 0/1 child deletion to remove it from here)

```
    else {
        root->right = deleteInBST(root->right, data);
        return root;
    }
```

(Things are done in inorder fashion)

```
int main() {
```

```
    root = deleteInBST(root, 5);
```

```
    inorder(root);
```

Check BST

```
bool isBST(node* root, int minV = (INT_MIN), int maxV = (INT_MAX)) {
```

```
    if (root == NULL) {
        return true;
    }
```

```
    if (root->data >= minV && root->data <= maxV && isBST(root->left, minV, root->data) && isBST(root->right, root->data, maxV)) {
```

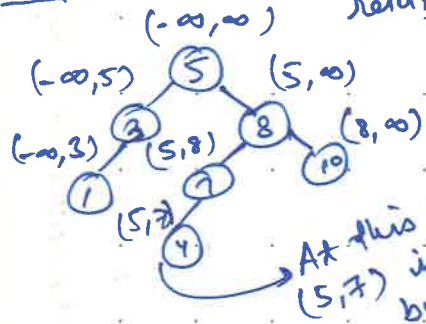
when we go left we set maxV

when we go right we set minV

```
        return true;
    }
```

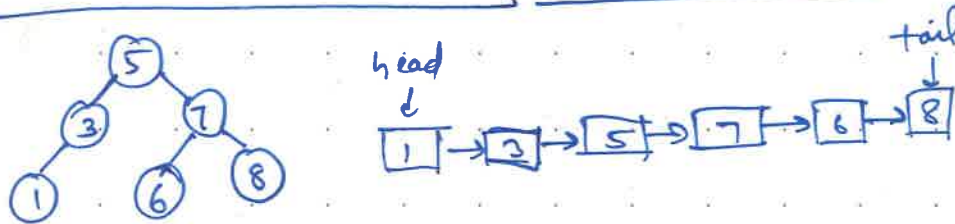
```
    return false;
}
```

Exp



At this node (5, 7) is range but node is 4 so we delete it.

BST to sorted linked list / Flatten a tree



Case:- We only have root



INTERNATIONAL TRAVEL INSURANCE



- Hospitalization Cover - IPD & OPD
- Pre-Existing Ailments covered in case of Life Threatening Conditions
- Double Sum Insured - Accidental Hospitalization
- Coverage for Loss of Passport & Checked-in Baggage
- Trip Delay & Cancellation
- Upgradation to Business Class

Conditions Apply

explore

INTERNATIONAL TRAVEL & HEALTH INSURANCE FOR STUDENTS



In-Patient Hospitalization



Coverage at
Home Country



Dental Expenses



Loss of Passport
& Laptop



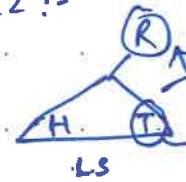
Loss/Delay of Checked-in
Baggage



Sponsor Protection

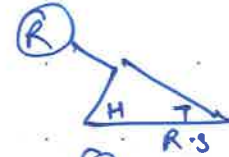
Conditions Apply

Case 2 :-



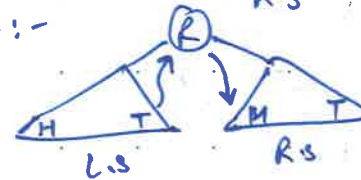
This left tree will
return head & tail $\Rightarrow H \dots T \rightarrow R$
we link the tail
to Root

Case 3 :-



The right subtree
will return h & t $\Rightarrow R \rightarrow H \dots T$
we link head to
root.

Case 4 :-



We link Tail of
L.S to Root
and Head of R.S
to Root.

$\Rightarrow$ Inorder is the
way of solving

Code

```
class linkedlist {
```

```
public:
    node* head;
    node* tail; };
```

```
linkedlist flatten(node* root) {
```

```
    linkedlist l;
```

```
    if (root == NULL) {
```

```
        l.head = l.tail = NULL;
```

```
        return l; }
```

leaf node

```
    if (root->left == NULL & root->right == NULL) {
```

```
        l.head = l.tail = root;
```

```
        return l; }
```

left is not null

```
    if (root->left != NULL & root->right == NULL) {
```

LinkedList leftLL = flatten(root → left);

leftLL.tail → right = root;

l.head = leftLL.head;

l.tail = root;

return l;

// Right is not NULL

if (root → left == NULL & root → right != NULL) {

LinkedList rightLL = flatten(root → right);

root → right = rightLL.head;

l.head = root;

l.tail = rightLL.tail;

return l;

// Both sides are not NULL

LinkedList leftLL = flatten(root → left);

LinkedList rightLL = flatten(root → right);

~~l.head = leftLL.head;~~

leftLL.tail → right = root;

root → right = rightLL.head;

l.head = leftLL.head;

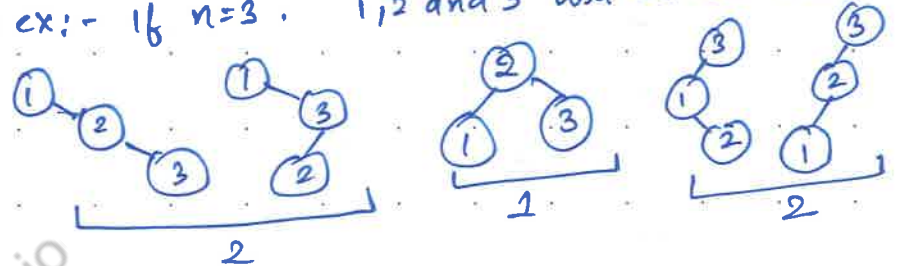
l.tail = leftLL.tail;

return l;

Catalan Number (Very Important)

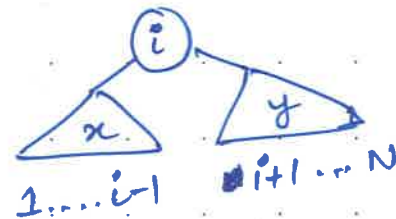
Q. Count no. of BST's that can be formed using N nodes numbered from $1, 2, 3, \dots, n$.

for ex:- If $n=3$. 1, 2 and 3 will be the nodes



→ Total 5 BSTs possible

if $1 \dots i \dots N$



We take a root node i and call the same func on left side and right side of i as x and y .

will give us possible combinations arrangements from both sides.

$$\text{Total no. of BSTs} = {}^xC_1 \cdot 1 \cdot {}^yC_1 = x \cdot y$$

$$f(N) = \sum_{i=1}^N f(i-1) \cdot f(N-i)$$

Idea :- left & Root & Right
 ↓ ↓ ↓
 recursive Fixed Recursive
 call for each call.

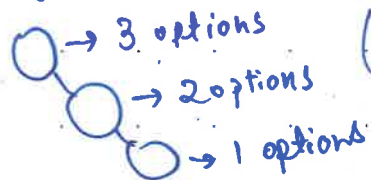
Dry Run :

$N=0$ $f(0) = 1$ → Only 1 possible way of not having a BST
 $N=1$ $f(1) = 1$ → root
 $N=2$ $i=1, f(0) \cdot f(1) = 1 + 1 = 2$
 $i=2, f(1) \cdot f(0) = 1$
 $N=3$ $i=1, f(0) \cdot f(2) = 1 + 2 = 3$
 $i=2, f(1) \cdot f(1) = 1 + 1 = 2$
 $i=3, f(2) \cdot f(0) = 2 + 1 = 3$
 Total = 5

Overlapping subproblems are there so better approach of solving it would be Bottom-up DP.

* Direct Formula :- $\frac{2^n C_n}{n+1}$
 of Catalan Number

→ For binary trees →
 if $n=3$



$(n!)$ for each kind of traversals

$$\therefore \left[\frac{n! \cdot \frac{2^n C_n}{n+1}}{n+1} \right]$$

→ For unlabelled Binary Trees → BST (with no nos. on it) = $\frac{2^n C_n}{n+1}$

Set (STL)

→ Unique elements :
 (1) Set
 (2) Unordered-set
 → sorted
 → random

Set

The value of the element cannot be modified once it is added to the set, though it is possible to remove and add the modified value of that element.

→ begin() → end() → size() → max-size()
 → empty()

Unordered set

Stores elements formed by comb. of key value and mapped value. The key value is used to uniquely identify the element and mapped value is content associated.

It's implemented using Hash-Table

∴ Avg T.C of search, insert and delete is $O(1)$.
 unlike ~~map~~ ~~unordered map~~ map which uses binary search and takes $O(\log n)$.

```

set<int> s;
for(0 → n) { s.insert(arr[i]); }
s.erase(10); → erases 10
auto it = s.find(11); → erases 11
s.erase(it);
for(auto it : s) { cout << it << " "; }

```

Multiset (STL)

- can store multiple elements that have same elements
- stored in a specific → sorted according to ^{integer} first element in each object
- values once can't be modified.
- associative container → elements are referred by their value and not by the index
- underlying data str → BST

```

int arr[] = {10, 20, 30, 20, 10, 10};
multiset<int> m(arr, arr+6);

```

```

m.erase(20);
m.insert(80);
for(auto x : m) { cout << x << " "; }
m.count(10); → give count of 10.

```

```

— x — x — x —
multiset<pair<string, int>> m;
m.insert(make_pair("A", 40));
m.insert("C", 100);
m.insert("D", 10);

```

```

for(auto p : m) { cout << p.first << " "; }

```



```

class person-compare {
public:

```

```

bool operator()(pair<str, int> p1, pair<str, int> p2) {

```

```

return p1.second < p2.second; } }

```

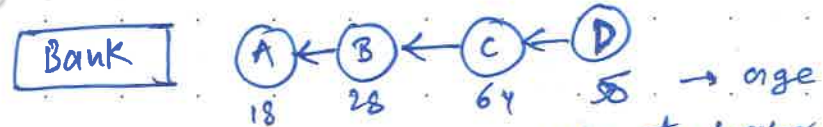
```

multiset<pair<str, int>, person-compare> m;

```

call like this

Heaps / Priority Queue



For ex. we ~~can't~~ give priority to senior citizens and thus first C → D → B → A will be entertained. This kind of problem can be best solved by heaps / P.Q.

Heap →

- ① insert/push — $O(\log n)$
- ② Pop() — $O(\log n)$
- ③ Get — $O(1)$

we maintain it in form of ~~array~~ ^{c.B.T.} ~~array~~ ^{that's y.}

NOTE:

- If we run loop and push all elements of array into priority-queue it will take $n \cdot \log n$
- But we can directly pass the whole array/vector into the P.Q. in $O(n)$ time. (Here STL will use its internal build heap)

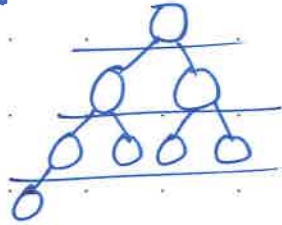
"Your body holds deep wisdom. Trust in it. Learn from it. Nourish it. Watch your life transform and be healthy."

Q What is Heap? (Condⁿ must be satisfied)

① Binary Tree → Heap must follow B.T in order to implement it.

② Complete Binary Tree Property (CBT)

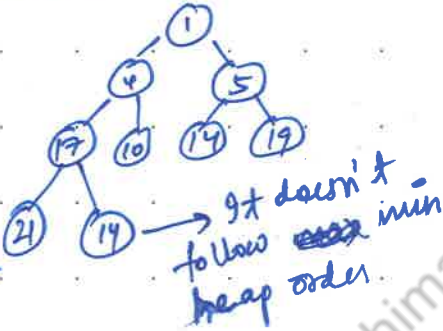
- ~~After~~ Completing each level from left to right and only after completing the whole level it can move to next level.



③ Heap order Property.

- max/min

level wise it should be ~~greater~~ smaller than previous level. (for all nodes)



min → parent child < children

max → parent child > children

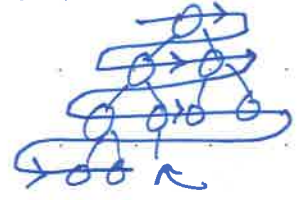
Array Representation of Heap

Q Why we use complete binary tree property to implement Heap?

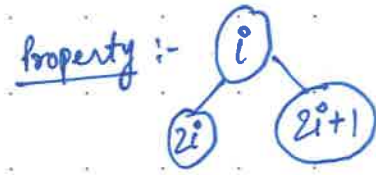
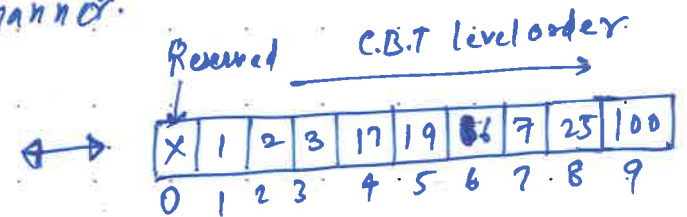
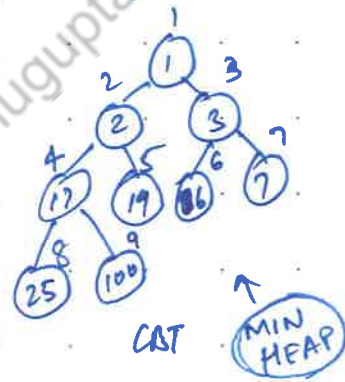
A The reason being that if we do insertion in binary tree so that it satisfies C.B.T property

we must traverse in level order manner and find which leftmost post is empty where a new node can be added.

Everytime we need to insert a new node we must go through all n nodes to add new node which takes $O(N)$ time and is expensive.

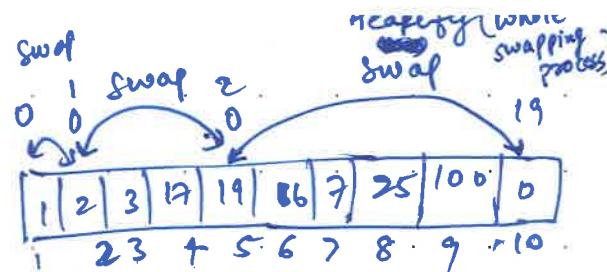
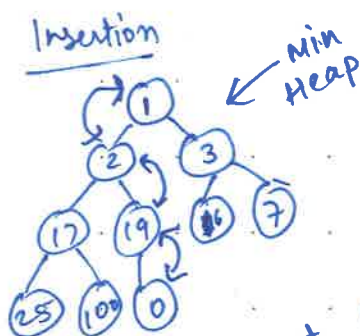


Interestingly, array follows C.B.T in a sequential manner.



- To find parent of a node using index (i) ,
 $i \rightarrow i/2 \rightarrow \text{floor value}$
 child parent

Now we can directly insert in array from rear and Insertion takes $O(1)$ here.
 Visualize in tree and execute using array.



• Heapify [swap until the inserted element remains lesser than its parent]

Insertion $\rightarrow O(\log n) \rightarrow$ Height of Tree (in worst case)
 Get min() $\rightarrow O(1) \rightarrow$ Top of the heap (at index 1)

Insertion code
 Implemented using vector (automatically resized)

```

class Heap {
private:
    vector<int> v;
    bool minHeap; // If true, we implement minHeap else maxHeap
public:
    Heap(int) {}
    bool compare(int a, int b) {
        if (minHeap) return a < b;
        else return a > b;
    }
    // by default we perform minHeap
    Heap(int default-size = 10, bool type = true) {
        v.reserve(default-size);
    }

```

```

v.push-back(-1);
minHeap = type;

void push(int d) {
    v.push-back(d);
    int idx = v.size() - 1; // newly added element idx
    int parent = idx / 2; // its parent

```

while (idx > 1 && compare(v[idx], v[parent])) {
 swap(v[idx], v[parent]);
 idx = parent;
 parent = parent / 2;
}

if idx = 1 then we can't find its parent so we stop here

we defined compare func. coz. its for minHeap and maxHeap as well.

```

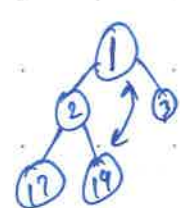
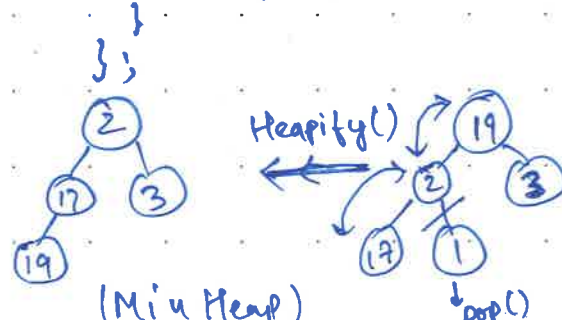
int top() {
    return v[1];
}

```

```

void pop() {
    int last = v.size() - 1;
    swap(v[1], v[last]);
    v.pop-back();
    heapify(1);
}

```



if we remove top element using pop()
 ① $\rightarrow$ We swap first element & last element

~~children~~ Heapify (int idx) { $\longrightarrow$ will be written in private section of heap class
 int left = 2 * idx;
 int right = left + 1;

int min_idx = idx;
 int last = v.size() - 1;

// In order to know whether children exist or not we check if index of each children is $\leq$ max size

if (left $\leq$ last && compare(v[left], v[idx])) {
 min_idx = left;
 }

if (right $\leq$ last && compare(v[right], v[min_idx])) {
 min_idx = right;
 }

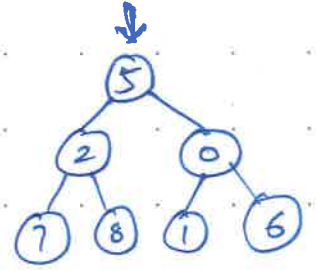
if (min_idx != idx) {
 swap(v[idx], v[min_idx]);
 heapify(min_idx);
 }

$\longrightarrow$ recursive call for ahead heapification

if we could find min children then

Case 1:- There isn't any child
 Case 2:- It is already min itself. stop heapify

Heapify()



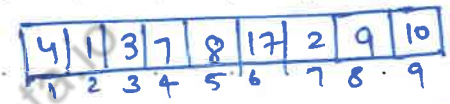
if we heapify(1)
 i.e. element 5

we select its two children and compare with it and swap with min children.

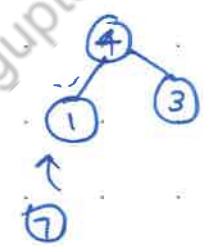
bool empty() {
 return v.size() == 1;
 }

int main() {
 Heap h; / Heap h(15, false); $\longrightarrow$ Max heap
 int n;
 cin >> n; $\longrightarrow$ our own initial max size
 }

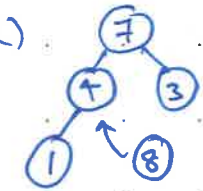
3. Converting Array (or Vector) into Heap (In Place)



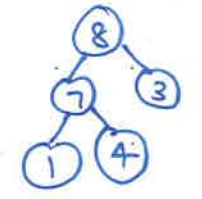
We are creating max heap here.



heapify()



heapify()



and soon...

T.e.

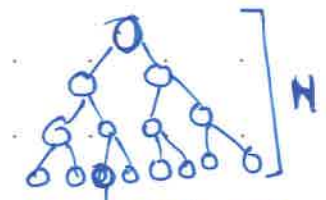
$$1 + 2 + 2^2 + \dots + 2^{h-2} + 2^{h-1} = N$$

$$\frac{2^{h-1} - 1}{2 - 1} + 2^{h-1} = N$$

$$2^{h-1} + 2^{h-1} = N + 1$$

$$2^{h-1} = \frac{N + 1}{2}$$

No. of nodes in last level



This new added element may ~~not~~ heapify to the top in worst case i.e. $O(\log N)$

So, T.C. = $\left(\frac{N+1}{2}\right) \log N = O(N \log N)$

nodes at 1st level $\downarrow$ approx.
for each node
max heapification length.

Code

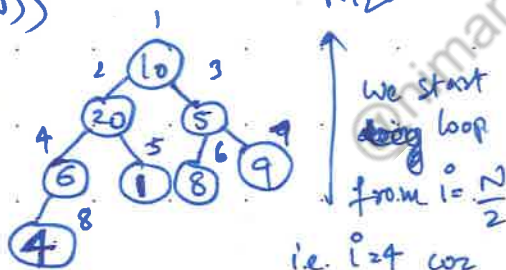
```
void buildHeap (vector<int> &v) {
    for (int i=2; i<v.size(); i++) {
        int idx = i;
        int parent = i/2;
        while (idx > 0 && v[idx] > v[parent]) {
            swap (v[idx], v[parent]);
            idx = parent;
            parent = parent/2;
        }
    }
}
```

$O(N \log N)$ $\rightarrow$

Optimized Approach ($O(N)$)

10 20 5 6 1 8 9 4

we will use bottom up.



i.e. $i \geq 4$ coz rest are leaf nodes.

- We don't need to heapify for each leaf nodes. (we are looking downwards)
- Now we move to 6 but only child it has is less than it, i.e. 4. (No need to heapify)
- We heapify 5 with 9 and so on for all nodes.

$O(N \log N)$ method vs $O(N)$ method

(1, 0) $\rightarrow$ (2 nodes, 1 opⁿ each) $\rightarrow$ (4 nodes, 0 opⁿ)

As we go up, Nodes $\uparrow$ Height $\downarrow$ $\therefore O(N)$

As we go down, Nodes $\uparrow$ Height $\uparrow$ $\therefore O(N \log N)$

Code (optimised)

```
void buildHeapOpt (vector<int> &v) {
    for (int i = (v.size()-1)/2; i >= 1; i--) {
        heapify (v, i);
    }
}
```

[Heapify $\Rightarrow$ Above $\rightarrow$ below]

HeapSort (In-Place)

Algo

1. Build a max heap from the input data.
2. At this point, the largest item is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of heap by 1. Finally, heapify the root of the tree.
3. Repeat step 2 while size of heap is greater than 1.

Diagram (Visual Explanation in other N.B.)

```
void heapsort (vector<int> &arr) {
```

```
    buildHeap (arr);
```

```
    int n = arr.size();
```

```
    while (n > 1) {
```

```
        swap(arr[1], arr[n-1]);
```

```
        n--; → decrease size of array
```

```
        heapify (arr, 1, n);
```

we introduce new argument of size which will heapify till size specified in this so that sorted portion doesn't get heapified by mistake.

```
    }
```

```
int main() {
```

```
    vector<int> v;
```

```
    v.push_back(-1);
```

```
    int n;
```

```
    cin >> n;
```

```
    for (int temp;
```

```
        for (int i = 0; i < n; i++) {
```

```
            cin >> temp;
```

```
            v.push_back(temp);
```

```
        }
        heapsort(v);
```

```
        print(v);
```

Priority Queue STL

specially designed such that the first element of the queue is the greatest of all elements in the queue and elements are in non increasing order. (By default it implements max heap)

You can also specify sep. to use it as min heap.

* Priority is given to max element when ~~poped~~ top() is executed. from the ~~queue~~ priority-queue.

Code

```
priority_queue<int> pq;
```

```
int n;
```

```
cin >> n;
```

```
for (int i = 0; i < n; i++) {
```

```
    int no;
```

```
    cin >> no;
```

```
    pq.push(no);
```

→ $O(\log n)$

```
}
while (!pq.empty()) {
```

```
    cout << pq.top() << " ";
```

```
    pq.pop();
```

Time Comp.

push() - $O(\log n)$

pop() - $O(\log n)$

top() - $O(1)$

I/P :-

5

1 10 7 4 3

O/P :-

10 7 4 3 1

Functional objects in C++ / Functors

```
class Fun {
```

```
public:
```

```
    void operator()() {
```

string s; → can also take string as I/O instances

```
        cout << "Having Fun Inside operator() Fun!";
```

```
    }
```

```
}
```

```
int main() {
```

```
    Fun f; // Constuctor
```

```
    f(); // Overloaded () operator - Func call where f is an object.
```

```
    f("C++");
```

```
}
```

Priority Queue with custom class

```
class Person {
public: string name;
       int age;
```

```
Person() {}
```

```
Person(string s, int a) {
    name = s;
    age = a;
};
```

```
class PersonCompare {
public:
    bool operator()
        (Person A, Person B) {
        return A.age < B.age;
    }
};
```

```
int main() {
```

```
    int n;
```

```
    (cin >> n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        string name;
```

```
        int age; cin >> age;
```

```
        Person p(name, age);
```

```
        priority_queue<Person> pq;
```

```
        pq.push(p);
    }
```

```
    int k;
```

```
    for (int i = 0; i < k; i++) {
```

```
        Person p = pq.top();
```

```
        cout << p.name << " " << p.age << endl;
```

```
        pq.pop();
    }
```

3

*** NOTE ***

When we define compare class for pq then we define predicate statement in opp. manner.

Generally we return $a < b$ (for increasing order) but in this we return $a > b$



```
class PersonCompare {
public:
    bool operator()
        (Person A, Person B) {
        return A.age < B.age;
    }
};
```

```
priority_queue<Person>
vector<Person>
PersonCompare p;
pq;
```

I/P :-

A 20

B 3

C 40

D 10

O/P :-

C 40

A 20

E 15

Reason is end of this topic

Q* When to use Priority Queue?

When we need to find top k largest/smallest elements in ~~an~~ array/vector. Because normal sorting will take $n \log n$ at best but P.Q. will get you each largest/smallest element in $O(\log N)$ time.

Ex:- If you want 3 largest element (3 $\log N$) time out of 1000 elements

* Join the Ropes

I/P :-

2

3

4

6

O/P :- 29

Q. Given N Ropes with diff. sizes, we have to join the ropes together. The cost of joining 2 ropes of different sizes A and B is (A+B). Find min cost to join all the ropes together.

Algo

① Pick two smallest ropes. Join them (Add the sizes)

Cost += (A+B)

② Add the new rope (rope of size A+B) in the min heap.

③ Repeat until a single rope left.

Take minHeap for this.

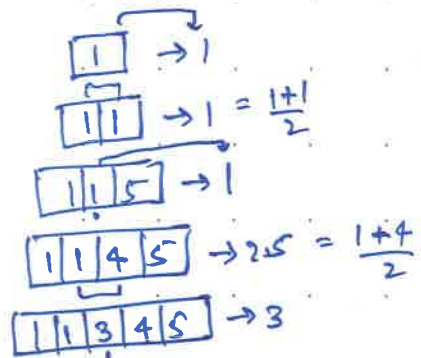
• `priority_queue<int, vector<int>, greater<int>>`

`pq(ropes, ropes+n);`

↳ we can directly assign the P.Q. with the array like this.

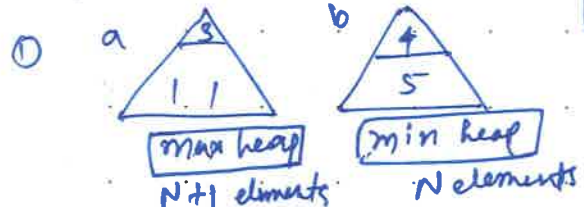
* Running Median of an Integer Stream

We will get an array of elements, of which we have to find median of. Also we have to find the new median after adding a new element in that array.



- If you get a no. larger than median, then your new median will depend on the next larger element.
- If no. < med, median will depend just smaller element than median.

Algo
we will take a max heap & a min heap.

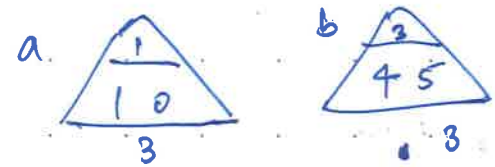


Median = a.top() in this case (N=2)

② Adding 0

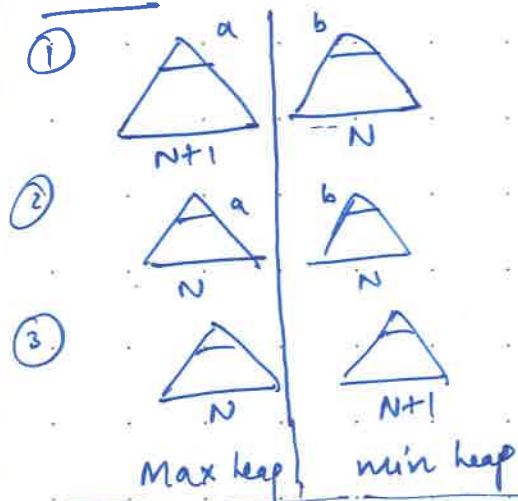
Since 0 is ≤ median (i.e. 2)

b.push(a.pop()) → we are balancing both the heaps such that on adding a new element to a doesn't make it N+2 then, a.push(0)



Median = $\frac{a.top() + b.top()}{2}$

Cases



median = a.top()

median = $\frac{a.top() + b.top()}{2}$

median = b.top()

Code

```
int main() {
    priority_queue<int> leftheap;
    priority_queue<int, vector<int>, greater<int>> rightheap;
```

Insertion technique
if new element ≤ median
a.push(new element)
↳ after balancing both the P.Q. first
if new element > median
b.push(new element)
↳ after balancing the both P.Q.s first

```
int d;
cin >> d;
leftheap.push(d);
float med = d; // cout Karlena (median)
cin >> d;
while (d != -1) {
    if (leftheap.size() > rightheap.size()) {
        if (d < med) rightheap.push(leftheap.top());
        leftheap.pop();
        leftheap.push(d);
        else rightheap.push(d);
    } else if (leftheap.size() == rightheap.size()) {
        if (d < med) leftheap.push(d);
```

med = $\frac{\text{both top sum}}{2}$

```

med = leftheap.top();
else { rightheap.push(d);
      med = rightheap.top(); }

```

```

else { "khud nikalna" }

```

cin >> d; → next new element

* Merge K sorted Arrays

2, 6, 12, 15
1, 3, 14, 20
3, 5, 8, 10

K=3
N=4

For ex. 3 arrays are given with size 4 each.
We have to sort these arrays into one array.

① Brute force.

- Apply mergesort on all arrays pairwise.
Total NK elements
 $NK \log(NK)$

② Better approach

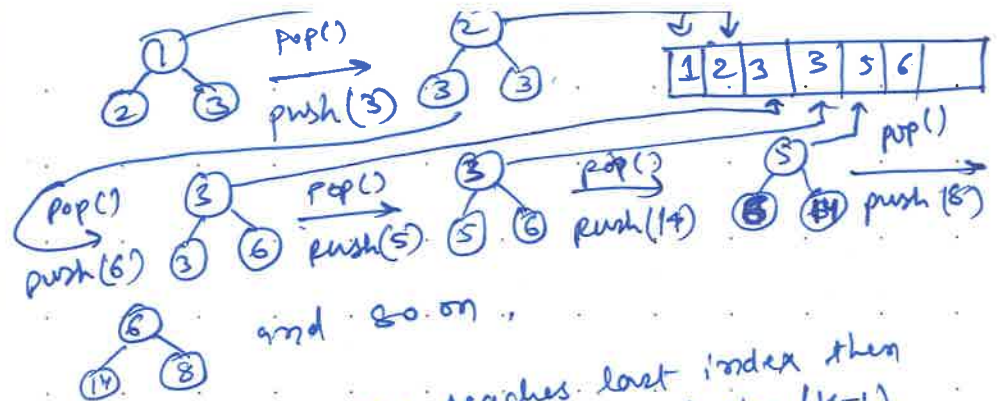
We will maintain a min heap of size K and will select top() and insert it into an array and then increase the index value of array from which top() was selected and push the element in the min heap for same process.
repetitive

2, 6, 12, 15
1, 3, 14, 20
3, 5, 8, 10

K=3

T.C. = $nk \log K$

- push elements at i th, j th and K th index in priority-queue (min heap) type.



• when an array reaches last index then we maintain a min heap of size $(K-1)$ and $K--$ as the other arrays also finish.
while $(K != 0)$

Code

```

typedef pair<int, pair<int, int>> node;
vector<int> mergeKsortedArrays(
    vector<vector<int>>> arr) {

```

```

    vector<int> result;
    priority_queue<node, vector<node>,
        greater<node>> > pq;

```

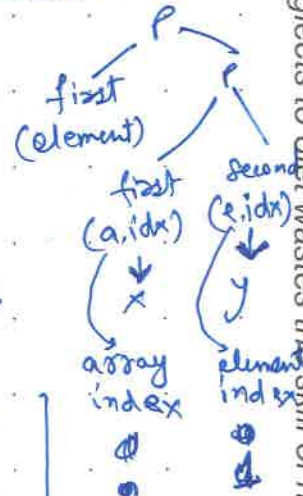
It will compare p.first elements and since it is int it can handle it without creating any compare class.

```

    for (int i=0; i<arr.size(); i++) {
        pq.push({arr[i][0], {i, 0}});
    }

```

no. of arrays in 2D array
column index
element array create pair object



// remove the elements one by one from min heap and add to result vector

while (!pq.empty()) {

node current = pq.top();

pq.pop();

int element = current.first; // element

int x = current.second.first; // row and

int y = current.second.second; // column

result.push_back(element);

// you need to push element at x, y+1 in P.Q.

if (y+1 < arr[x].size()) {
pq.push({arr[x][y+1], {x, y+1}});

}
return result; }

Time complexity = $O(n \log(K))$.

where kn = total no. of elements
 K = no. of arrays

- K^{th} smallest Element in Row & Col Wise Sorted Array

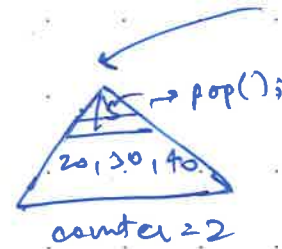
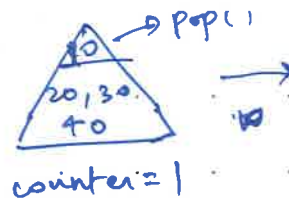
10	20	30	40
15	25	35	45
24	29	37	48
32	33	39	50

Algo

We know that leftmost upper element is smallest.

what we do first is we push the entire 1st row in priority-queue (min heap)

and we maintain a counter.



After 10, only candidates for next smaller elements will be 15 and 20. But we already push 20 in PQ so we will push 15 in.

Now what we have to do. Check only right and downward element for being next smaller elements. If they are already present in P.Q. then good.

$O(N + K \log N)$
for buildheap of first row

push and heapify ($O(\log N)$) for K times

We can optimise size of heap at every instant so that it becomes $O(\log K)$.

$O(K + K \log K) \rightarrow T.C.$

• Instead of taking entire row we will push only K elements from first row in min heap.

* Top K Frequent Elements [LeetCode]

* Deepak and his Journey (CB) #greedy #PQ

Priority Queue STL uses the func. to maintain the elements sorted in a way that preserves heap properties (i.e. that the elements popped is the last according to this strict weak ordering). It maintains a sorted vector separately and pops element from last.

Greedy Algorithms

- are one of the most intuitive algo.
- easy to understand / formulate.
- Greedy Algo always make the choice that looks best at the moment.
- You hope that by choosing a local optimum at each step, you will end up at a global optimum.

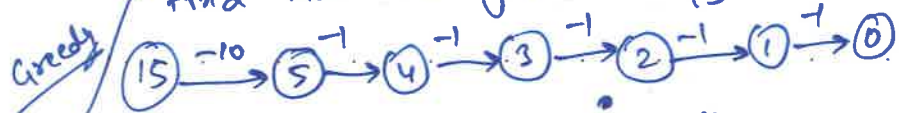
⊛ NOTE:- Greedy sometimes fails

Greedy goes for ~~best~~ local optimum ~~and~~ to find global optimum solⁿ whereas D.P. tries out all the possible ways and gives you the best out of it.

- ⊙ For some cases, greedy works ~~fast~~ much faster than D.P. but in some cases greedy fails and D.P. gives the solⁿ.

Ex: Coins $\rightarrow [1, 7, 10]$

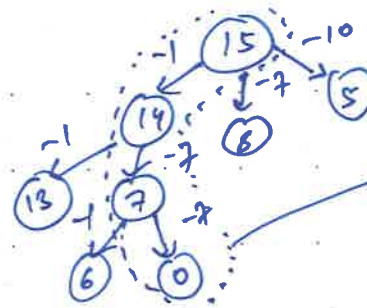
Find min no. of coins for 15.



6 coins

NOTE:- Every greedy solⁿ can be solved by D.P.

P.P.



3 coins only

- Indian coin change code [For which greedy works fine]

```

int coin(int *coins, int n, int money) {
    while (money > 0) {
        int idx = upper_bound(coins, coins + n, money) - 1;
        cout << coins[idx];
        money = money - coins[idx];
        ans++;
    }
    return ans;
}
  
```

↳ gives me just greater element's address of money we have.

coins[] = {1, 2, 5, 10, 20, 50, 100, 200, 500, 2000};

BUSY MAN / ACTIVITY SELECTION CONCEPT

#spoj.com

You have a big schedule of activities. Your aim is to do as much activities as pos.

best idea $\rightarrow$ sort acc. to start time. If:

- $\rightarrow$ duration
- $\rightarrow$ end time.

Check start time and duration.

Code

```

vector<pair<int, int>> v;
cin >> n;
for (0 < n) {
    cin >> s >> e;
    v.push_back(make_pair(s, e));
}
  
```

opt:- 1

we move one element leftwards

Sort (v.begin(), v.end(), compare);
 → bool compare(pair<int> p1, pair<int> p2) {
 // start picking activities
 return p1.second < p2.second; }
 int res = 1;
 int fin = v[0].second;

// iterate over remaining activities

for (int i = 1; i < n; i++) {
 if (v[i].first >= fin) {
 fin = v[i].second;
 res++; }
 cout << res;
 v.clear(); }
 → starting time of next should be more than ending time previous activity

Connecting Wires

There are n white dots & n black dots equally spaced in a line.

Connect white dot with black dot, with min total length of wire. Find total length of wire.

Ex:-



Algo
 Connect white dot with its nearest black dot.

white dot[] = [3, 4, 1]

black dot[] = [2, 5, 6]

Sort both arrays and subtract corresponding elements.

- Biased Standing concept - (training...)
 (DIY)

Kingdom Defense Code:

Algo

1) Sort x coordinates and y coordinates array of towers.

2) Pairwise find max diff in each case (i.e. x coords and y coords). [Consider corner cases]
 from left to end of grid → $x[0] - 1 - 0$ from start to first x coord
 from left to end of grid → $x[n-1]$ height of grid

same for y coordinate array also.

- Chopsticks [TACHSTCK - Problem code (Codechef)]

Done.

- Load Balancer [spoj.com]

I/P

0 6 10 0

O/P

4

Exp.

$(6+10)/4 \Rightarrow 4$

① [1 5 9 1]

② [2 4 8 2]

③ [3 4 6 3]

④ [4 4 4 4]

→ 4 steps

[4 4 4 4]

The array we want

We need to distribute elements such that each bucket contains an equal no. of elements. You have to tell how many steps it will take if we can shift only 1 element to either or both neighbours at a time.

Algo

0 6 10 0

Corner case :- if sum/no. of buckets $\neq 0$
print(-1)

else.

0 6 10 0 $\rightarrow (-4)$

Requires
4 elements
 (-4)

Can give
off 2
elements
 $(+2)$

So this
subarray
requires
2 elements from rest of the array
 $(-4 + 2 = -2)$

load = sum/n;
diff = 0;
for (int i = 0; i < n; i++)
diff += (arr[i] - load);
int ans = abs(diff);
max-load = max(max-load, ans);
}

max-load comes out to
be 4. (requirement for a
subarray)

max
no. of element
shifts required

at any
instant [until 4 shifts takes
place all of the
minor shifts take
place simultaneously]

Expedition (Spoj.com)



P = 10

Petrol truck
has in the bag

Petrol at this stop
10 5 2 4 Town
10 14 20 4 25

distance

Q. Consider the capacity of
truck's petrol tank to be
infinite. find the min.
no. of stops truck needs

to stop in order to reach town.

Algo

(i) First we arrive at first station at
10 units distance. we will use priority Queue
to store the petrol value. we insert
10 units of petrol. At this stop, we already
used our reserve of 10 units petrol so
now $P = 0 + 10 = 10$.

(ii) Now at next stop we push 5.
At this point we are left with $P = 6$
we can also reach next stop with this
much petrol, so we push 2 also. At
this point we are left with $P = 0$ petrol.
(iii) So $P = 0 + P.g. top()$ $\rightarrow 5$ in this case
 $P = 5$

(iv) Next stop at 21 we push 4. At this
point $P = 4$. we still have enough to
reach next stop i.e. Town.

$\therefore$ Ans :- 2 stops

Code

```
flag = 0; ans = 0;
vector<pair<int, int>> v;
priority_queue<int> pq;
for (int i = 0; i < n; i++) {
    cin >> d >> f;
    v.push_back(make_pair(d, f));
}
sort(v.begin(), v.end(), compare);
```

I/P:-

1	(Testcase)	
4	(stops)	
4 4	(D1 F1)	
5 2	(D2 F2)	
11 5	(D3 F3)	
15 10	(D4 F4)	Petrol
25 10	(L P)	Distance from Town

Distance from
Town

DIY.

"Exercise is king. Nutrition is queen. Put them together and you've got a kingdom."

```

cin >> D >> F; // F is fuel tank has
for (int i = 0; i < n; i++) {
    V[i].first = D - V[i].first;
}
prev = 0;
x = 0;
while (x < n) {
    if (F >= (V[x].first - prev)) {
        F = F - (V[x].first - prev);
        Pq.push(V[x].second);
        prev = V[x].first;
    } else {
        // Check if you have some fuelling station
        // visited in the priority queue.
        if (Pq.empty()) {
            flag = 1;
            break;
        }
        F += Pq.top(); // otherwise refuel the
        // truck with max available petrol
        Pq.pop();
        ans = ans + 1;
        continue;
    }
    x++;
}
// travelled n fuel
// stations
// now the case b/w
// last station and
// town.

```

distance of each stop w.r.t truck

Total distance of truck from town.

That means truck can reach this point with its petrol.

Situation where truck has to go 14 distance to reach town/next stop. In that case after taking in 8 still $P=8$ not enough so we continue unless $(F \geq V[x].first - prev)$ becomes true.

8 3 6

HEALTH INSURANCE WITH MATERNITY & NEW BORN COVER



Wait-period of just
9 months for maternity
related claims



Covers delivery
expenses including
pre & post natal



Expenses from birth till
90 days are covered



In-Patient Hospitalization



30-Day Pre & 60-Day
Post Hospitalization



Day Care Treatment

HIGH DEDUCTIBLE HEALTH INSURANCE PLAN



Flexibility to choose
combination of deductible
& Sum Insured



Annual Health
Check-up



In-Patient Hospitalization



30-Day Pre & 60-Day
Post Hospitalization



Day Care Treatment



Organ Donor Cover

```

if (flag == 1) {
    cout << r-1 << endl; }
    continue; } // otherwise
D = D - v[n-1].first;
if (F == D)
    cout << ans << endl;
    continue; }
while (F < D) {
    if (pq.empty()) {
        flag = 1; break; }
    f = pq.top();
    pq.pop();
    ans = ans + f;
}
if (flag == 1) { cout << r-1; }
    
```

doesn't execute rest
of the bottom code
and goes to next
testcase (while(t--))

- If we have upper bound to a greedy solⁿ then always we can use binary search to find the sol.

Maximum circles

We can change this Q into activity selection problem.

No. of ~~one~~ circles to be removed in order to make all remaining circles non-overlapping = Total circles - max No. of non-overlapping circles

Dynamic Programming

• Where we use?

① Optimal Substructure

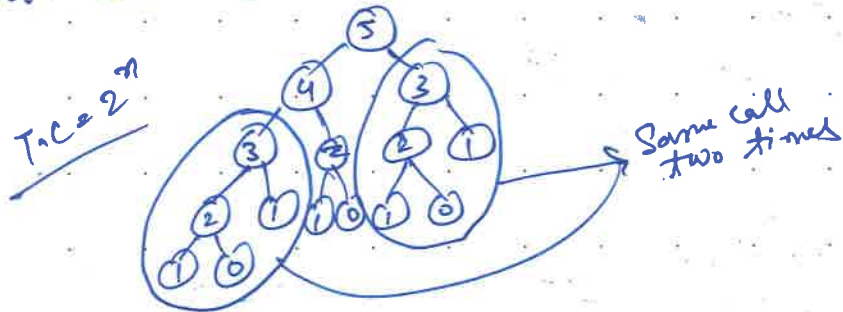
② Overlapping Subproblems

→ a problem is said, if an optimal solution can be constructed from optimal solⁿ of its subproblems.

(breaking down problems to get optimal solⁿ out of it calculate bigger one)

→ if problem is said to have overlapping subproblems if the problem can be broken down into subproblems which are reused several times or a recursive algo for the problem solves the same subproblem over & over rather than always generating new subproblems.

For ex:- Fibonacci code



Two methods

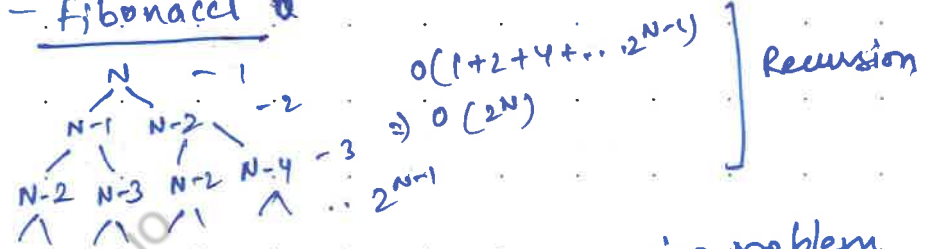
- Top down approach (Recursion + Memoisation)

(Memoisation Approach)

then easily you can use storing the new values

- Bottom up Approach (Iterative)
(Tabulation method)

- Fibonacci



③ DP can be applied any recursive problem with overlapping problem.

Top Down (Fibonacci)

```
int fib(int n, int dp[]){
```

```
    if (n <= 0 || n <= 1) {
```

```
        return n;
```

```
    // look up
```

```
    if (dp[n] != 0) {
```

```
        return dp[n];
```

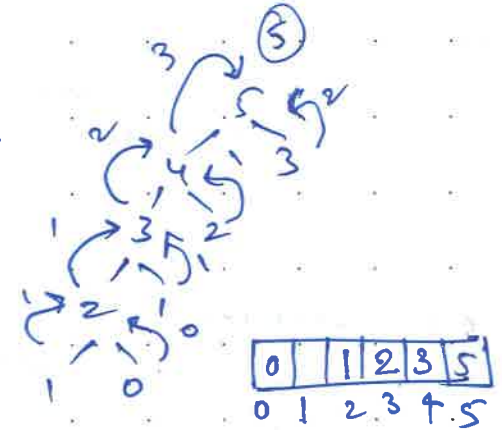
```
    int ans;
```

```
    ans = fib(n-1) + fib(n-2);
```

```
    return dp[n] = ans;
```

```
}
```

it returns the update array



firstly, dp[] is initialized with 0. then as we move from down to up we store values.

Complexity $\rightarrow$ Space $\rightarrow O(N) \rightarrow$ call stack + array
 Time $\rightarrow O(N)$.

Bottom Up Approach (Iterative Approach)

We start solving from bottom/base case itself and start storing the value in array. (and find further values using previous values)

```
int fib(int n){
    int dp[100] = {0};
    dp[1] = 1;
    for (int i=2; i<=n; i++){ dp[i] = dp[i-1] + dp[i-2]; }
    return dp[n]; }
```

Time Complexity = $O(N)$

Space Complexity = $O(N)$

We can optimise space by maintaining only two variables (instead of array) to store previous two values.

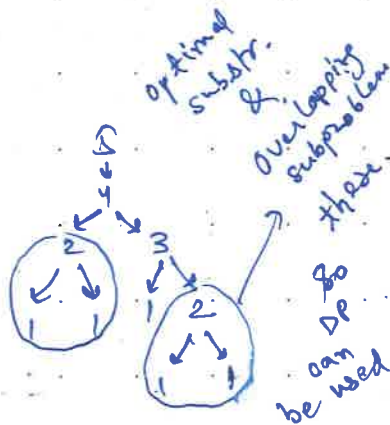
$a=0, b=1$ for $(i=2; i<=n; i++)$ {
 $c = a+b;$
 $a = b;$
 $b = c;$ }

Matrix Exponentiation will help us reduce the time comp. to $O(\log N)$.

- Min Steps to One

$n \rightarrow 1$

- ① $n \Rightarrow n/3$ if $n \% 3 == 0$
- ② $n \Rightarrow n/2$ if $n \% 2 == 0$
- ③ $n \Rightarrow n-1$

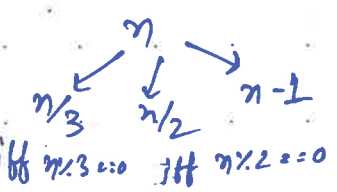


$\Rightarrow$ Greedy won't work

Top Down

```
int minSteps(int n){
    if (n==1) return 0;
    //lookup if n is already completed
    if (dp[n] != 0) return dp[n];
    int op1, op2, op3;
    op1 = op2 = op3 = INT_MAX;
    if (n%3 == 0) op1 = minSteps(n/3) + 1;
    if (n%2 == 0) op2 = minSteps(n/2) + 1;
    op3 = minSteps(n-1) + 1;
    int ans = min(min(op1, op2), op3);
    return dp[n] = ans; }
```

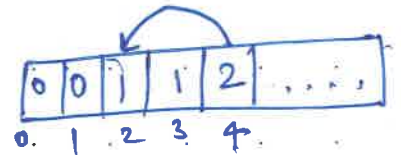
dp[1] = 0
 case checked firstly everytime



use pre-assign max value so that while return min we return min.

Bottom Up

```
int minStep(int n){
    int dp[100] = {0};
    dp[1] = 0;
    for (int i=2; i<=n; i++){
        int op1, op2, op3;
        op1 = op2 = op3 = INT_MAX;
        if (i%3 == 0) op1 = dp[i/3];
        if (i%2 == 0) op2 = dp[i/2];
        op3 = dp[i-1];
        dp[i] = min(min(op1, op2), op3);
    }
```



return dp[n]; }

Min. Coin Change

Greedy works fine for Indian currency -
but not for other types of currency.

For ex:- coins = [1, 7, 10]

$N = 15 - 10 = 5$
 $5 - 1 = 4$
 $4 - 1 = 3$
 $3 - 1 = 2$
 $2 - 1 = 1$
 $1 - 1 = 0$
 6 coins (greedy fails)

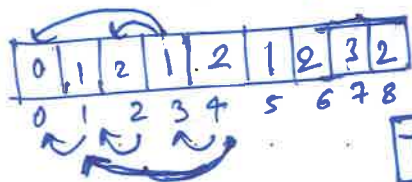
$7, 7, 1$
 $\downarrow$
 15
 3 coins

Algo (Top down)

$f(n) = \min [f(n - \text{coins}[i])] , \text{ if } n - \text{coins}[i] \geq 0$
 $\forall i$

Bottom up Approach

coins = [1, 3, 5] $N = 8$ Rs.



$dp[n] = \min [dp[n - \text{coins}[i]]]$
 $\forall i$

$T, C \rightarrow O(TN)$
 No. of coins
 Types of coins

Code

```

int minCoins(int n, int coins[], int T, int dp[]) {
    if (n == 0) {
        return 0;
    }
    if (dp[n] != 0) {
        return dp[n];
    }
    int ans = INT_MAX;
    for (int i = 0; i < T; i++) {

```

if (n - coins[i] > 0) {

int subprob = minCoins(n - coins[i], coins, T, dp);

ans = min(ans, subprob);

dp[n] = ans;
 return dp[n];
 }



Bottom up (Code)

```

int minCoins(int n, int coins[], int T) {
    int dp[100] = {0};
    for (int m = 1; m <= N; m++) {
        dp[m] = INT_MAX;
        for (int i = 0; i < T; i++) {
            if (n - coins[i] >= 0) {
                int subprob = dp[n - coins[i]];
                dp[m] = min(dp[m], subprob + 1);
            }
        }
    }
    return dp[N];
}

```

Wines Problem

wines prices = [2, 3, 5, 1, 4]

P[i] = Price of i-th wine

Every year price wines increases such that
 $y^{\text{th}} \text{ year} = P[i] * y$

Each year you can sell only one wine from either extreme sides of the array. Find max Profit.

"Drinking water is essential to a healthy lifestyle."

Greedy App. $2(1) + 3(2) + 4(3) + 1(4) + 5(5) = 49$ Rs

Another way:- $2(1) + 4(2) + 1(3) + 3(4) + 5(5) = 50$ Rs

Greedy fails ✗

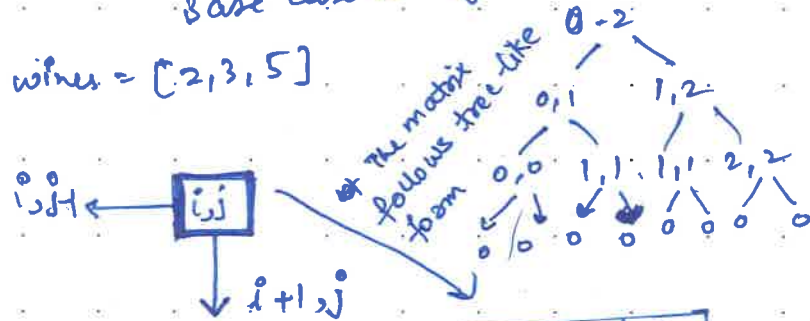
$$f(i, j) = \begin{bmatrix} p[i] * y + f(i+1, j), y+1 \\ \max \left[p[j] * y + f(i, j-1), y+1 \right] \end{bmatrix}$$

(Overlapping subproblems + optimal substructure) is there so DP.

Base case :- if $i > j \rightarrow 0$

Ex:-

wines = [2, 3, 5]



6 _{0,0}	13 _{0,1}	23 _{0,2}
0 _{1,0}	9 _{1,1}	21 _{1,2}
0 _{2,0}	0 _{2,1}	15 _{2,2}

For arrays (1,0), (2,0), (2,1) ~~and~~ $i > j$ and violates the base case so value will be 0.

To reduce the time & space we can directly start from right diagonal.

(0,0) $\rightarrow$ ~~at~~ $[0,0] = 2$ and after 2 years and in 3rd year this case will come. ~~also~~ In 3rd year if we remove 2 then profit $= 2 * 3 = 6$

• for (1,1) $\rightarrow [1,1] = 3$. Max profit at 3rd year $= 3 * 3 = 9$

• for (2,2) $\rightarrow 5 * 3 = 15$

• Now subarrays of size 2 case will come :-

$\rightarrow (0,1) \rightarrow [0,1] = [2,3]$ $\rightarrow 2 * 2 + (1,1)$
(In this, we removed 2 at 2nd year) $= 4 + 9 = 13$

(In this, we removed 3 at 2nd year) $3 * 2 + (0,0)$

$$\Rightarrow 6 + 6 = 12$$

Max = 13

$$\therefore (0,1) = 13$$

$\rightarrow (1,2) \rightarrow 3 * 2 + (2,2) = 6 + 15 = 21$

$$5 * 2 + (1,1) = 10 + 9 = 19$$

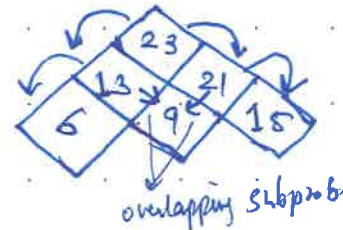
• Now comes Subarray of size 3 i.e [2,3,5]

We removed 2 at 1st year

$$2 * 1 + (1,2) = 2 + 21 = 23$$

At 1st year we removed 5 $\rightarrow 5 * 1 + (0,1) = 5 + 13 = 18$

Max = 23



Code

```

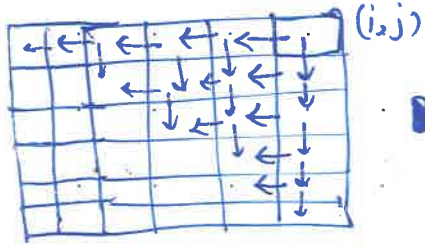
int profit (int wines[], int i, int j, int y) {
    if (i > j) return 0;
    if (dp[i][j] != 0) return dp[i][j];
    int op1 = wines[i] * y + profit(wines, i+1, j, y+1);
    int op2 = wines[j] * y + profit(wines, i, j-1, y+1);
    dp[i][j] = max(op1, op2);
    return dp[i][j];
}

```

```

int op2 = wires[j] * y + profit(wires, i, j-1, y+1);
dp[i][j] = max(op1, op2);
return dp[i][j];
}

```



• Base. Traversal (cells) in dp 2D array. Cells will stop and won't go beyond right diagonal because $(i > j)$ ← Base case.

Maximum Subarray Sum (Follows Kadane's Algo)

max-so-far = 0 2 7 7 12 15 15 20 20 20 → Ans.

dp[] = 0 2 7 6 12 15 13 20 15 17

-3 2 5 -1 6 3 -2 7 -5 2

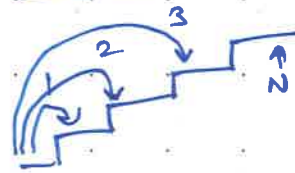
```

int maxsum(int a[], int n) {
    int dp[100] = {0};
    dp[0] = a[0] > 0 ? a[0] : 0; // If first element is
    // tve then dp[0] = a[0]
    else dp[0] = 0;
    int max-so-far = dp[0];
    for (int i = 1; i < n; i++) {
        dp[i] = dp[i-1] + a[i];
        if (dp[i] < 0) { dp[i] = 0; } // If current-sum comes
        // out to be -ve then we rather set dp[i] = 0 so that it won't
        // affect further +ve values.
        max-so-far = max(dp[i], max-so-far);
    }
    return max-so-far;
}

```

• Space optimisation - Instead of taking whole dp[] we can maintain a variable (current-sum) because to find sum at each i we require current sum of previous posⁿ only. ∴ current-sum += a[i];

Ladders problem



N, K
↓
Max. no. of steps you can take
If $K = 3$,
then 1, 2 & 3 size of jumps can be taken.

$f(n) = f(n-1) + f(n-2) + \dots + f(n-K)$

Code (Topdown)

```

int ladders(int n, int K, int dp[]) {
    if (n == 0) {
        return 1;
    }
    if (dp[n] != 0) {
        return dp[n];
    }
    int ways = 0;
    for (int i = 1; i <= K; i++) {
        if (n-i >= 0) {
            ways += ladders(n-i, K, dp);
        }
    }
    return dp[n] = ways;
}

```

If you have recursive calls as "OR" options then sum those calls for ex. in ladders problem I can either come to N from N-1 / N-2 / ... N-K so we add all of them.

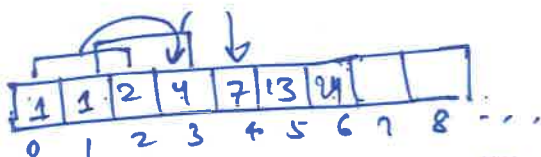
Bottom up

Code

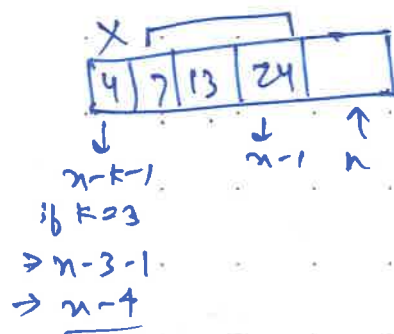
```
int lad(int n, int k) {
    if (int dp[100] = {0})
        dp[0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            dp[i] = 0;
            for (int i = 1; j <= k; j++) {
                dp[i] += dp[i-j];
            }
        }
    }
    return dp[n];
}
```

for 2, previous 2 elements
for 3, previous 3 elements sum
for 4, previous 3 elements sum
and so on.

$O(N \cdot K)$
for n all we iterate back over K cells



Optimisation (we will maintain last 3 elements)



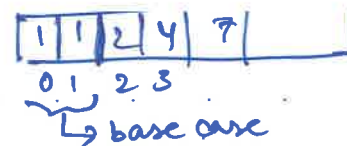
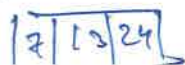
dp[n] requires last three elements' sum
so we ~~to~~ remove first element from previous sliding window i.e. 4 in this case.

$$dp[n-1] - dp[n-k-1]$$

$$dp[n] = 2 * dp[n-1] - dp[n-k-1]$$

this is sum of first two element of new window

i.e. $[7, 13] = 20$
we again add dp[n-1],



```
int lad(n, k) {
    int dp[100] = {0};
    dp[0] = dp[1] = 1;
    for (i = 2; i <= k; i++) {
        dp[i] = 2 * dp[i-1];
    }
    for (k+1 <= n) {
        dp[i] = 2 * dp[i-1] - dp[i-k-1];
    }
    return dp[n];
}
```

MCM (concept in DAA N.B.)

Code (Bottom up)

```
int MCM(vector<int> A) {
    // Top-down is easy
}
```

```
int dp[200][200];
memset(dp, 0, sizeof(dp));
int n = A.size();
for (int i = 1; i <= n; i++) {
    for (int j = i+1; j <= n; j++) {
        dp[i][j] = 0;
        while (i < j) {
            int val = INT_MAX;
            for (int pivot = i+1; pivot < j; pivot++) {
                // ...
            }
        }
    }
}
```

```
int MCM(int p[], int n) {
    int m[n][n];
    int i, j, k, l, q;
```

For index 2,
 $dp[2] = 2 * dp[2-1] - 0$
 $= 2 * 1 = 2 - 0$
 $= 2$

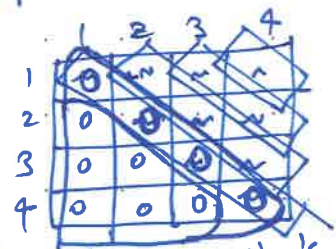
for index 3,
 $n-k = 3-3 = 0$
It's zero so we don't have to remove anything

$$\therefore dp[3] = 2 * [3-1] - dp[n-k]$$

$$= 2 * 2 - 0$$

$$= 4$$

$\therefore$ if $(n-k \leq 0)$
then $dp[n-k] = 0$
else $dp[n-k] > 0$



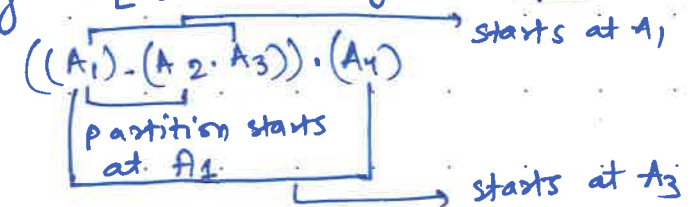
this position's gonna be 0.

so ignore it.
We will solve diagonally as shown

In this case we only require $m[1,4]$ element because that's cost of multiplication of all matrices. i.e 158.

so min. no. of multiplications = 158
order of multiplication can be found by using $s[i][j]$ array i.e

1	2	3	4
1	1	3	



Optional Game Strategy

Ex:- $\{2, 1, 7, 3\}$

We can only pick from extreme ends. P1 and P2 plays game where P1 starts picking first. Find with how much value P1 can win. (Both play optimally)

→ Greedy won't work because at a time we are restricted to pick up out of 2 values and after removing the picked up value we don't know what values are on extreme ends now.

Algo

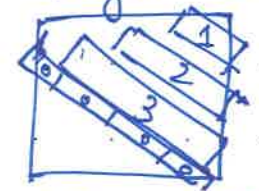
$\downarrow$
7, 3, 1, 6, 2, 8, 10, 11
 $\downarrow$

→ if P1 picks up 7 then P2 can pick either 3 or 11. so left up array for P1 will be either $arr[i+2, j]$ or $arr[i+1, j-1]$
→ if P1 picks up 11 then on P1's next move it will have $arr[i, j-2]$ or $arr[i+1, j-1]$.

```
for (i=1; i<n; i++) {
    // m[1,1], m[2,2]
    // m[3,3], m[4,4]
    m[i][i] = 0;
}
```

```
for (L=2; L<n; L++) {
    for (i=1; i<n-L+1; i++) {
        j=i+L-1;
```

// L is ~~the~~ left diagonal lengths



// i is row coordinate

// pivot is k in the formula of MCM

$$m[i][j] = \min_{i \leq k < j} \{ m[i][k] + m[k+1][j] + d_i * d_k * d_j \}$$

$$\begin{bmatrix} & & & \end{bmatrix} \times \begin{bmatrix} & & & \end{bmatrix} = \begin{bmatrix} & & & \end{bmatrix}$$

a x b b x c a x c

• No. of multiplications = a x b x c

```
    m[i][j] = INT_MAX;
    for (k=i; k<j-1; k++) {
```

```
        q = m[i][k] + m[k+1][j]
        + p[i-1] * p[k] * p[j];
```

```
        if (q < m[i][j]) {
            m[i][j] = q;
        }
```

// we want min scalar multiplications so if $q < m[i][j]$ we set it as q.

```
    }
}
return m[1][n-1];
```

For ex:- for Q.

A_1, A_2, A_3, A_4
5x4 4x6 6x2 2x7
S 1 2 3 4

matrix of partition value.

		1	1	3
2			2	3
3				3
4				

m	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

(Refer my DAF notes)

$$\therefore f(i, j) = \max \left[\begin{aligned} & a[i] + \min \left[\begin{aligned} & f(i+2, j), \\ & f(i+1, j+1) \end{aligned} \right] \\ & a[j] + \min \left[\begin{aligned} & f(i, j-2), \\ & f(i+1, j-1) \end{aligned} \right] \end{aligned} \right]$$

Finds out
max value for P1.

if (i > j) {
return 0; }
// base case

* ~~case~~ ATQ, P2 plays optimally
So P2 will leave minimum
possible subarrays
extreme end values
for us.

Bottom up:

	1	2	3	4	5	6
1	0					
2	0	0				
3	0	0	0			
4	0	0	0	0		
5	0	0	0	0	0	
6	0	0	0	0	0	0

array size →
6 4 2 0

i = j only one element
left but
we are eliminating
elements at rate
of 2 (Both P1 & P2
plays)
So last element
won't left.

~~i = j + 1~~

for (j = i + 1)

only 2 elements
will be left

So greedy way will work

→ (can be taken as one more
base case)

~~Longest Common Subsequence~~ [DP problem notes]

Rod Cutting Problem

	0	1	2	3	4	5	6	7	← Index
N=8									
length	1	2	3	4	5	6	7	8	
Price	1	5	8	9	10	17	17	20	

Max Profit
= 22

- if you divide this rod into 8 parts with length 1. You will get profit $1 \times 8 = 8$
- if you don't divide and consider complete rod then $20 \times 1 = 20$.



$$(17) + (5) = 22 \rightarrow \text{Max profit}$$

Algo

if we pick size of rod to be cut first as:-
price[0] + f(N-1)



$$\text{price}[1] + f(N-2)$$

$$\text{price}[2] + f(N-3)$$

we consider
price in sum
of cut rod

↳ call recursion
on left out
rod.

$$\text{price}[N-1] + 0$$

Code

```
int max-prod(int prices[], int n) {
    if (n <= 0) {
        return 0;
    }
```

```
    int ans = INT_MIN;
```

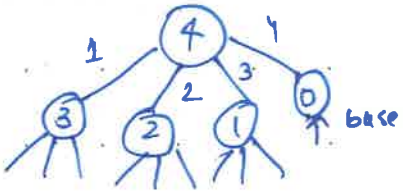
```
    for (int i = 0; i < n; i++) {
```

```
        int cut = i + 1;
```

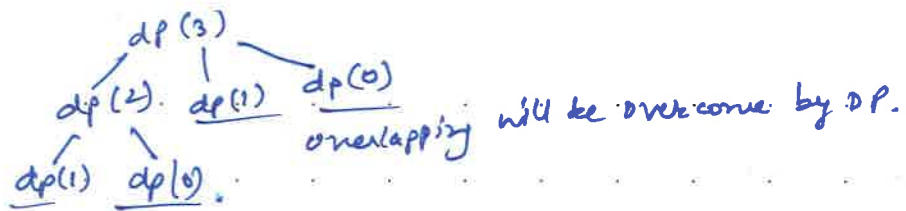
```
        int current-ans = prices[i] + max-profit(prices, n-cut);
```

```
        ans = max(ans, current-ans);
```

```
    }
    return ans;
}
```



cut length = index + 1

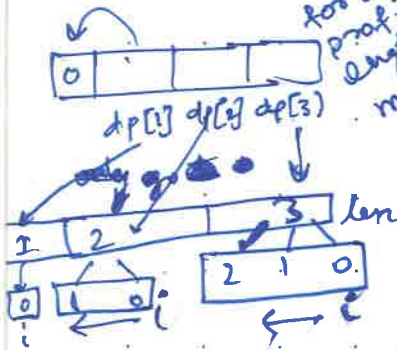


Code (Bottom up)

```
int max-pro (int *prices, int n) {
    int dp[n+1];
    dp[0] = 0;
    for (int len = 1; len <= n; len++) {
        for (int i = 0; i < len; i++) {
            int cut = i+1;
            int current-ans = prices[i] + dp[len-cut];
            ans = max(current-ans, ans);
        }
        dp[len] = ans;
    }
    return dp[n];
}
```

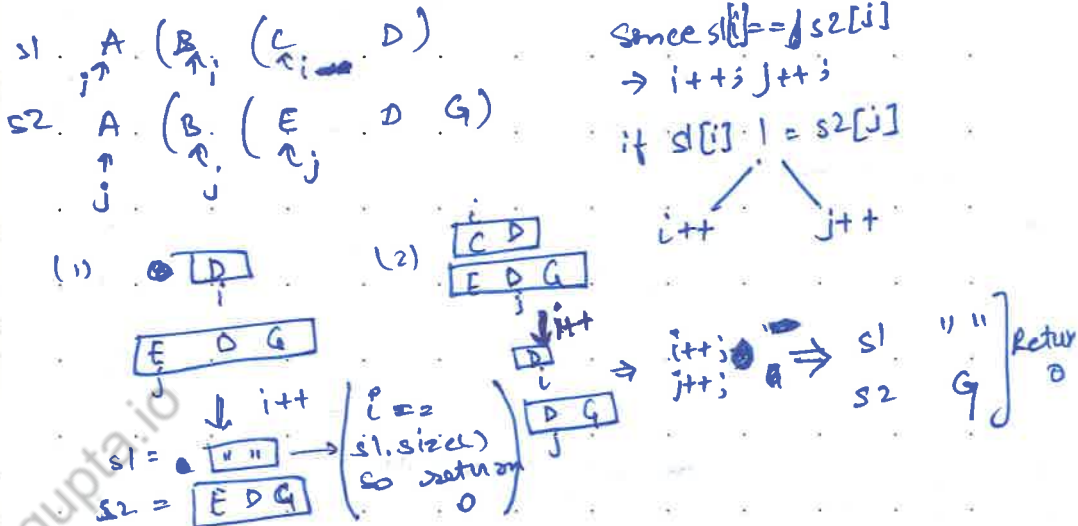
Look for all values of length to be cut at an instant

Look where we take a predefined size of rod and find for all possible profits for that length. then we move on bigger size of rods.



Count no. of binary strings with consecutive 1's

LCS (Longest common subseq) [Refer my python notes] for tree representation



Code (Recursion)

```
int lcs (string s1, string s2, int i, int j) {
    if (i == s1.length() || j == s2.length()) {
        return 0;
    }
    if (s1[i] == s2[j]) {
        return 1 + lcs(s1, s2, i+1, j+1);
    }
    int op1 = lcs(s1, s2, i+1, j);
    int op2 = lcs(s1, s2, i, j+1);
    return max(op1, op2);
}
```

Top down

```
int lcs (string s1, string s2, int i, int j, vector<vector<int>> &dp) {
    if (i == s1.length() || j == s2.length()) {
        return dp[i][j] = 0;
    }
    if (dp[i][j] != -1) {
        return dp[i][j];
    }
    if (s1[i] == s2[j]) {
        return dp[i][j] = 1 + lcs(s1, s2, i+1, j+1, dp);
    }
    int op1 = lcs(s1, s2, i+1, j, dp);
    int op2 = lcs(s1, s2, i, j+1, dp);
    return dp[i][j] = max(op1, op2);
}
```

```

if (s1[i] == s2[j]) {
    return dp[i][j] = 1 + lcs(s1, s2, i+1, j+1, dp);
}

```

```

int op1 = lcs(s1, s2, i+1, j, dp);
int op2 = lcs(s1, s2, i, j+1, dp);
return dp[i][j] = max(op1, op2);
}

```

K-Ordered LCS

If given that at most k times we can change char of a string. Then find LCS. static int ptr=0;

```

if (ptr < k) {
    ptr++;
    return dp[i][j] = 1 + lcs(s1, s2, i+1, j+1, dp);
}

```

Longest Increasing Subsequence

array [1, 5, 2, 3, 4, 9, 6, 10]

1 2 3 4 6 10 → 6

1 5 9 10 → 4

1 2 3 4 9 10 → 6

dp[i] → length of LIS ending at i. (included a[i])

Standard

0	1	2	3	4	5	6	7
1	5	2	3	4	9	6	10

$O(N^2)$

dp → [1, 2, 2, 3, 4, 5, 5, 6]

$dp[i] = \max(dp[j]) + 1$ $j < i$ & $a[j] < a[i]$

- for a $dp[i]$ we traverse preceding indexes to get max value of $dp[j]$ such that its also $a[j] < a[i]$

code

```

int dp[n] = {0};
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        if (a[j] < a[i]) dp[i] = max(dp[j], dp[i]);
    }
    dp[i]++;
}

```

Better method (Using Binary search)

$O(n \log n)$

$dp[pos] < a[i] < dp[pos+1]$ — *

$dp[pos+1] = a[i]$

where $dp[pos+1] > dp[pos]$

0	1	2	3	4	5
$1e9$	$1e9$	$1e9$	$1e9$	$1e9$	$1e9$

abs of monotonic order, → binary search.

Ex: -

1	3	5	9	15
---	---	---	---	----

if we want to insert 10 then between 9 & 15 we see that Cond* is satisfied so we can update $dp[pos+1] = a[i]$

Here what we are doing is that we are updating dp array where a smaller no. comes and then we see if it lies b/w a range in dp[] and if it does then we replace it with strictly larger one. So that we'll be able to get max no. of numbers in LIS.

Q How we find replaceable no. in the sorted array → we will use upperbound() to find just greater element.

For array [1, 2, 3, 4, 5, 6]

0	1	2	3	4	5	6
$1e9$	$1e9$	$1e9$	$1e9$	$1e9$	$1e9$	$1e9$

In stat, we will have when we check arr[0] only 1 length comes out we find upper bound and find out $1e9$ ~~stat~~

(length of LIS own)

at index 1 is replaceable, so we $dp[1] = 1$, and
 so for 2, $1 < 2 < 10^9$ so we $dp[2] = 2$
 and we finally get $dp[1] \rightarrow dp[2]$ (where $dp[2] > dp[1]$)

dp

1e9	1	2	3	4	5	6
-----	---	---	---	---	---	---

LIS = 6

(with no condⁿ below)

For non-decreasing subseq

0	1	2	3	4	5	6
1e9	1e9	1e9	1e9	1e9	1e9	1e9

INT-MIN INT-MAX N=6

Code

$dp[0] = \text{INT_MIN};$

for (int i = 1; i <= n; i++)

$dp[i] = \text{INT_MAX};$

for (int i = 0; i < n; i++)

int length = upper_bound(dp, dp+n+1, a[i]) - dp;

if ($dp[\text{length}-1] < a[i]$ && $a[i] < dp[\text{length}]$)

$dp[\text{length}] = a[i];$

int lis = 0;

for (int i = 1; i <= n; i++)

if ($dp[i] \neq \text{INT_MAX}$) lis = i;

cout << lis;

We ignore i=0 where INT_MIN is stored

Dry run

for [1, 5, 2, 3, 4, 9, 6, 10]

Condⁿ to ensure $dp[i-1] < a[i] < dp[n]$ is maintained because we want increasing dp array.

if $dp[n] = 5$
 $dp[n+1] = 6$
 and we found upper bound of 5 i.e. 6 but $dp[n] = a[i]$
 initial + replace

* use lower bound for increasing subseq (or we above method with condⁿ)

Cell Mitosis

Scientist has only a single cell. He wants n cells in his container.

He can perform following opⁿ:-

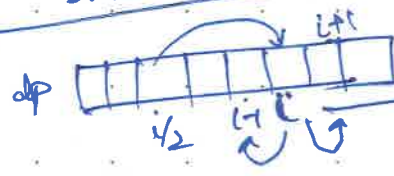
1. Double the no. of cell $\rightarrow$ costs x
2. Inc. by 1 $\rightarrow$ costs y
3. Dec. by 1. $\rightarrow$ costs z

$$f(i) = \min(f(2i), f(i+1), f(i-1))$$

$\begin{matrix} & N & \\ x \swarrow & & \searrow 2 \\ 2N & N+1 & N-1 \end{matrix}$

Tricky part to handle

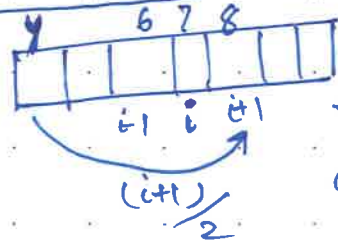
if we apply Bottom Up



To find the value of $dp[i]$ we can check $dp[i/2]$ / $dp[i-1]$

$dp[i+1]$ but it's not found yet.

if i is odd,



To find $dp[7]$ we can't check $dp[i/2]$ because i is odd. So $dp[i-1]$ or $dp[i+1]$ are the only options.

To check the value of $dp[i]$ and update $dp[i] = dp[i-1] + y$ (cost of inc.)

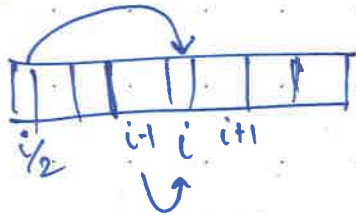
In case of $dp[i+1]$ we don't know its value.
 So we ~~check~~ consider value at half of
 $(i+1)^{th}$ index value of dp . i.e. ~~$dp[i+1]$~~
 $dp[(i+1)/2]$

$$\therefore dp[i] = dp\left[\frac{i+1}{2}\right] + x + z$$

$\swarrow$ cost of increment by double $\searrow$ cost of decrement by 1.

$$\therefore dp[i] = \min \begin{bmatrix} dp[i-1] + y \\ dp\left[\frac{i+1}{2}\right] + x + z \end{bmatrix}$$

if i is even



$$dp[i] = \min \begin{bmatrix} dp[i/2] + x \\ dp[i-1] + y \end{bmatrix}$$

In this $dp[i/2]$ & $dp[i-1]$ would be optimal places to search from. because $dp[i+1]$ will cost a lot.

Code

```

ll solve(n, x, y, z) {
    dp[0] = 0; // To construct 0 & 1 cell => 0 cost
    dp[1] = 0; // already have 1.
    for (int i = 2; i <= n; i++) {
        if (i % 2 == 0) {
            dp[i] = min(dp[i/2] + x, dp[i-1] + y);
        } else {
            // ...
        }
    }
}
    
```

$dp[i] = \min(dp[i-1] + y, dp[(i+1)/2] + x + z)$
 }
 return $dp[n]$;

LES (Bottom Up)

S1 = A B C D
 S2 = A B E D C

$$dp(i, i) = 0 + dp(i-1)$$

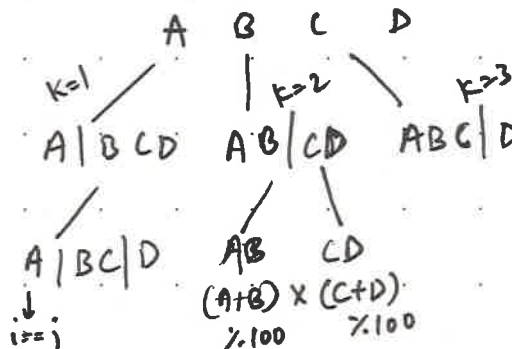
$dp(i, i)$ of state (i, i)

if $dp(i, i)$

* [Code & Exp in Python Notes]

Extra $\Rightarrow$ • LES for 3 strings

Mixtures # spoj.com



(MCM kinda approach of making partition)

$i=j$
 Smoke
 (base case)

9+ goes deeper till the

Top Down

$$f(i, j) = f(i, k) + f(k+1, j) + csum(i, k) * csum(k+1, j)$$

$$csum = \sum a[i] \% 100$$

code

```

any ll solve(i, j, dp[i][100]) {
    if (i >= j) { return 0; }
    if (dp[i][j] != -1) {
        return dp[i][j];
    }
    dp[i][j] = INT_MAX;
    for (int k = i; k <= j; k++) {
        dp[i][j] = min(dp[i][j], solve(i, k, dp) +
            solve(k+1, j, dp) +
            tsum(i, k, dp) * csum(k+1, j, dp));
    }
}

```

ok as return dp[i][j]; }

```

ll csum(s, e) {
    ll ans = 0;
    for (i = s; i <= e; i++) { ans += a[i]; }
    return ans;
}

```

Friend's Pairing Problem

N friends go to party. They can either go solo/ pair. No. of ways?

1, 2, 3

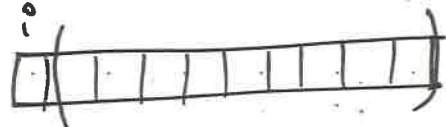
(1)(2)(3)

(1,2)(3)

(1)(2)(3)

(1,3)(2)

4 ways



if i is the first person.

~~(Using Memo/Tabulation)~~
~~approach to solve it~~

Case 1 :- He wants to go solo. (N-1 array)
Case 2 :- He wants to go with someone. (He chooses someone from N-1 array)

$$f(N) = f(N-1) + N-1C_1 + f(N-2)$$

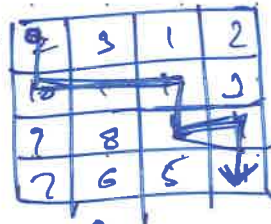
↓ case 1 ↓ He chooses someone from N-1 ↓ N-2 array call

Grid DP. 1 - Minimum Cost Path

Common variants -

- 1) Finding the min/max cost path in a grid.
- 2) finding the no. of ways to reach a particular point from a given starting point in 2D Grid and so on.

Q.

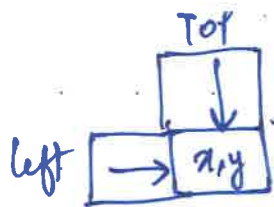


Q Find the path to (m,n) so that cost is min.

Cost 2D array = (R+1, C-1)

Algo.

if we consider a cell for only two ways are there from where the path will come



$$dp[x][y] = \min(dp[x-1][y], dp[x][y-1]) + cost[x][y]$$

(+ x, y)

→ $O(N^2)$

1	5	2
7	1	1
8	1	3

cost

1	6	8
8	7	8
16	8	11

dp

code

```
int min-cost(int R, int C){
```

```
for (0 → R){
```

```
for (0 → C){
```

```
if (i==0 & k==0){
```

```
dp[i][j] = cost[0][0];}
```

```
else if (i==0){
```

```
dp[i][j] = dp[0][j-1] + cost[i][j];}
```

```
else if (j==0){
```

```
dp[i][j] = 3
```

```
else {
```

```
dp[i][j] = min( , ) + cost[i][j];}
```

```
return dp[R-1][C-1];}
```

Croci DP2 - Rat & Elephant - 2D

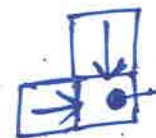
Finding no. of ways to reach a particular position from a given starting point in 2D grid & given :-

- 1) Rat (Right & Down 1 step - 1 cell)
- 2) Elephant in chess (Right & down 1 step - k cell)

① Rat (Bottom-up)

m-1

1	1	1	1
1	2	3	4
1	3	6	10
1	4	10	20



Two ways of coming to this cell:



row case



column case (only 1 way)

$$dp[x][y] = dp[x-1][y] + dp[x][y-1]$$

(+ x & y ≥ 1)

$O(N^2)$

$$(f & c) \Rightarrow \frac{(m-1) + (n-1)}{(m-1)! (n-1)!}$$

if any cell is blocked then $dp[x][y] = 0$ for that case

② Elephant (Bottom-up)

1	2	2	4
1	2	5	12
2			

2,1
1,1,1
3
1,2

4 ways

$$dp[x][y] = \sum_{k=1}^{x-1} dp[k][y] + \sum_{k=0}^{y-1} dp[x][k] + x, y$$

$O(N^2 \cdot N) = O(N^3)$
 whole grid $\rightarrow$ row & column seq for each cell

(2D DP problem of ladder problem)

Q DP-3. | Robots & Paths $\rightarrow$ Codechef

[Code: -CD1IT4]

src		
X		X
		Dest

Find no. of ways to reach from src to dest.
 If some of cells are blocked.

(Src & dest its can be blocked point or in that case)

(Bottom up)

1	1	1
1	2	3
X	2	X
0	2	2

Set (first row) & (first col) cells 1 until we get a blockage. (after blockage ignore)

if upper cell is blocked or not) $dp[i][j] = dp[i-1][j] \% Mod;$

if $(dp[i][j-1] / e-1) \neq 0$
 $dp[i][j] = (dp[i][j-1] + dp[i-1][j]) \% Mod;$
 if left cell is also not blocked then through first if condⁿ for upper cell we would have updated $dp[i][j]$ so we add this with left cells value directly.

[Code done on codechef]

MCM (Top down)

```
int dp[100][100];
int MCM(int *p, int i, int j) {
    if (i >= j) return 0;
    if (dp[i][j] != -1) return dp[i][j];
    dp[i][j] = INT_MAX;
    for (int k = i; k < j; k++) {
        dp[i][j] = min(
            dp[i][k], MCM(p, k+1, j)
            + p[i-1] * p[k] * p[j]
        );
    }
    return dp[i][j];
}
```

$p = \{1, 2, 3, 4\}$

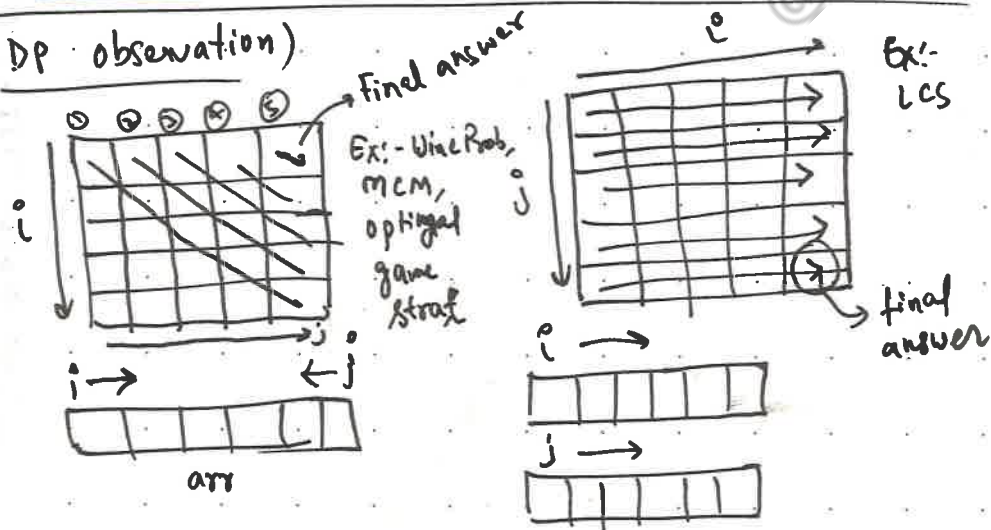
A ₁	A ₂	A ₃
1x2	2x3	3x4
P ₀	P ₁	P ₂ P ₃

Time complexity of loop

- 1) $O(1) \rightarrow$ // set of non-recursive & non-loop statements
A loop or recursion that runs a constant no. of times is also considered $O(1)$.
for ($i=1; i \leq c; i++$) { // $O(1) op^n$ }
- 2) $O(n) \rightarrow$ for ($0 \rightarrow n$) { } , for ($n \rightarrow 0$) { }
- 3) $O(n^c) \rightarrow$ for ($1 \rightarrow n$) { for ($1 \rightarrow n$) { ... c times } }
- 4) $O(\log n) \rightarrow$ if loop variables is divided/multiplied
 • for ($i=1; i \leq n; i *= c$) { $O(1) op^n$ }
 • for ($i=n; i > 0; i /= c$) { $O(1) op^n$ }
 $\frac{n}{2} + \frac{n}{4} + \dots$
- 5) $O(\log \log n) \rightarrow$ if loop variables is reduced/increased exponentially by a constant amount.
 • for ($i=2; i \leq n; i = \text{pow}(i, c)$) { $O(1) op^n$ }
 • for ($i=n; i > 1; i = \text{fun}(i)$) { }

Prints $\frac{n}{2} + \frac{n}{3} + \frac{n}{5} + \frac{n}{7} + \dots$ $\downarrow$ sum/cuberoot

(DP observation)



COMPILE TIME - INTERACTIVE PROBLEMS

```
int main() {
    ios-base :: sync_with_stdio (false);
    cin.tie (NULL); cout.tie (NULL); // I/P: -
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        cout << "n is - " << n << '\n';
    }
```

I/P: -
3
1
2
3
O/P: - n is - 1
n is - 2
n is - 3

Same code without this 2 lines

I/P: -
3
1
n is - 1
2
n is - 2
3
n is - 3

So those two lines stores outputs in a buffer and pile up and print at last after all I/Ps have been taken.
(Compiler doesn't have to give I/P & take output over & over, saves a lot of time.)

In this after cout << if we cout << flush; we get O/P of second case. Why coz it flushes each output after cout.
endl = '\n' << flush; [endl moves to next line and flushes the buffered O/P at the same time]
 $\therefore$ '\n' is faster than endl.